

2 Small Worlds and Large Worlds

When Cristoforo Colombo (Christopher Columbus) infamously sailed west in the year 1492, he believed that the Earth was spherical. In this, he was like most educated people of his day. He was unlike most people, though, in that he also believed the planet was much smaller than it actually is—only 30,000 km around its middle instead of the actual 40,000 km (FIGURE 2.1).³⁵ This was one of the most consequential mistakes in European history. If Colombo had believed instead that the Earth was 40,000 km around, he would have correctly reasoned that his fleet could not carry enough food and potable water to complete a journey all the way westward to Asia. But at 30,000 km around, Asia would lie a bit west of the coast of California. It was possible to carry enough supplies to make it that far. Emboldened in part by his unconventional estimate, Colombo set sail, eventually making landfall in the Bahamas.

Colombo made a prediction based upon his view that the world was small. But since he lived in a large world, aspects of the prediction were wrong. In his case, the error was lucky. His small world model was wrong in an unanticipated way: There was a lot of land in the way. If he had been wrong in the expected way, with nothing but ocean between Europe and Asia, he and his entire expedition would have run out of supplies long before reaching the East Indies.

Colombo's small and large worlds provide a contrast between model and reality. All statistical modeling has these same two frames: the *small world* of the model itself and the *large world* we hope to deploy the model in.³⁶ Navigating between these two worlds remains a central challenge of statistical modeling. The challenge is aggravated by forgetting the distinction.

The **SMALL WORLD** is the self-contained logical world of the model. Within the small world, all possibilities are nominated. There are no pure surprises, like the existence of a huge continent between Europe and Asia. Within the small world of the model, it is important to be able to verify the model's logic, making sure that it performs as expected under favorable assumptions. Bayesian models have some advantages in this regard, as they have reasonable claims to optimality: No alternative model could make better use of the information in the data and support better decisions, assuming the small world is an accurate description of the real world.³⁷

The **LARGE WORLD** is the broader context in which one deploys a model. In the large world, there may be events that were not imagined in the small world. Moreover, the model is always an incomplete representation of the large world, and so will make mistakes, even if all kinds of events have been properly nominated. The logical consistency of a model in the small world is no guarantee that it will be optimal in the large world. But it is certainly a warm comfort.



FIGURE 2.1. Illustration of Martin Behaim's 1492 globe, showing the small world that Colombo anticipated. Europe lies on the right-hand side. Asia lies on the left. The big island labeled "Cipangu" is Japan.

In this chapter, you will begin to build Bayesian models. The way that Bayesian models learn from evidence is arguably optimal in the small world. When their assumptions approximate reality, they also perform well in the large world. But large world performance has to be demonstrated rather than logically deduced. Passing back and forth between these two worlds allows both formal methods, like Bayesian inference, and informal methods, like peer review, to play an indispensable role.

This chapter focuses on the small world. It explains probability theory in its essential form: counting the ways things can happen. Bayesian inference arises automatically from this perspective. Then the chapter presents the stylized components of a Bayesian statistical model, a model for learning from data. Then it shows you how to animate the model, to produce estimates.

All this work provides a foundation for the next chapter, in which you'll learn to summarize Bayesian estimates, as well as begin to consider large world obligations.

Rethinking: Fast and frugal in the large world. The natural world is complex, as trying to do science serves to remind us. Yet everything from the humble tick to the industrious squirrel to the idle sloth manages to frequently make adaptive decisions. But it's a good bet that most animals are not Bayesian, if only because being Bayesian is expensive and depends upon having a good model. Instead, animals use various heuristics that are fit to their environments, past or present. These heuristics take adaptive shortcuts and so may outperform a rigorous Bayesian analysis, once costs of information gathering and processing (and overfitting, Chapter 6) are taken into account.³⁸ Once you already know which information to ignore or attend to, being fully Bayesian is a waste. It's neither necessary nor sufficient for making good decisions, as real animals demonstrate. But for human animals, Bayesian analysis provides a general way to discover relevant information and process it logically. Just don't think that it is the only way.

2.1. The garden of forking data

Our goal in this section will be to build Bayesian inference up from humble beginnings, so there is no superstition about it. Bayesian inference is really just counting and comparing of possibilities. Consider by analogy Jorge Luis Borges' short story "The Garden of Forking Paths." The story is about a man who encounters a book filled with contradictions. In most books, characters arrive at plot points and must decide among alternative paths. A protagonist may arrive at a man's home. She might kill the man, or rather take a cup of tea. Only

one of these paths is taken—murder or tea. But the book within Borges’ story explores all paths, with each decision branching outward into an expanding garden of forking paths.

This is the same device that Bayesian inference offers. In order to make good inference about what actually happened, it helps to consider everything that could have happened. A Bayesian analysis is a garden of forking data, in which alternative sequences of events are cultivated. As we learn about what did happen, some of these alternative sequences are pruned. In the end, what remains is only what is logically consistent with our knowledge.

This approach provides a quantitative ranking of hypotheses, a ranking that is maximally conservative, given the assumptions and data that go into it. The approach cannot guarantee a correct answer, on large world terms. But it can guarantee the best possible answer, on small world terms, that could be derived from the information fed into it.

Consider the following toy example.

2.1.1. Counting possibilities. Suppose there’s a bag, and it contains four marbles. These marbles come in two colors: blue and white. We know there are four marbles in the bag, but we don’t know how many are of each color. We do know that there are five possibilities: (1) [○○○○], (2) [●○○○], (3) [●●○○], (4) [●●●○], (5) [●●●●]. These are the only possibilities consistent with what we know about the contents of the bag. Call these five possibilities the *conjectures*.

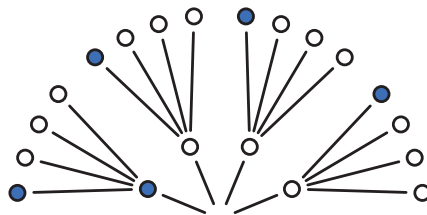
Our goal is to figure out which of these conjectures is most plausible, given some evidence about the contents of the bag. We do have some evidence: A sequence of three marbles is pulled from the bag, one at a time, replacing the marble each time and shaking the bag before drawing another marble. The sequence that emerges is: ●○○, in that order. These are the data.

So now let’s plant the garden and see how to use the data to infer what’s in the bag. Let’s begin by considering just the single conjecture, [●○○○], that the bag contains one blue and three white marbles. On the first draw from the bag, one of four things could happen, corresponding to one of four marbles in the bag. So we can visualize the possibilities branching outward:



Notice that even though the three white marbles look the same from a data perspective—we just record the color of the marbles, after all—they are really different events. This is important, because it means that there are three more ways to see ○ than to see ●.

Now consider the garden as we get another draw from the bag. It expands the garden out one layer:



Now there are 16 possible paths through the garden, one for each pair of draws. On the second draw from the bag, each of the paths above again forks into four possible paths. Why?

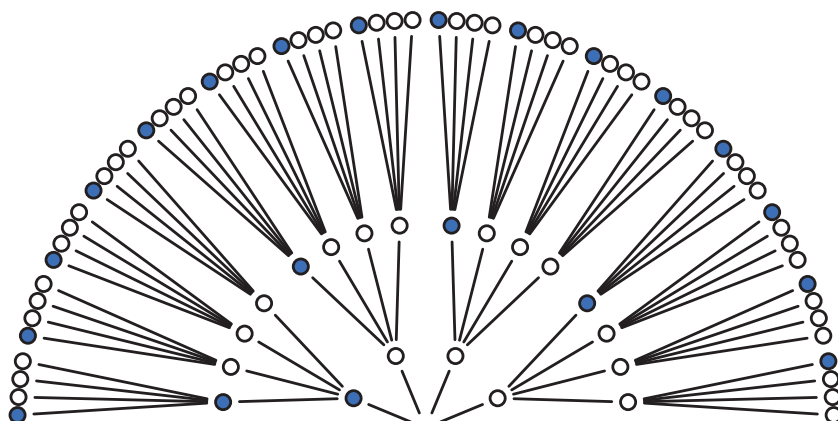


FIGURE 2.2. The 64 possible paths generated by assuming the bag contains one blue and three white marbles.

Because we believe that our shaking of the bag gives each marble a fair chance at being drawn, regardless of which marble was drawn previously. The third layer is built in the same way, and the full garden is shown in FIGURE 2.2. There are $4^3 = 64$ possible paths in total.

As we consider each draw from the bag, some of these paths are logically eliminated. The first draw turned out to be ●, recall, so the three white paths at the bottom of the garden are eliminated right away. If you imagine the real data tracing out a path through the garden, it must have passed through the one blue path near the origin. The second draw from the bag produces ○, so three of the paths forking out of the first blue marble remain. As the data trace out a path, we know it must have passed through one of those three white paths (after the first blue path), but we don't know which one, because we recorded only the color of each marble. Finally, the third draw is ●. Each of the remaining three paths in the middle layer sustain one blue path, leaving a total of three ways for the sequence ●○○ to appear, assuming the bag contains [●○○○]. FIGURE 2.3 shows the garden again, now with logically eliminated paths grayed out. We can't be sure which of those three paths the actual data took. But as long as we're considering only the possibility that the bag contains one blue and three white marbles, we can be sure that the data took one of those three paths. Those are the only paths consistent with both our knowledge of the bag's contents (four marbles, white or blue) and the data (●○○).

This demonstrates that there are three (out of 64) ways for a bag containing [●○○○] to produce the data ●○○. We have no way to decide among these three ways. The inferential power comes from comparing this count to the numbers of ways each of the other conjectures of the bag's contents could produce the same data. For example, consider the conjecture [○○○○]. There are zero ways for this conjecture to produce the observed data, because even one ● is logically incompatible with it. The conjecture [●●●●] is likewise logically incompatible with the data. So we can eliminate these two conjectures, because neither provides even a single path that is consistent with the data.

FIGURE 2.4 displays the full garden now, for the remaining three conjectures: [●○○○], [●●○○], and [●●●○]. The upper-left wedge displays the same garden as FIGURE 2.3. The upper-right shows the analogous garden for the conjecture that the bag contains three blue marbles and one white marble. And the bottom wedge shows the garden for two blue

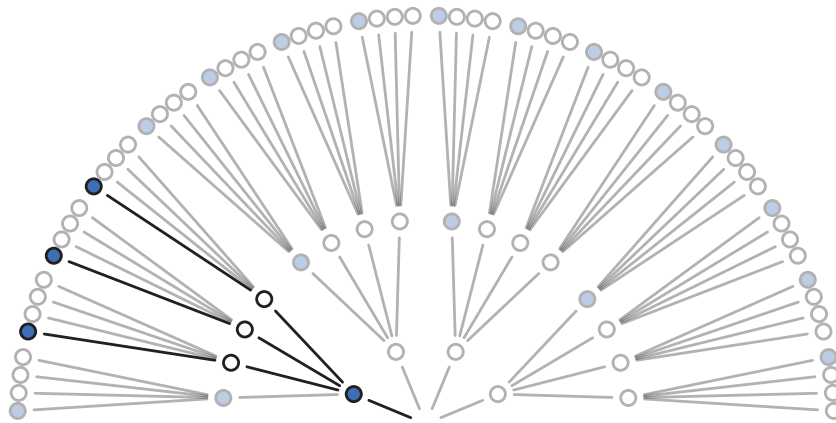


FIGURE 2.3. After eliminating paths inconsistent with the observed sequence, only 3 of the 64 paths remain.

and two white marbles. Now we count up all of the ways each conjecture could produce the observed data. For one blue and three white, there are three ways, as we counted already. For two blue and two white, there are eight paths forking through the garden that are logically consistent with the observed sequence. For three blue and one white, there are nine paths that survive.

To summarize, we've considered five different conjectures about the contents of the bag, ranging from zero blue marbles to four blue marbles. For each of these conjectures, we've counted up how many sequences, paths through the garden of forking data, could potentially produce the observed data, $\bullet\circ\bullet$:

Conjecture	Ways to produce $\bullet\circ\bullet$
$[\circ\circ\circ\circ]$	$0 \times 4 \times 0 = 0$
$[\bullet\circ\circ\circ]$	$1 \times 3 \times 1 = 3$
$[\bullet\bullet\circ\circ]$	$2 \times 2 \times 2 = 8$
$[\bullet\bullet\bullet\circ]$	$3 \times 1 \times 3 = 9$
$[\bullet\bullet\bullet\bullet]$	$4 \times 0 \times 4 = 0$

Notice that the number of ways to produce the data, for each conjecture, can be computed by first counting the number of paths in each “ring” of the garden and then by multiplying these counts together. This is just a computational device. It tells us the same thing as [FIGURE 2.4](#), but without having to draw the garden. The fact that numbers are multiplied during calculation doesn't change the fact that this is still just counting of logically possible paths. This point will come up again, when you meet the more formal representation of Bayesian inference.

So what good are these counts? By comparing these counts, we have part of a solution for a way to rate the relative plausibility of each conjectured bag composition. But it's only a part of a solution, because in order to compare these counts we first have to decide how many ways each conjecture could itself be realized. We might argue that when we have no reason to assume otherwise, we can just consider each conjecture equally plausible and compare the counts directly. But often we do have reason to assume otherwise.

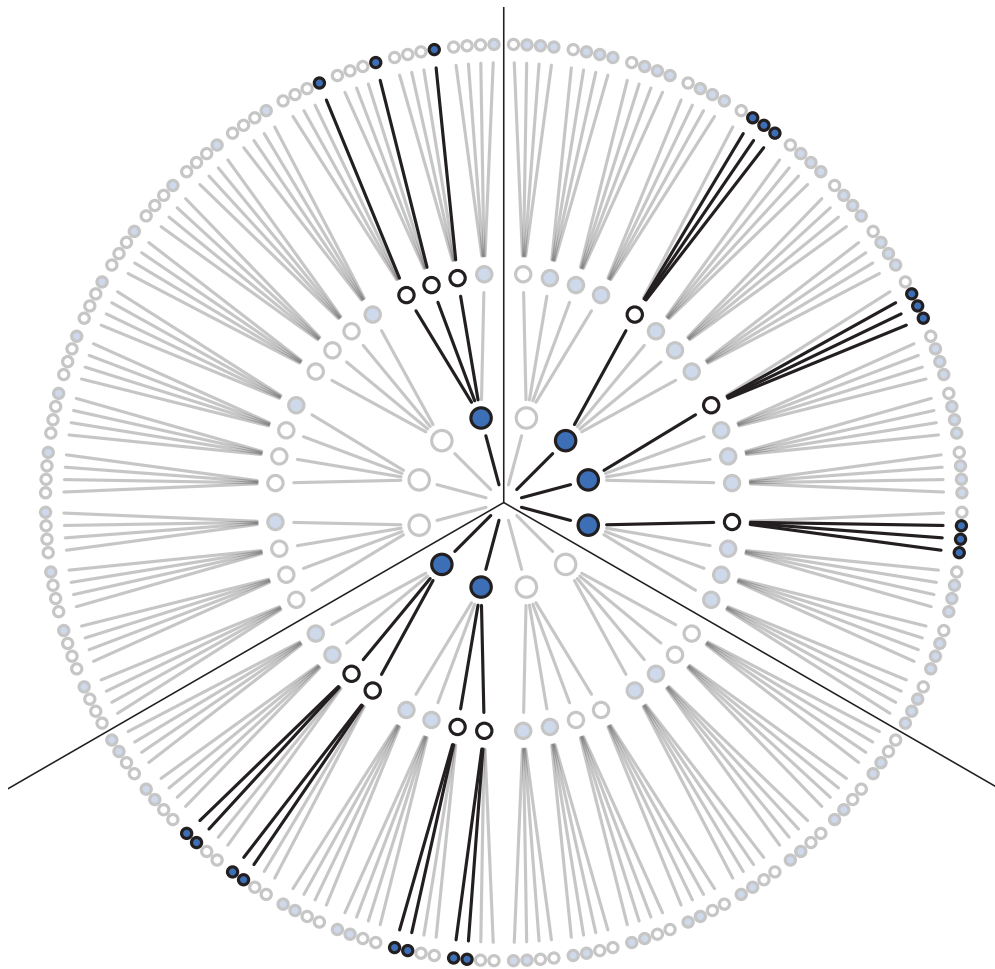


FIGURE 2.4. The garden of forking data, showing for each possible composition of the bag the forking paths that are logically compatible with the data.

Rethinking: Justification. Using these counts of paths through the garden as measures of relative plausibility can be justified in several ways. The justification here is logical: If we wish to reason about plausibility and remain consistent with ordinary logic—statements about *true* and *false*—then we should obey this procedure.³⁹ There are several other justifications that lead to the same mathematical procedure. Regardless of how you choose to philosophically justify it, notice that it actually works. Justifications and philosophy motivate procedures, but it is the results that matter. The many successful real world applications of Bayesian inference may be all the justification you need. Twentieth century opponents of Bayesian data analysis argued that Bayesian inference was easy to justify, but hard to apply.⁴⁰ That is luckily no longer true. Just be careful not to assume that because Bayesian inference is justified that no other approach can also be justified. Golems come in many types, and some of all types are useful.

2.1.2. Using prior information. We may have prior information about the relative plausibility of each conjecture. This prior information could arise from knowledge of how the contents of the bag were generated. It could also arise from previous data. Or we might want to act as if we had prior information, so we can build conservatism into the analysis. Whatever the source, it would help to have a way to use prior information. Luckily there is a natural solution: Just multiply the prior count by the new count.

To grasp this solution, suppose we're willing to say each conjecture is equally plausible at the start. So we just compare the counts of ways in which each conjecture is compatible with the observed data. This comparison suggests that $[\bullet\bullet\bullet\circ]$ is slightly more plausible than $[\bullet\bullet\circ\circ]$, and both are about three times more plausible than $[\bullet\circ\circ\circ]$.

Now suppose we draw another marble from the bag to get another observation: $\bullet$. Now you have two choices. You could start all over again, making a garden with four layers to trace out the paths compatible with the data sequence $\bullet\circ\bullet\bullet$. Or you could take the previous counts over conjectures (0, 3, 8, 9, 0) and just update them in light of the new observation. It turns out that these two methods are mathematically identical, as long as the new observation is logically independent of the previous observations.

Here's how to do it. First we count the numbers of ways each conjecture could produce the new observation, $\bullet$. Then we multiply each of these new counts by the previous numbers of ways for each conjecture. In table form:

Conjecture	Ways to produce $\bullet$	Previous counts	New count
$[\circ\circ\circ\circ]$	0	0	$0 \times 0 = 0$
$[\bullet\circ\circ\circ]$	1	3	$3 \times 1 = 3$
$[\bullet\bullet\circ\circ]$	2	8	$8 \times 2 = 16$
$[\bullet\bullet\bullet\circ]$	3	9	$9 \times 3 = 27$
$[\bullet\bullet\bullet\bullet]$	4	0	$0 \times 4 = 0$

The new counts in the right-hand column above summarize all the evidence for each conjecture. As new data arrive, and provided those data are independent of previous observations, then the number of logically possible ways for a conjecture to produce all the data up to that point can be computed just by multiplying the new count by the old count.

This updating approach amounts to nothing more than asserting that (1) when we have previous information suggesting there are W_{prior} ways for a conjecture to produce a previous observation D_{prior} and (2) we acquire new observations D_{new} that the same conjecture can produce in W_{new} ways, then (3) the number of ways the conjecture can account for both D_{prior} as well as D_{new} is just the product $W_{\text{prior}} \times W_{\text{new}}$. For example, in the table above the conjecture $[\bullet\bullet\circ\circ]$ has $W_{\text{prior}} = 8$ ways to produce $D_{\text{prior}} = \bullet\circ\bullet$. It also has $W_{\text{new}} = 2$ ways to produce the new observation $D_{\text{new}} = \bullet$. So there are $8 \times 2 = 16$ ways for the conjecture to produce both D_{prior} and D_{new} . Why multiply? Multiplication is just a shortcut to enumerating and counting up all of the paths through the garden that could produce all the observations.

In this example, the prior data and new data are of the same type: marbles drawn from the bag. But in general, the prior data and new data can be of different types. Suppose for example that someone from the marble factory tells you that blue marbles are rare. So for every bag containing $[\bullet\bullet\bullet\circ]$, they made two bags containing $[\bullet\bullet\circ\circ]$ and three bags containing $[\bullet\circ\circ\circ]$. They also ensured that every bag contained at least one blue and one white marble. We can update our counts again:

Conjecture	Prior count	Factory count	New count
[○○○○]	0	0	$0 \times 0 = 0$
[●○○○]	3	3	$3 \times 3 = 9$
[●●○○]	16	2	$16 \times 2 = 32$
[●●●○]	27	1	$27 \times 1 = 27$
[●●●●]	0	0	$0 \times 0 = 0$

Now the conjecture [●●○○] is most plausible, but barely better than [●●●○]. Is there a threshold difference in these counts at which we can safely decide that one of the conjectures is the correct one? You'll spend the next chapter exploring that question.

Rethinking: Original ignorance. Which assumption should we use, when there is no previous information about the conjectures? The most common solution is to assign an equal number of ways that each conjecture could be correct, before seeing any data. This is sometimes known as the **PRINCIPLE OF INDIFFERENCE**: When there is no reason to say that one conjecture is more plausible than another, weigh all of the conjectures equally. The issue of choosing a representation of “ignorance” is surprisingly complicated. The issue will arise again in later chapters. For the sort of problems we examine in this book, the principle of indifference results in inferences very comparable to mainstream non-Bayesian approaches, most of which contain implicit equal weighting of possibilities. For example a typical non-Bayesian confidence interval weighs equally all of the possible values a parameter could take, regardless of how implausible some of them are. Many non-Bayesian procedures have moved away from this, through the use of penalized likelihood and other methods. We'll discuss these in Chapter 6.

2.1.3. From counts to probability. It is helpful to think of this strategy as adhering to a principle of honest ignorance: *When we don't know what caused the data, potential causes that may produce the data in more ways are more plausible.* This leads us to count paths through the garden of forking data.

It's hard to use these counts though, so we almost always standardize them in a way that transforms them into probabilities. Why is it hard to work with the counts? First, since relative value is all that matters, the size of the counts 3, 8, and 9 contain no information of value. They could just as easily be 30, 80, and 90. The meaning would be the same. It's just the relative values that matter. Second, as the amount of data grows, the counts will very quickly grow very large and become difficult to manipulate. By the time we have 10 data points, there are already more than one million possible sequences. We'll want to analyze data sets with thousands of observations, so explicitly counting these things isn't practical.

Luckily, there's a mathematical way to compress all of this. Specifically, we define the updated plausibility of each possible composition of the bag, after seeing the data, as:

$$\begin{aligned}
 &\text{plausibility of } [\bullet \circ \circ \circ] \text{ after seeing } \bullet \circ \bullet \\
 &\quad \propto \\
 &\quad \text{ways } [\bullet \circ \circ \circ] \text{ can produce } \bullet \circ \bullet \\
 &\quad \times \\
 &\quad \text{prior plausibility } [\bullet \circ \circ \circ]
 \end{aligned}$$

That little $\propto$ means *proportional to*. We want to compare the plausibility of each possible bag composition. So it'll be helpful to define p as the proportion of marbles that are blue. For [

●○○○], $p = 1/4 = 0.25$. Also let $D_{\text{new}} = \bullet\circ\bullet$. And now we can write:

plausibility of p after $D_{\text{new}} \propto$ ways p can produce $D_{\text{new}} \times$ prior plausibility of p

The above just means that for any value p can take, we judge the plausibility of that value p as proportional to the number of ways it can get through the garden of forking data. This expression just summarizes the calculations you did in the tables of the previous section.

Finally, we construct probabilities by standardizing the plausibility so that the sum of the plausibilities for all possible conjectures will be one. All you need to do in order to standardize is to add up all of the products, one for each value p can take, and then divide each product by the sum of products:

$$\text{plausibility of } p \text{ after } D_{\text{new}} = \frac{\text{ways } p \text{ can produce } D_{\text{new}} \times \text{prior plausibility } p}{\text{sum of products}}$$

A worked example is needed for this to really make sense. So consider again the table from before, now updated using our definitions of p and “plausibility”:

Possible composition	p	Ways to produce data	Plausibility
[○○○○]	0	0	0
[●○○○]	0.25	3	0.15
[●●○○]	0.5	8	0.40
[●●●○]	0.75	9	0.45
[●●●●]	1	0	0

You can quickly compute these plausibilities in R:

```
ways <- c( 0 , 3 , 8 , 9 , 0 )
ways/sum(ways)
```

R code
2.1

```
[1] 0.00 0.15 0.40 0.45 0.00
```

The values in `ways` are the products mentioned before. And `sum(ways)` is the denominator “sum of products” in the expression near the top of the page.

These plausibilities are also *probabilities*—they are non-negative (zero or positive) real numbers that sum to one. And all of the mathematical things you can do with probabilities you can also do with these values. Specifically, each piece of the calculation has a direct partner in applied probability theory. These partners have stereotyped names, so it’s worth learning them, as you’ll see them again and again.

- A conjectured proportion of blue marbles, p , is usually called a **PARAMETER** value. It’s just a way of indexing possible explanations of the data.
- The relative number of ways that a value p can produce the data is usually called a **LIKELIHOOD**. It is derived by enumerating all the possible data sequences that could have happened and then eliminating those sequences inconsistent with the data.
- The prior plausibility of any specific p is usually called the **PRIOR PROBABILITY**.
- The new, updated plausibility of any specific p is usually called the **POSTERIOR PROBABILITY**.

In the next major section, you’ll meet the more formal notation for these objects and see how they compose a simple statistical model.

Rethinking: Randomization. When you shuffle a deck of cards or assign subjects to treatments by flipping a coin, it is common to say that the resulting deck and treatment assignments are *randomized*. What does it mean to randomize something? It just means that we have processed the thing so that we know almost nothing about its arrangement. Shuffling a deck of cards changes our state of knowledge, so that we no longer have any specific information about the ordering of cards. However, the bonus that arises from this is that, if we really have shuffled enough to erase any prior knowledge of the ordering, then the order the cards end up in is very likely to be one of the many orderings with high **INFORMATION ENTROPY**. The concept of information entropy will be increasingly important as we progress, and will be unpacked in Chapters 6 and 9.

2.2. Building a model

By working with probabilities instead of raw counts, Bayesian inference is made much easier, but it looks much harder. So in this section, we follow up on the garden of forking data by presenting the conventional form of a Bayesian statistical model. The toy example we'll use here has the anatomy of a typical statistical analysis, so it's the style that you'll grow accustomed to. But every piece of it can be mapped onto the garden of forking data. The logic is the same.

Suppose you have a globe representing our planet, the Earth. This version of the world is small enough to hold in your hands. You are curious how much of the surface is covered in water. You adopt the following strategy: You will toss the globe up in the air. When you catch it, you will record whether or not the surface under your right index finger is water or land. Then you toss the globe up in the air again and repeat the procedure.⁴¹ This strategy generates a sequence of surface samples from the globe. The first nine samples might look like:

W L W W W L W L W

where W indicates water and L indicates land. So in this example you observe six W (water) observations and three L (land) observations. Call this sequence of observations the *data*.

To get the logic moving, we need to make assumptions, and these assumptions constitute the model. Designing a simple Bayesian model benefits from a design loop with three steps.

- (1) Data story: Motivate the model by narrating how the data might arise.
- (2) Update: Educate your model by feeding it the data.
- (3) Evaluate: All statistical models require supervision, leading possibly to model revision.

The next sections walk through these steps, in the context of the globe tossing evidence.

2.2.1. A data story. Bayesian data analysis usually means producing a story for how the data came to be. This story may be *descriptive*, specifying associations that can be used to predict outcomes, given observations. Or it may be *causal*, a theory of how some events produce other events. Typically, any story you intend to be causal may also be descriptive. But many descriptive stories are hard to interpret causally. But all data stories are complete, in the sense that they are sufficient for specifying an algorithm for simulating new data. In the next chapter, you'll see examples of doing just that, as simulating new data is a useful part of model criticism.

You can motivate your data story by trying to explain how each piece of data is born. This usually means describing aspects of the underlying reality as well as the sampling process. The data story in this case is simply a restatement of the sampling process:

- (1) The true proportion of water covering the globe is p .
- (2) A single toss of the globe has a probability p of producing a water (W) observation. It has a probability $1 - p$ of producing a land (L) observation.
- (3) Each toss of the globe is independent of the others.

The data story is then translated into a formal probability model. This probability model is easy to build, because the construction process can be usefully broken down into a series of component decisions. Before meeting these components, however, it'll be useful to visualize how a Bayesian model behaves. After you've become acquainted with how such a model learns from data, we'll pop the machine open and investigate its engineering.

Rethinking: The value of storytelling. The data story has value, even if you quickly abandon it and never use it to build a model or simulate new observations. Indeed, it is important to eventually discard the story, because many different stories always correspond to the same model. As a result, showing that a model does a good job does not in turn uniquely support our data story. Still, the story has value because in trying to outline the story, often one realizes that additional questions must be answered. Most data stories are much more specific than are the verbal hypotheses that inspire data collection. Hypotheses can be vague, such as “it's more likely to rain on warm days.” When you are forced to consider sampling and measurement and make a precise statement of how temperature predicts rain, many stories and resulting models will be consistent with the same vague hypothesis. Resolving that ambiguity often leads to important realizations and model revisions, before any model is fit to data.

2.2.2. Bayesian updating. Our problem is one of using the evidence—the sequence of globe tosses—to decide among different possible proportions of water on the globe. These proportions are like the conjectured marbles inside the bag, from earlier in the chapter. Each possible proportion may be more or less plausible, given the evidence. A Bayesian model begins with one set of plausibilities assigned to each of these possibilities. These are the prior plausibilities. Then it updates them in light of the data, to produce the posterior plausibilities. This updating process is a kind of learning, called **BAYESIAN UPDATING**. The details of this updating—how it is mechanically achieved—can wait until later in the chapter. For now, let's look only at how such a machine behaves.

For the sake of the example only, let's program our Bayesian machine to initially assign the same plausibility to every proportion of water, every value of p . Now look at the top-left plot in **FIGURE 2.5**. The dashed horizontal line represents this initial plausibility of each possible value of p . After seeing the first toss, which is a “W,” the model updates the plausibilities to the solid line. The plausibility of $p = 0$ has now fallen to exactly zero—the equivalent of “impossible.” Why? Because we observed at least one speck of water on the globe, so now we know there is *some* water. The model executes this logic automatically. You don't have it instruct it to account for this consequence. Probability theory takes care of it for you, because it is essentially counting paths through the garden of forking data, as in the previous section.

Likewise, the plausibility of $p > 0.5$ has increased. This is because there is not yet any evidence that there is land on the globe, so the initial plausibilities are modified to be consistent with this. Note however that the relative plausibilities are what matter, and there isn't

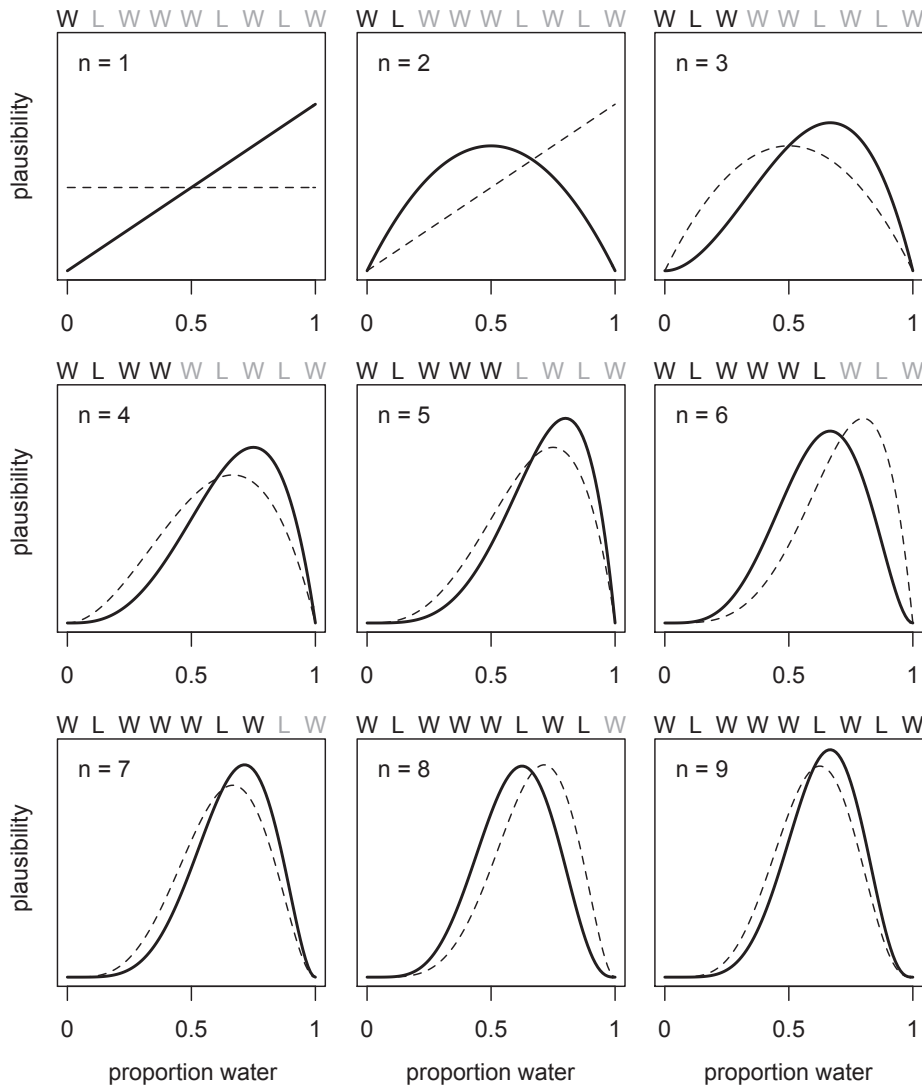


FIGURE 2.5. How a Bayesian model learns. Each toss of the globe produces an observation of water (W) or land (L). The model's estimate of the proportion of water on the globe is a plausibility for every possible value. The lines and curves in this figure are these collections of plausibilities. In each plot, previous plausibilities (dashed curve) are updated in light of the latest observation to produce a new set of plausibilities (solid curve).

yet much evidence. So the differences in plausibility are not yet very large. In this way, the amount of evidence seen so far is embodied in the plausibilities of each value of p .

In the remaining plots in [FIGURE 2.5](#), the additional samples from the globe are introduced to the model, one at a time. Each dashed curve is just the solid curve from the previous plot, moving left to right and top to bottom. Every time a "W" is seen, the peak of the plausibility curve moves to the right, towards larger values of p . Every time an "L" is seen, it moves

the other direction. The maximum height of the curve increases with each sample, meaning that fewer values of p amass more plausibility as the amount of evidence increases. As each new observation is added, the curve is updated consistent with all previous observations.

Notice that every updated set of plausibilities becomes the initial plausibilities for the next observation. Every conclusion is the starting point for future inference. However, this updating process works backwards, as well as forwards. Given the final set of plausibilities in the bottom-right plot of [FIGURE 2.5](#), and knowing the final observation (W), it is possible to mathematically divide out the observation, to infer the previous plausibility curve. So the data could be presented to your model in any order, or all at once even. In most cases, you will present the data all at once, for the sake of convenience. But it's important to realize that this merely represents abbreviation of an iterated learning process.

Rethinking: Sample size and reliable inference. It is common to hear that there is a minimum number of observations for a useful statistical estimate. For example, there is a widespread superstition that 30 observations are needed before one can use a Gaussian distribution. Why? In non-Bayesian statistical inference, procedures are often justified by the method's behavior at very large sample sizes, so-called *asymptotic* behavior. As a result, performance at small samples sizes is questionable.

In contrast, Bayesian estimates are valid for any sample size. This does not mean that more data isn't helpful—it certainly is. Rather, the estimates have a clear and valid interpretation, no matter the sample size. But the price for this power is dependency upon the initial estimates, the prior. If the prior is a bad one, then the resulting inference will be misleading. There's no free lunch,⁴² when it comes to learning about the world. A Bayesian golem must choose an initial plausibility, and a non-Bayesian golem must choose an estimator. Both golems pay for lunch with their assumptions.

2.2.3. Evaluate. The Bayesian model learns in a way that is demonstrably optimal, provided that the real, large world is accurately described by the model. This is to say that your Bayesian machine guarantees perfect inference, within the small world. No other way of using the available information, and beginning with the same state of information, could do better.

Don't get too excited about this logical virtue, however. The calculations may malfunction, so results always have to be checked. And if there are important differences between the model and reality, then there is no logical guarantee of large world performance. And even if the two worlds did match, any particular sample of data could still be misleading. So it's worth keeping in mind at least two cautious principles.

First, the model's certainty is no guarantee that the model is a good one. As the amount of data increases, the globe tossing model will grow increasingly sure of the proportion of water. This means that the curves in [FIGURE 2.5](#) will become increasingly narrow and tall, restricting plausible values within a very narrow range. But models of all sorts—Bayesian or not—can be very confident about an estimate, even when the model is seriously misleading. This is because the estimates are conditional on the model. What your model is telling you is that, given a commitment to this particular model, it can be very sure that the plausible values are in a narrow range. Under a different model, things might look differently.

Second, it is important to supervise and critique your model's work. Consider again the fact that the updating in the previous section works in any order of data arrival. We could shuffle the order of the observations, as long as six W 's and three L 's remain, and still end up with the same final plausibility curve. That is only true, however, because the model assumes that order is irrelevant to inference. When something is irrelevant to the machine, it won't

affect the inference directly. But it may affect it indirectly, because the data will depend upon order. So it is important to check the model's inferences in light of aspects of the data it does not know about. Such checks are an inherently creative enterprise, left to the analyst and the scientific community. Golems are very bad at it.

In Chapter 3, you'll see some examples of such checks. For now, note that the goal is not to test the truth value of the model's assumptions. We know the model's assumptions are never exactly right, in the sense of matching the true data generating process. Therefore there's no point in checking if the model is true. Failure to conclude that a model is false must be a failure of our imagination, not a success of the model. Moreover, models do not need to be exactly true in order to produce highly precise and useful inferences. All manner of small world assumptions about error distributions and the like can be violated in the large world, but a model may still produce a perfectly useful estimate. This is because models are essentially information processing machines, and there are some surprising aspects of information that cannot be easily captured by framing the problem in terms of the truth of assumptions.⁴³

Instead, the objective is to check the model's adequacy for some purpose. This usually means asking and answering additional questions, beyond those that originally constructed the model. Both the questions and answers will depend upon the scientific context. So it's hard to provide general advice. There will be many examples, throughout the book, and of course the scientific literature is replete with evaluations of the suitability of models for different jobs—prediction, comprehension, measurement, and persuasion.

Rethinking: Deflationary statistics. It may be that Bayesian inference is the best general purpose method of inference known. However, Bayesian inference is much less powerful than we'd like it to be. There is no approach to inference that provides universal guarantees. No branch of applied mathematics has unfettered access to reality, because math is not discovered, like the proton. Instead it is invented, like the shovel.⁴⁴

2.3. Components of the model

Now that you've seen how the Bayesian model behaves, it's time to open up the machine and learn how it works. Consider three different kinds of things we counted in the previous sections.

- (1) The number of ways each conjecture could produce an observation
- (2) The accumulated number of ways each conjecture could produce the entire data
- (3) The initial plausibility of each conjectured cause of the data

Each of these things has a direct analog in conventional probability theory. And so the usual way we build a statistical model involves choosing distributions and devices for each that represent the relative numbers of ways things can happen.

These distributions and devices are (1) a likelihood function, (2) one or more parameters, and (3) a prior. These components are usually chosen in that order. In this section, you'll meet these components in some detail and see how each relates to the counting you did earlier in the chapter.

2.3.1. Likelihood. The first and most influential component of a Bayesian model is the **LIKELIHOOD**. The likelihood is a mathematical formula that specifies the plausibility of the data.

What this means is that the likelihood maps each conjecture—such as a proportion of water on the globe—onto the relative number of ways the data could occur, given that possibility.

You can build your own likelihood formula from basic assumptions of your story for how the data arise. That’s what we did in the globe tossing example earlier. Or you can use one of several off-the-shelf likelihoods that are common in the sciences. Later in the book, you’ll see how information theory justifies many of the conventional choices of likelihood. But however you get the likelihood function, the likelihood needs to tell you the probability of any possible observation, for any possible state of the (small) world, such as a proportion of water on a globe.

In the case of the globe tossing model, the likelihood can be derived from the data story. Begin by nominating all of the possible events. There are two: *water* (W) and *land* (L). There are no other events. The globe never gets stuck to the ceiling, for example. When we observe a sample of W’s and L’s of length N (nine in the actual sample), we need to say how likely that exact sample is, out of the universe of potential samples of the same length. That might sound challenging, but it’s the kind of thing you get good at very quickly, once you start practicing.

In this case, once we add our assumptions that (1) every toss is independent of the other tosses and (2) the probability of W is the same on every toss, probability theory provides a unique answer, known as the *binomial distribution*. This is the common “coin tossing” distribution. And so the probability of observing w W’s in n tosses, with a probability p of W, is:

$$\Pr(w|n, p) = \frac{n!}{w!(n-w)!} p^w (1-p)^{n-w}$$

Read the above as:

The count of “water” observations w is distributed binomially, with probability p of “water” on each toss and n tosses in total.

And the binomial distribution formula is built into R, so you can easily compute the likelihood of the data—six W’s in nine tosses—under any value of p with:

```
dbinom( 6 , size=9 , prob=0.5 )
```

R code
2.2

```
[1] 0.1640625
```

That number is the relative number of ways to get six W’s, holding p at 0.5 and n at nine. So it does the job of counting relative number of paths through the garden. Change the 0.5 to any other value, to see how the value changes.

Sometimes, likelihoods are written $L(p|w, n)$: the likelihood of p , conditional on w and n . Note however that this notation reverses what is on the left side of the $|$ symbol. Just keep in mind that the job of the likelihood is to tell us the relative number of ways to see the data w , given values for p and n .

Overthinking: Names and probability distributions. The “d” in `dbinom` stands for *density*. Functions named in this way almost always have corresponding partners that begin with “r” for random samples and that begin with “p” for cumulative probabilities. See for example the help `?dbinom`.

Rethinking: A central role for likelihood. A great deal of ink has been spilled focusing on how Bayesian and non-Bayesian data analyses differ. Focusing on differences is useful, but sometimes it distracts us from fundamental similarities. Notably, the most influential assumptions in both Bayesian and many non-Bayesian models are the likelihood functions and their relations to the parameters. The assumptions about the likelihood influence inference for every piece of data, and as sample size increases, the likelihood matters more and more. This helps to explain why Bayesian and non-Bayesian inferences are often so similar.

2.3.2. Parameters. For most likelihood functions, there are adjustable inputs. In the binomial likelihood, these inputs are p (the probability of seeing a W), n (the sample size), and w (the number of W's). One or all of these may also be quantities that we wish to estimate from data, **PARAMETERS**, and they represent the different conjectures for causes or explanations of the data. In our globe tossing example, both n and w are data—we believe that we have observed their values without error. That leaves p as an unknown parameter, and our Bayesian machine's job is to describe what the data tell us about it.

But in other analyses, different inputs of the likelihood may be the targets of our analysis. It's common in field biology, for example, to estimate both p and n . Such an analysis may be called mark-recapture, capture-recapture, or sight-resight. The goal is usually to estimate population size, the n input of the binomial distribution. In other cases, we might know p and n , but not w . Your Bayesian machine can tell you what the data say about any parameter, once you tell it the likelihood and which bits have been observed. In some cases, the golem will just tell you that not much can be learned—Bayesian data analysis isn't magic, after all. But it is usually an advance to learn that our data don't discriminate among the possibilities.

In future chapters, there will be more parameters in your models, in excess of the small number that may appear in the likelihood. In statistical modeling, many of the most common questions we ask about data are answered directly by parameters:

- What is the average difference between treatment groups?
- How strong is the association between a treatment and an outcome?
- Does the effect of the treatment depend upon a covariate?
- How much variation is there among groups?

You'll see how these questions become extra parameters inside the likelihood.

Rethinking: Datum or parameter? It is typical to conceive of data and parameters as completely different kinds of entities. Data are measured and known; parameters are unknown and must be estimated from data. Usefully, in the Bayesian framework the distinction between a datum and a parameter is fuzzy. A datum can be recast as a very narrow probability density for a parameter, and a parameter as a datum with uncertainty. Much later in the book (Chapter 14), you'll see how to exploit this continuity between certainty (data) and uncertainty (parameters) to incorporate measurement error and missing data into your modeling.

2.3.3. Prior. For every parameter you intend your Bayesian machine to estimate, you must provide to the machine a **PRIOR**. A Bayesian machine must have an initial plausibility assignment for each possible value of the parameter. The prior is this initial set of plausibilities. When you have a previous estimate to provide to the machine, that can become the prior, as in the steps in **FIGURE 2.5**. Back in **FIGURE 2.5**, the machine did its learning one piece of data at a time. As a result, each estimate becomes the prior for the next step. But this

doesn't resolve the problem of providing a prior, because at the dawn of time, when $n = 0$, the machine still had an initial state of information for the parameter p : a flat line specifying equal plausibility for every possible value.

Overthinking: Prior as probability distribution. You could write the prior in the example here as:

$$\Pr(p) = \frac{1}{1 - 0} = 1.$$

The prior is a probability distribution for the parameter. In general, for a uniform prior from a to b , the probability of any point in the interval is $1/(b - a)$. If you're bothered by the fact that the probability of every value of p is 1, remember that every probability distribution must sum (integrate) to 1. The expression $1/(b - a)$ ensures that the area under the flat line from a to b is equal to 1. There will be more to say about this in Chapter 4.

So where do priors come from? They are engineering assumptions, chosen to help the machine learn. The flat prior in [FIGURE 2.5](#) is very common, but it is hardly ever the best prior. You'll see later in the book that priors that gently nudge the machine usually improve inference. Such priors are sometimes called **REGULARIZING** or **WEAKLY INFORMATIVE** priors. They are so useful that non-Bayesian statistical procedures have adopted a mathematically equivalent approach, **PENALIZED LIKELIHOOD**. These priors are conservative, in that they tend to guard against inferring strong associations between variables.

More generally, priors are useful for constraining parameters to reasonable ranges, as well as for expressing any knowledge we have about the parameter, before any data are observed. For example, in the globe tossing case, you know before the globe is tossed even once that the values $p = 0$ and $p = 1$ are completely implausible. You also know that any values of p very close to 0 and 1 are less plausible than values near $p = 0.5$. Even vague knowledge like this can be useful, when evidence is scarce.

There is a school of Bayesian inference that emphasizes choosing priors based upon the personal beliefs of the analyst.⁴⁵ While this **SUBJECTIVE BAYESIAN** approach thrives in some statistics and philosophy and economics programs, it is rare in the sciences. Within Bayesian data analysis in the natural and social sciences, the prior is considered to be just part of the model. As such it should be chosen, evaluated, and revised just like all of the other components of the model. In practice, the subjectivist and the non-subjectivist will often analyze data in nearly the same way.

None of this should be understood to mean that any statistical analysis is not inherently subjective, because of course it is—lots of little subjective decisions are involved in all parts of science. It's just that priors and Bayesian data analysis are no more inherently subjective than are likelihoods and the repeat sampling assumptions required for significance testing.⁴⁶ Anyone who has visited a statistics help desk at a university has probably experienced this subjectivity—statisticians do not in general exactly agree on how to analyze anything but the simplest of problems. The fact that statistical inference uses mathematics does not imply that there is only one reasonable or useful way to conduct an analysis. Engineering uses math as well, but there are many ways to build a bridge.

Beyond all of the above, there's no law mandating we use only one prior. If you don't have a strong argument for any particular prior, then try different ones. Because the prior is an assumption, it should be interrogated like other assumptions: by altering it and checking how sensitive inference is to the assumption. No one is required to swear an oath to the assumptions of a model, and no set of assumptions deserves our obedience.

Rethinking: Prior, prior pants on fire. Historically, some opponents of Bayesian inference objected to the arbitrariness of priors. It's true that priors are very flexible, being able to encode many different states of information. If the prior can be anything, isn't it possible to get any answer you want? Indeed it is. Regardless, after a couple hundred years of Bayesian calculation, it hasn't turned out that people use priors to lie. If your goal is to lie with statistics, you'd be a fool to do it with priors, because such a lie would be easily uncovered. Better to use the more opaque machinery of the likelihood. Or better yet—don't actually take this advice!—massage the data, drop some “outliers,” and otherwise engage in motivated data transformation.

It is true though that choice of the likelihood is much more conventionalized than choice of prior. But conventional choices are often poor ones, smuggling in influences that can be hard to discover. In this regard, both Bayesian and non-Bayesian models are equally harried, because both traditions depend heavily upon likelihood functions and conventionalized model forms. And the fact that the non-Bayesian procedure doesn't have to make an assumption about the prior is of little comfort. This is because non-Bayesian procedures need to make choices that Bayesian ones do not, such as choice of estimator or likelihood penalty. Often, such choices can be shown to be equivalent to some Bayesian choice of prior or rather choice of loss function. (You'll meet loss functions later in Chapter 3.)

2.3.4. Posterior. Once you have chosen a likelihood, which parameters are to be estimated, and a prior for each parameter, a Bayesian model treats the estimates as a purely logical consequence of those assumptions. For every unique combination of data, likelihood, parameters, and prior, there is a unique set of estimates. The resulting estimates—the relative plausibility of different parameter values, conditional on the data—are known as the **POSTERIOR DISTRIBUTION**. The posterior distribution takes the form of the probability of the parameters, conditional on the data: $\Pr(p|n, w)$.

The mathematical procedure that defines the logic of the posterior distribution is **BAYES' THEOREM**. This is the theorem that gives Bayesian data analysis its name. But the theorem itself is a trivial implication of probability theory. Here's a quick derivation of it, in the context of the globe tossing example. Really this will just be a re-expression of the garden of forking data derivation from earlier in the chapter. What makes it look different is that it will use the rules of probability theory to coax out the updating rule. But it is still just counting.

For simplicity, I'm going to omit n from the notation. The joint probability of the data w and any particular value of p is:

$$\Pr(w, p) = \Pr(w|p) \Pr(p)$$

This just says that the probability of w and p is the product of likelihood $\Pr(w|p)$ and the prior probability $\Pr(p)$. This is like saying that the probability of rain and cold on the same day is equal to the probability of rain, when it's cold, times the probability that it's cold. This much is just definition. But it's just as true that:

$$\Pr(w, p) = \Pr(p|w) \Pr(w)$$

All I've done is reverse which probability is conditional, on the right-hand side. It is still a true definition. It's like saying that the probability of rain and cold on the same day is equal to the probability that it's cold, when it's raining, times the probability of rain. Compare this statement to the one in the previous paragraph.

Now since both right-hand sides are equal to the same thing, we can set them equal to one another and solve for the posterior probability, $\Pr(p|w)$:

$$\Pr(p|w) = \frac{\Pr(w|p) \Pr(p)}{\Pr(w)}$$

And this is Bayes' theorem. It says that the probability of any particular value of p , considering the data, is equal to the product of the likelihood and prior, divided by this thing $\Pr(w)$, which I'll call the *average likelihood*. In word form:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Average Likelihood}}$$

The average likelihood, $\Pr(w)$, can be confusing. It is commonly called the “evidence” or the “probability of the data,” neither of which is a transparent name. The probability $\Pr(w)$ is merely the *average likelihood* of the data. Averaged over what? Averaged over the prior. It's job is to standardize the posterior, to ensure it sums (integrates) to one. In mathematical form:

$$\Pr(w) = E(\Pr(w|p)) = \int \Pr(w|p) \Pr(p) dp$$

The operator E means to take an *expectation*. Such averages are commonly called *marginals* in mathematical statistics, and so you may also see this same probability called a *marginal likelihood*. And the integral above just defines the proper way to compute the average over a continuous distribution of values, like the infinite possible values of p .

The key lesson is that the posterior is proportional to the product of the prior and the likelihood. Why? Because the number of paths through the garden of forking data is the product of the prior number of paths and the new number of paths. The likelihood indicates the number of paths, and the prior indicates the prior number. Multiplication is just compressed counting. The average likelihood on the bottom just standardizes the counts so they sum to one. So while Bayes' theorem looks complicated, because the relationship with counting paths is obscured, it just expresses the counting that logic demands.

FIGURE 2.6 illustrates the multiplicative interaction of a prior and a likelihood. On each row, a prior on the left is multiplied by the likelihood in the middle to produce a posterior on the right. The likelihood in each case is the same, the likelihood for the globe toss data. The priors however vary. As a result, the posterior distributions vary.

Rethinking: Bayesian data analysis isn't about Bayes' theorem. A common notion about Bayesian data analysis, and Bayesian inference more generally, is that it is distinguished by the use of Bayes' theorem. This is a mistake. Inference under any probability concept will eventually make use of Bayes' theorem. Common introductory examples of “Bayesian” analysis using HIV and DNA testing are not uniquely Bayesian. Since all of the elements of the calculation are frequencies of observations, a non-Bayesian analysis would do exactly the same thing. Instead, Bayesian approaches get to use Bayes' theorem more generally, to quantify uncertainty about theoretical entities that cannot be observed, like parameters and models. Powerful inferences can be produced under both Bayesian and non-Bayesian probability concepts, but different justifications and sacrifices are necessary.

2.4. Making the model go

Recall that your Bayesian model is a machine, a figurative golem. It has built-in definitions for the likelihood, the parameters, and the prior. And then at its heart lies a motor that

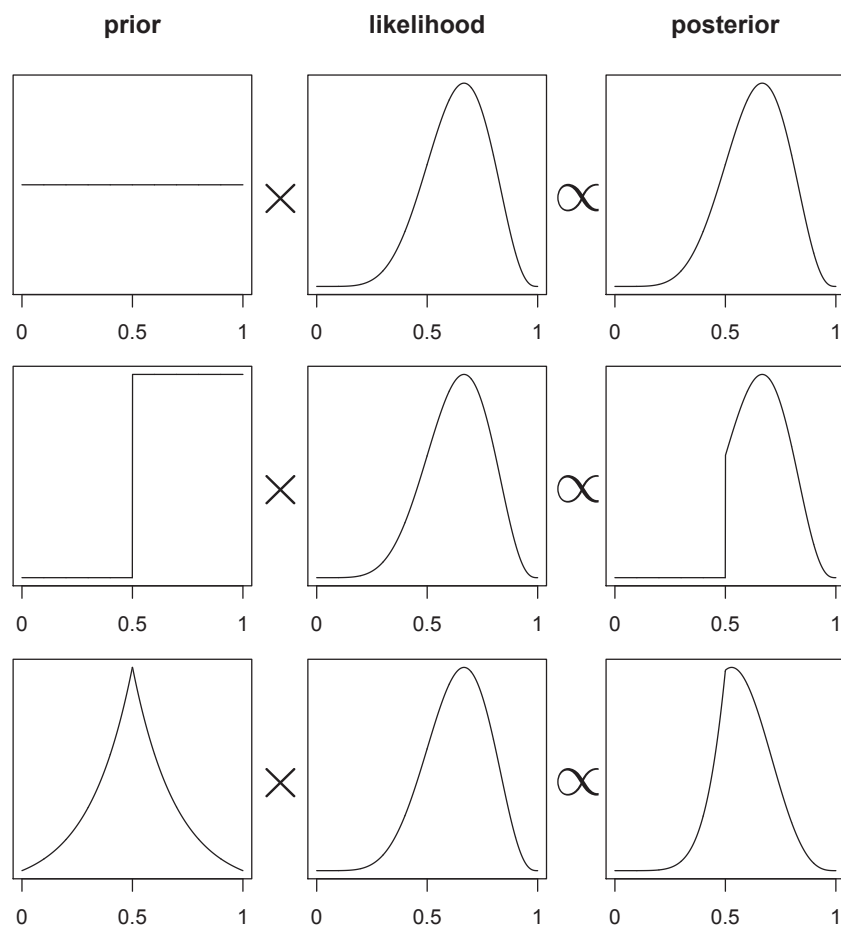


FIGURE 2.6. The posterior distribution, as a product of the prior distribution and likelihood. Top row: A flat prior constructs a posterior that is simply proportional to the likelihood. Middle row: A step prior, assigning zero probability to all values less than 0.5, resulting in a truncated posterior. Bottom row: A peaked prior that shifts and skews the posterior, relative to the likelihood.

processes data, producing a posterior distribution. The action of this motor can be thought of as *conditioning* the prior on the data. As explained in the previous section, this conditioning is governed by the rules of probability theory, which defines a uniquely logical posterior for every prior, likelihood, and data.

However, knowing the mathematical rule is often of little help, because many of the interesting models in contemporary science cannot be conditioned formally, no matter your skill in mathematics. And while some broadly useful models like linear regression can be conditioned formally, this is only possible if you constrain your choice of prior to special forms that are easy to do mathematics with. We'd like to avoid forced modeling choices of

this kind, instead favoring conditioning engines that can accommodate whichever prior is most useful for inference.

What this means is that various numerical techniques are needed to approximate the mathematics that follows from the definition of Bayes' theorem. In this book, you'll meet three different conditioning engines, numerical techniques for computing posterior distributions:

- (1) Grid approximation
- (2) Quadratic approximation
- (3) Markov chain Monte Carlo (MCMC)

There are many other engines, and new ones are being invented all the time. But the three you'll get to know here are common and widely useful. In addition, as you learn them, you'll also learn principles that will help you understand other techniques.

Rethinking: How you fit the model is part of the model. Earlier in this chapter, I implicitly defined the model as a composite of a prior and a likelihood. That definition is typical. But in practical terms, we should also consider how the model is fit to data as part of the model. In very simple problems, like the globe tossing example that consumes this chapter, calculation of the posterior density is trivial and foolproof. In even moderately complex problems, however, the details of fitting the model to data force us to recognize that our numerical technique influences our inferences. This is because different mistakes and compromises arise under different techniques. The same model fit to the same data using different techniques may produce different answers. When something goes wrong, every piece of the machine may be suspect. And so our golems carry with them their updating engines, as much slaves to their engineering as they are to the priors and likelihoods we program into them.

2.4.1. Grid approximation. One of the simplest conditioning techniques is grid approximation. While most parameters are *continuous*, capable of taking on an infinite number of values, it turns out that we can achieve an excellent approximation of the continuous posterior distribution by considering only a finite grid of parameter values. At any particular value of a parameter, p' , it's a simple matter to compute the posterior probability: just multiply the prior probability of p' by the likelihood at p' . Repeating this procedure for each value in the grid generates an approximate picture of the exact posterior distribution. This procedure is called **GRID APPROXIMATION**. In this section, you'll see how to perform a grid approximation, using simple bits of R code.

Grid approximation will mainly be useful as a pedagogical tool, as learning it forces the user to really understand the nature of Bayesian updating. But in most of your real modeling, grid approximation isn't practical. The reason is that it scales very poorly, as the number of parameters increases. So in later chapters, grid approximation will fade away, to be replaced by other, more efficient techniques. Still, the conceptual value of this exercise will carry forward, as you graduate to other techniques.

In the context of the globe tossing problem, grid approximation works extremely well. So let's build a grid approximation for the model we've constructed so far. Here is the recipe:

- (1) Define the grid. This means you decide how many points to use in estimating the posterior, and then you make a list of the parameter values on the grid.
- (2) Compute the value of the prior at each parameter value on the grid.
- (3) Compute the likelihood at each parameter value.
- (4) Compute the unstandardized posterior at each parameter value, by multiplying the prior by the likelihood.

(5) Finally, standardize the posterior, by dividing each value by the sum of all values.

In the globe tossing context, here's the code to complete all five of these steps:

```
R code
2.3
# define grid
p_grid <- seq( from=0 , to=1 , length.out=20 )

# define prior
prior <- rep( 1 , 20 )

# compute likelihood at each value in grid
likelihood <- dbinom( 6 , size=9 , prob=p_grid )

# compute product of likelihood and prior
unstd.posterior <- likelihood * prior

# standardize the posterior, so it sums to 1
posterior <- unstd.posterior / sum(unstd.posterior)
```

The above code makes a grid of only 20 points. To display the posterior distribution now:

```
R code
2.4
plot( p_grid , posterior , type="b" ,
      xlab="probability of water" , ylab="posterior probability" )
mtext( "20 points" )
```

You'll get the right-hand plot in [FIGURE 2.7](#). Try sparser grids (5 points) and denser grids (100 or 1000 points). The correct density for your grid is determined by how accurate you want your approximation to be. More points means more precision. In this simple example, you can go crazy and use 100,000 points, but there won't be much change in inference after the first 100.

Now to replicate the different priors in [FIGURE 2.5](#), try these lines of code—one at a time—for the prior grid:

```
R code
2.5
prior <- ifelse( p_grid < 0.5 , 0 , 1 )
prior <- exp( -5*abs( p_grid - 0.5 ) )
```

The rest of the code remains the same.

Overthinking: Vectorization. One of R's useful features is that it makes working with lists of numbers almost as easy as working with single values. So even though both lines of code above say nothing about how dense your grid is, whatever length you chose for the vector `p_grid` will determine the length of the vector `prior`. In R jargon, the calculations above are *vectorized*, because they work on lists of values, *vectors*. In a vectorized calculation, the calculation is performed on each element of the input vector—`p_grid` in this case—and the resulting output therefore has the same length. In other computing environments, the same calculation would require a *loop*. R can also use loops, but vectorized calculations are typically faster. They can however be much harder to read, when you are starting out with R. Be patient, and you'll soon grow accustomed to vectorized calculations.

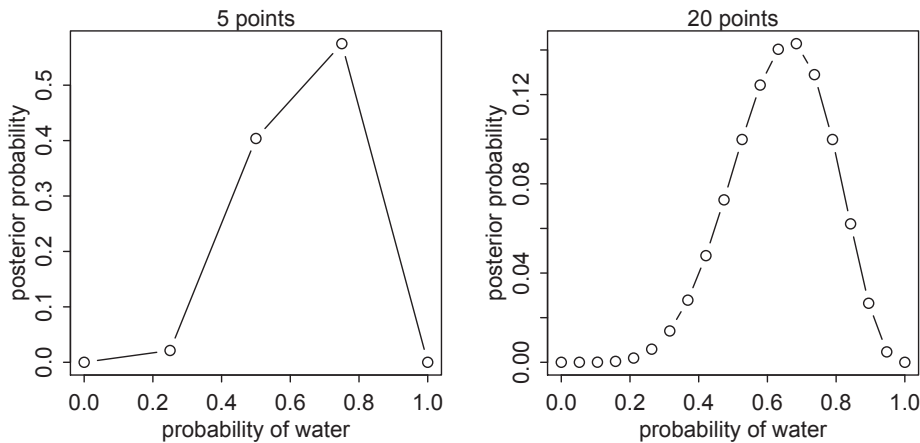


FIGURE 2.7. Computing posterior distribution by grid approximation. In each plot, the posterior distribution for the globe toss data and model is approximated with a finite number of evenly spaced points. With only 5 points (left), the approximation is terrible. But with 20 points (right), the approximation is already quite good. Compare to the analytically solved, exact posterior distribution in [FIGURE 2.5](#) (page 30).

2.4.2. Quadratic approximation. We'll stick with the grid approximation to the globe tossing posterior, for the rest of this chapter and the next. But before long you'll have to resort to another approximation, one that makes stronger assumptions. The reason is that the number of unique values to consider in the grid grows rapidly as the number of parameters in your model increases. For the single-parameter globe tossing model, it's no problem to compute a grid of 100 or 1000 values. But for two parameters approximated by 100 values each, that's already $100^2 = 10000$ values to compute. For 10 parameters, the grid becomes many billions of values. These days, it's routine to have models with hundreds or thousands of parameters. The grid approximation strategy scales very poorly with model complexity, so it won't get us very far.

A useful approach is **QUADRATIC APPROXIMATION**. Under quite general conditions, the region near the peak of the posterior distribution will be nearly Gaussian—or “normal”—in shape. This means the posterior distribution can be usefully approximated by a Gaussian distribution. A Gaussian distribution is convenient, because it can be completely described by only two numbers: the location of its center (mean) and its spread (variance).

A Gaussian approximation is called “quadratic approximation” because the logarithm of a Gaussian distribution forms a parabola. And a parabola is a quadratic function. So this approximation essentially represents any log-posterior with a parabola.

We'll use quadratic approximation for much of the first half of this book. For many of the most common procedures in applied statistics—linear regression, for example—the approximation works very well. Often, it is even exactly correct, not actually an approximation at all. Computationally, quadratic approximation is very inexpensive, at least compared to grid approximation and MCMC (discussed next). The procedure, which R will happily conduct at your command, contains two steps.

- (1) Find the posterior mode. This is usually accomplished by some optimization algorithm, a procedure that virtually “climbs” the posterior distribution, as if it were a mountain. The golem doesn’t know where the peak is, but it does know the slope under its feet. There are many well-developed optimization procedures, most of them more clever than simple hill climbing. But all of them try to find peaks.
- (2) Once you find the peak of the posterior, you must estimate the curvature near the peak. This curvature is sufficient to compute a quadratic approximation of the entire posterior distribution. In some cases, these calculations can be done analytically, but usually your computer uses some numerical technique instead.

To compute the quadratic approximation for the globe tossing data, we’ll use a tool in the *rethinking* package: `map`. The abbreviation MAP stands for **MAXIMUM A POSTERIORI**, which is just a fancy Latin name for the mode of the posterior distribution. We’re going to be using `map` a lot in the book. It’s a flexible model fitting tool that will allow us to specify a large number of different “regression” models. So it’ll be worth trying it out right now. You’ll get a more thorough understanding of it later.

To compute the quadratic approximation to the globe tossing data:

R code
2.6

```
library(rethinking)
globe.qa <- map(
  alist(
    w ~ dbinom(9,p) , # binomial likelihood
    p ~ dunif(0,1)    # uniform prior
  ) ,
  data=list(w=6) )

# display summary of quadratic approximation
precis( globe.qa )
```

To use `map`, you provide a *formula*, a list of *data*, and a list of *start* values for the parameters. The formula defines the likelihood and prior. I’ll say much more about these formulas in Chapter 4. The data list is just the count of water (6). Now let’s see the output:

```
Mean StdDev 5.5% 94.5%
p 0.67  0.16 0.42  0.92
```

The function `precis` presents a brief summary of the quadratic approximation. In this case, it shows a MAP value of $p = 0.67$, which it calls the “Mean.” The curvature is labeled “Std-Dev.” This stands for *standard deviation*. This value is the standard deviation of the posterior distribution, while the mean value is its peak. Finally, the last two values in the `precis` output show the 89% percentile interval, which you’ll learn more about in the next chapter. You can read this kind of approximation like: *Assuming the posterior is Gaussian, it is maximized at 0.67, and its standard deviation is 0.16.*

Since we already know the posterior, let’s compare to see how good the approximation is. I’ll use the analytical approach here, which uses `dbeta`. I won’t explain this calculation, but it ensures that we have exactly the right answer, with no approximations. You can find an explanation and derivation of it in just about any mathematical textbook on Bayesian inference.

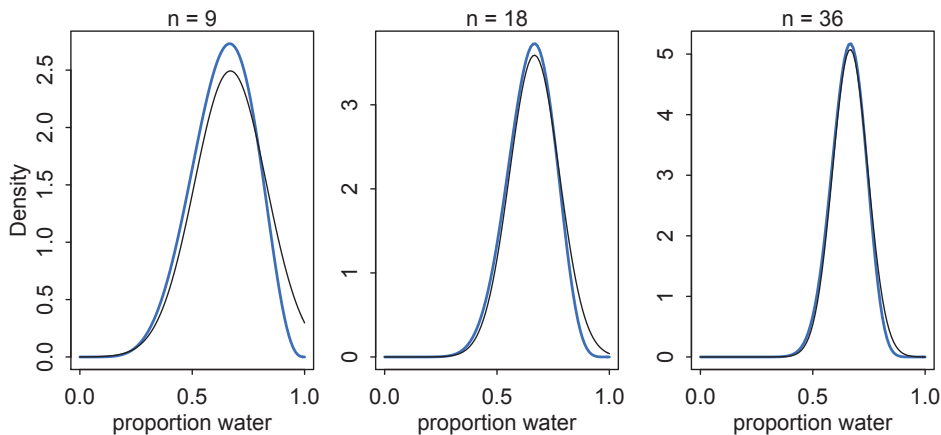


FIGURE 2.8. Accuracy of the quadratic approximation. In each plot, the exact posterior distribution is plotted in blue, and the quadratic approximation is plotted as the black curve. Left: The globe tossing data with $n = 9$ tosses and $w = 6$ waters. Middle: Double the amount of data, with the same fraction of water, $n = 18$ and $w = 12$. Right: Four times as much data, $n = 36$ and $w = 24$.

```
# analytical calculation
w <- 6
n <- 9
curve( dbeta( x , w+1 , n-w+1 ) , from=0 , to=1 )
# quadratic approximation
curve( dnorm( x , 0.67 , 0.16 ) , lty=2 , add=TRUE )
```

R code
2.7

You can see this plot (with a little extra formatting) on the left in [FIGURE 2.8](#). The blue curve is the analytical posterior and the black curve is the quadratic approximation. The black curve does alright on its left side, but looks pretty bad on its right side. It even assigns positive probability to $p = 1$, which we know is impossible, since we saw at least one land sample.

As the amount of data increases, however, the quadratic approximation gets better. In the middle of [FIGURE 2.8](#), the sample size is doubled to $n = 18$ tosses, but with the same fraction of water, so that the mode of the posterior is in the same place. The quadratic approximation looks better now, although still not great. At quadruple the data, on the right side of the figure, the two curves are nearly the same now.

This phenomenon, where the quadratic approximation improves with the amount of data, is very common. It's one of the reasons that so many classical statistical procedures are nervous about small samples: Those procedures use quadratic (or other) approximations that are only known to be safe with infinite data. Often, these approximations are useful with less than infinite data, obviously. But the rate of improvement as sample size increases varies greatly depending upon the details. In some model types, the quadratic approximation can remain terrible even with thousands of samples.

Using the quadratic approximation in a Bayesian context brings with it all the same concerns. But you can always lean on some algorithm other than quadratic approximation, if

you have doubts. Indeed, grid approximation works very well with small samples, because in such cases the model must be simple and the computations will be quite fast. You can also use MCMC, which is introduced next.

Rethinking: Maximum likelihood estimation. The quadratic approximation, either with a uniform prior or with a lot of data, is usually equivalent to a **MAXIMUM LIKELIHOOD ESTIMATE** (MLE) and its **STANDARD ERROR**. The MLE is a very common non-Bayesian parameter estimate. This equivalence between a Bayesian approximation and a common non-Bayesian estimator is both a blessing and a curse. It is a blessing, because it allows us to re-interpret a wide range of published non-Bayesian model fits in Bayesian terms. It is a curse, because maximum likelihood estimates have some curious drawbacks, which we'll explore in later chapters.

Overthinking: The Hessians are coming. Sometimes it helps to know more about how the quadratic approximation is computed. In particular, the approximation sometimes fails. When it does, chances are you'll get a confusing error message that says something about the "Hessian." Students of world history may know that the Hessians were German mercenaries hired by the British in the 18th century to do various things, including fight against the American revolutionary George Washington. These mercenaries are named after a region of what is now central Germany, Hesse.

The Hessian that concerns us here has little to do with mercenaries. It is named after mathematician Ludwig Otto Hesse (1811–1874). A *Hessian* is a square matrix of second derivatives. It is used for many purposes in mathematics, but in the quadratic approximation it is second derivatives of the log of posterior probability with respect to the parameters. It turns out that these derivatives are sufficient to describe a Gaussian distribution, because the logarithm of a Gaussian distribution is just a parabola. Parabolas have no derivatives beyond the second, so once we know the center of the parabola (the posterior mode) and its second derivative, we know everything about it. And indeed the second derivative (with respect to the outcome) of the logarithm of a Gaussian distribution is proportional to its inverse squared standard deviation (its "precision": page 76). So knowing the standard deviation tells us everything about its shape.

The standard deviation is typically computed from the Hessian, so computing the Hessian is nearly always a necessary step. But sometimes the computation goes wrong, and your golem will choke while trying to compute the Hessian. In those cases, you have several options. Not all hope is lost. But for now it's enough to recognize the term and associate it with an attempt to find the standard deviation for a quadratic approximation.

2.4.3. Markov chain Monte Carlo. There are lots of important model types, like multi-level (mixed-effects) models, for which neither grid approximation nor quadratic approximation is always satisfactory. Such models can easily have hundreds or thousands or tens-of-thousands of parameters. Grid approximation obviously fails here, because it just takes too long—the Sun will go dark before your computer finishes the grid. Special forms of quadratic approximation might work, if everything is just right. But commonly, something is not just right. Furthermore, multilevel models do not always allow us to write down a single, unified function for the posterior distribution. This means that the function to maximize (when finding the MAP) is not known, but must be computed in pieces.

As a result, various counterintuitive model fitting techniques have arisen. The most popular of these is **MARKOV CHAIN MONTE CARLO** (MCMC), which is a family of conditioning engines capable of handling highly complex models. It is fair to say that MCMC is largely responsible for the insurgence of Bayesian data analysis that began in the 1990s. While MCMC

is older than the 1990s, affordable computer power is not, so we must also thank the engineers. Much later in the book (Chapter 8), you'll meet simple and precise examples of MCMC model fitting, aimed at helping you understand the technique.

The conceptual challenge with MCMC lies in its highly non-obvious strategy. Instead of attempting to compute or approximate the posterior distribution directly, MCMC techniques merely draw samples from the posterior. You end up with a collection of parameter values, and the frequencies of these values correspond to the posterior plausibilities. You can then build a picture of the posterior from the histogram of these samples.

We nearly always work directly with these samples, rather than first constructing some mathematical estimate from them. And the samples are in many ways more convenient than having the posterior, because they are easier to think with. And so that's where we turn in the next chapter, to thinking with samples.

2.5. Summary

This chapter introduced the conceptual mechanics of Bayesian data analysis. The target of inference in Bayesian inference is a posterior probability distribution. Posterior probabilities state the relative numbers of ways each conjectured cause of the data could have produced the data. These relative numbers indicate plausibilities of the different conjectures. These plausibilities are updated in light of observations, a process known as Bayesian updating.

More mechanically, a Bayesian model is a composite of a likelihood, a choice of parameters, and a prior. The likelihood provides the plausibility of an observation (data), given a fixed value for the parameters. The prior provides the plausibility of each possible value of the parameters, before accounting for the data. The rules of probability tell us that the logical way to compute the plausibilities, after accounting for the data, is to use Bayes' theorem. This results in the posterior distribution.

In practice, Bayesian models are fit to data using numerical techniques, like grid approximation, quadratic approximation, and Markov chain Monte Carlo. Each method imposes different trade-offs.

2.6. Practice

Easy.

2E1. Which of the expressions below correspond to the statement: *the probability of rain on Monday*?

- (1) $\Pr(\text{rain})$
- (2) $\Pr(\text{rain}|\text{Monday})$
- (3) $\Pr(\text{Monday}|\text{rain})$
- (4) $\Pr(\text{rain}, \text{Monday}) / \Pr(\text{Monday})$

2E2. Which of the following statements corresponds to the expression: $\Pr(\text{Monday}|\text{rain})$?

- (1) The probability of rain on Monday.
- (2) The probability of rain, given that it is Monday.
- (3) The probability that it is Monday, given that it is raining.
- (4) The probability that it is Monday and that it is raining.

2E3. Which of the expressions below correspond to the statement: *the probability that it is Monday, given that it is raining?*

- (1) $\Pr(\text{Monday}|\text{rain})$
- (2) $\Pr(\text{rain}|\text{Monday})$
- (3) $\Pr(\text{rain}|\text{Monday}) \Pr(\text{Monday})$
- (4) $\Pr(\text{rain}|\text{Monday}) \Pr(\text{Monday}) / \Pr(\text{rain})$
- (5) $\Pr(\text{Monday}|\text{rain}) \Pr(\text{rain}) / \Pr(\text{Monday})$

2E4. The Bayesian statistician Bruno de Finetti (1906–1985) began his book on probability theory with the declaration: “PROBABILITY DOES NOT EXIST.” The capitals appeared in the original, so I imagine de Finetti wanted us to shout this statement. What he meant is that probability is a device for describing uncertainty from the perspective of an observer with limited knowledge; it has no objective reality. Discuss the globe tossing example from the chapter, in light of this statement. What does it mean to say “the probability of water is 0.7”?

Medium.

2M1. Recall the globe tossing model from the chapter. Compute and plot the grid approximate posterior distribution for each of the following sets of observations. In each case, assume a uniform prior for p .

- (1) W, W, W
- (2) W, W, W, L
- (3) L, W, W, L, W, W, W

2M2. Now assume a prior for p that is equal to zero when $p < 0.5$ and is a positive constant when $p \geq 0.5$. Again compute and plot the grid approximate posterior distribution for each of the sets of observations in the problem just above.

2M3. Suppose there are two globes, one for Earth and one for Mars. The Earth globe is 70% covered in water. The Mars globe is 100% land. Further suppose that one of these globes—you don’t know which—was tossed in the air and produced a “land” observation. Assume that each globe was equally likely to be tossed. Show that the posterior probability that the globe was the Earth, conditional on seeing “land” ($\Pr(\text{Earth}|\text{land})$), is 0.23.

2M4. Suppose you have a deck with only three cards. Each card has two sides, and each side is either black or white. One card has two black sides. The second card has one black and one white side. The third card has two white sides. Now suppose all three cards are placed in a bag and shuffled. Someone reaches into the bag and pulls out a card and places it flat on a table. A black side is shown facing up, but you don’t know the color of the side facing down. Show that the probability that the other side is also black is $2/3$. Use the counting method (Section 2 of the chapter) to approach this problem. This means counting up the ways that each card could produce the observed data (a black side facing up on the table).

2M5. Now suppose there are four cards: B/B, B/W, W/W, and another B/B. Again suppose a card is drawn from the bag and a black side appears face up. Again calculate the probability that the other side is black.

2M6. Imagine that black ink is heavy, and so cards with black sides are heavier than cards with white sides. As a result, it’s less likely that a card with black sides is pulled from the bag. So again assume there are three cards: B/B, B/W, and W/W. After experimenting a number of times, you conclude that for every way to pull the B/B card from the bag, there are 2 ways to pull the B/W card and 3 ways to pull the W/W card. Again suppose that a card is pulled and a black side appears face up. Show that the probability the other side is black is now 0.5. Use the counting method, as before.

2M7. Assume again the original card problem, with a single card showing a black side face up. Before looking at the other side, we draw another card from the bag and lay it face up on the table. The face that is shown on the new card is white. Show that the probability that the first card, the one showing a black side, has black on its other side is now 0.75. Use the counting method, if you can. Hint: Treat this like the sequence of globe tosses, counting all the ways to see each observation, for each possible first card.

Hard.

2H1. Suppose there are two species of panda bear. Both are equally common in the wild and live in the same places. They look exactly alike and eat the same food, and there is yet no genetic assay capable of telling them apart. They differ however in their family sizes. Species A gives birth to twins 10% of the time, otherwise birthing a single infant. Species B births twins 20% of the time, otherwise birthing singleton infants. Assume these numbers are known with certainty, from many years of field research.

Now suppose you are managing a captive panda breeding program. You have a new female panda of unknown species, and she has just given birth to twins. What is the probability that her next birth will also be twins?

2H2. Recall all the facts from the problem above. Now compute the probability that the panda we have is from species A, assuming we have observed only the first birth and that it was twins.

2H3. Continuing on from the previous problem, suppose the same panda mother has a second birth and that it is not twins, but a singleton infant. Compute the posterior probability that this panda is species A.

2H4. A common boast of Bayesian statisticians is that Bayesian inference makes it easy to use all of the data, even if the data are of different types.

So suppose now that a veterinarian comes along who has a new genetic test that she claims can identify the species of our mother panda. But the test, like all tests, is imperfect. This is the information you have about the test:

- The probability it correctly identifies a species A panda is 0.8.
- The probability it correctly identifies a species B panda is 0.65.

The vet administers the test to your panda and tells you that the test is positive for species A. First ignore your previous information from the births and compute the posterior probability that your panda is species A. Then redo your calculation, now using the birth data as well.

3 Sampling the Imaginary

Lots of books on Bayesian statistics introduce posterior inference by using a medical testing scenario. To repeat the structure of common examples, suppose there is a blood test that correctly detects vampirism 95% of the time. This implies $\Pr(\text{positive}|\text{vampire}) = 0.95$. It's a very accurate test. It does make mistakes, though, in the form of false positives. One percent of the time, it incorrectly diagnoses normal people as vampires, implying $\Pr(\text{positive}|\text{mortal}) = 0.01$. The final bit of information we are told is that vampires are rather rare, being only 0.1% of the population, implying $\Pr(\text{vampire}) = 0.001$. Suppose now that someone tests positive for vampirism. What's the probability that he or she is a bloodsucking immortal?

The correct approach is just to use Bayes' theorem to invert the probability, to compute $\Pr(\text{vampire}|\text{positive})$. The calculation can be presented as:

$$\Pr(\text{vampire}|\text{positive}) = \frac{\Pr(\text{positive}|\text{vampire}) \Pr(\text{vampire})}{\Pr(\text{positive})}$$

where $\Pr(\text{positive})$ is the average probability of a positive test result, that is,

$$\begin{aligned} \Pr(\text{positive}) &= \Pr(\text{positive}|\text{vampire}) \Pr(\text{vampire}) \\ &\quad + \Pr(\text{positive}|\text{mortal}) (1 - \Pr(\text{vampire})) \end{aligned}$$

Performing the calculation in R:

```
PrPV <- 0.95
PrPM <- 0.01
PrV <- 0.001
PrP <- PrPV*PrV + PrPM*(1-PrV)
( PrVP <- PrPV*PrV / PrP )
```

R code
3.1

```
[1] 0.08683729
```

That corresponds to an 8.7% chance that the suspect is actually a vampire.

Most people find this result counterintuitive. And it's a very important problem, because it mimics the structure of many realistic testing contexts, such as HIV and DNA testing, criminal profiling, and even statistical significance testing (see the rethinking box at the end of this section). Whenever the condition of interest is very rare, having a test that finds all the true cases is still no guarantee that a positive result carries much information at all. The reason is that most positive results are false positives, even when all the true positives are detected correctly.

But I don't like these examples, for two reasons. First, there's nothing really "Bayesian" about them. Remember: Bayesian inference is distinguished by a broad view of probability, not by the use of Bayes' theorem. Since all of the probabilities I provided above reference frequencies of events, rather than theoretical parameters, all major statistical philosophies would agree to use Bayes' theorem in this case. Second, and more important to our work in this chapter, these examples make Bayesian inference seem much harder than it has to be. Few people find it easy to remember which number goes where, probably because they never grasp the logic of the procedure. It's just a formula that descends from the sky.

There is a way to present the same problem that does make it more intuitive, however. Suppose that instead of reporting probabilities, as before, I tell you the following:

- (1) In a population of 100,000 people, 100 of them are vampires.
- (2) Of the 100 who are vampires, 95 of them will test positive for vampirism.
- (3) Of the 99,900 mortals, 999 of them will test positive for vampirism.

Now tell me, if we test all 100,000 people, what proportion of those who test positive for vampirism actually are vampires? Many people, although certainly not all people, find this presentation a lot easier.⁴⁷ Now we can just count up the number of people who test positive: $95 + 999 = 1094$. Out of these 1094 positive tests, 95 of them are real vampires, so that implies:

$$\Pr(\text{vampire}|\text{positive}) = \frac{95}{1094} \approx 0.087$$

It's exactly the same answer as before, but without a seemingly arbitrary rule.

The second presentation of the problem, using counts rather than probabilities, is often called the *frequency format* or *natural frequencies*. Why a frequency format helps people intuit the correct approach remains contentious. Some people think that human psychology naturally works better when it receives information in the form a person in a natural environment would receive it. In the real world, we encounter counts only. No one has ever seen a probability, the thinking goes. But everyone sees counts ("frequencies") in their daily lives. Maybe so.

Rethinking: The natural frequency phenomenon is not unique. Changing the representation of a problem often makes it easier to address or inspires new ideas that were not available in an old representation.⁴⁸ In physics, switching between Newtonian and Lagrangian mechanics can make problems much easier. In evolutionary biology, switching between inclusive fitness and multilevel selection sheds new light on old models. And in statistics, switching between Bayesian and non-Bayesian representations often teaches us new things about both approaches.

Regardless of the explanation for this phenomenon, we can exploit it. And in this chapter we exploit it by taking the probability distributions from the previous chapter and sampling from them to produce counts. The posterior distribution is a probability distribution. And like all probability distributions, we can imagine drawing *samples* from it. The sampled events in this case are parameter values. Most parameters have no exact empirical realization. The Bayesian formalism treats parameter distributions as relative plausibility, not as any physical random process. In any event, randomness is always a property of information, never of the real world. But inside the computer, parameters are just as empirical as the outcome of a coin flip or a die toss or an agricultural experiment. The posterior defines the expected frequency that different parameter values will appear, once we start plucking parameters out of it.

This chapter teaches you basic skills for working with samples from the posterior distribution. It will seem a little silly to work with samples at this point, because the posterior distribution for the globe tossing model is very simple. It's so simple that it's no problem to work directly with the grid approximation or even the exact mathematical form.⁴⁹ But there are two reasons to adopt the sampling approach early on, before it's really necessary.

First, many scientists are quite shaky about integral calculus, even though they have strong and valid intuitions about how to summarize data. Working with samples transforms a problem in calculus into a problem in data summary, into a frequency format problem. An integral in a typical Bayesian context is just the total probability in some interval. That can be a challenging calculus problem. But once you have samples from the probability distribution, it's just a matter of counting values in the interval. Even seemingly simple calculations, like confidence intervals, are made difficult once a model has many parameters. In those cases, one must average over the uncertainty in all other parameters, when describing the uncertainty in a focal parameter. This requires a complicated integral, but only a very simple data summary. An empirical attack on the posterior allows the scientist to ask and answer more questions about the model, without relying upon a captive mathematician. For this reason, it is often easier and more intuitive to work with samples from the posterior, than to work with probabilities and integrals directly.

Second, some of the most capable methods of computing the posterior produce nothing but samples. Many of these methods are variants of Markov chain Monte Carlo techniques (MCMC). So if you learn early on how to conceptualize and process samples from the posterior, when you inevitably must fit a model to data using MCMC—and chances are you will—you will already know how to make sense of the output. Beginning with Chapter 8 of this book, you will use MCMC to open up the types and complexity of models you can practically fit to data. MCMC is no longer a technique only for experts, but rather part of the standard toolkit of quantitative science. So it's worth planning ahead.

So in this chapter we'll begin to use samples to summarize and simulate model output. The skills you learn here will apply to every problem in the remainder of the book, even though the details of the models, how they are fit to data, and how the samples are produced will vary.

Rethinking: Why statistics can't save bad science. The vampirism example at the start of this chapter has the same logical structure as many different *signal detection* problems: (1) There is some binary state that is hidden from us; (2) we observe an imperfect cue of the hidden state; (3) we (should) use Bayes' theorem to logically deduce the impact of the cue on our uncertainty.

Scientific inference is often framed in similar terms: (1) An hypothesis is either true or false; (2) we use a statistical procedure and get an imperfect cue of the hypothesis' falsity; (3) we (should) use Bayes' theorem to logically deduce the impact of the cue on the status of the hypothesis. It's the third step that is hardly ever done. But let's do it, for a toy example, so you can see how little statistical procedures—Bayesian or not—may do for us.

Suppose the probability of a positive finding, when an hypothesis is true, is $\Pr(\text{sig}|\text{true}) = 0.95$. That's the *power* of the test. Suppose that the probability of a positive finding, when an hypothesis is false, is $\Pr(\text{sig}|\text{false}) = 0.05$. That's the false-positive rate, like the 5% of conventional significance testing. Finally, we have to state the *base rate* at which hypotheses are true. Suppose for example that 1 in every 100 hypotheses turns out to be true. Then $\Pr(\text{true}) = 0.01$. No one knows this value, but the history of science suggests it's small. See Chapter 15 for more discussion. Now use Bayes' to

compute the posterior:

$$\Pr(\text{true}|\text{pos}) = \frac{\Pr(\text{pos}|\text{true}) \Pr(\text{true})}{\Pr(\text{pos})} = \frac{\Pr(\text{pos}|\text{true}) \Pr(\text{true})}{\Pr(\text{pos}|\text{true}) \Pr(\text{true}) + \Pr(\text{pos}|\text{false}) \Pr(\text{false})}$$

Plug in the appropriate values, and the answer is approximately $\Pr(\text{true}|\text{pos}) = 0.16$. So a positive finding corresponds to a 16% chance that the hypothesis is true. This is the same low base-rate phenomenon that applies in medical (and vampire) testing. You can shrink the false-positive rate to 1% and get this posterior probability up to around 0.5, only as good as a coin flip. The most important thing to do is to improve the base rate, $\Pr(\text{true})$, and that requires thinking, not testing.⁵⁰

3.1. Sampling from a grid-approximate posterior

Before beginning to work with samples, we need to generate them. Here's a reminder for how to compute the posterior for the globe tossing model, using grid approximation:

```
R code 3.2
p_grid <- seq( from=0 , to=1 , length.out=1000 )
prior <- rep( 1 , 1000 )
likelihood <- dbinom( 6 , size=9 , prob=p_grid )
posterior <- likelihood * prior
posterior <- posterior / sum(posterior)
```

Now we wish to draw 10,000 samples from this posterior. Imagine the posterior is a bucket full of parameter values, numbers such as 0.1, 0.7, 0.5, 1, etc. Within the bucket, each value exists in proportion to its posterior probability, such that values near the peak are much more common than those in the tails. We're going to scoop out 10,000 values from the bucket. Provided the bucket is well mixed, the resulting samples will have the same proportions as the exact posterior density. Therefore the individual values of p will appear in our samples in proportion to the posterior plausibility of each value.

Here's how you can do this in R, with one line of code:

```
R code 3.3
samples <- sample( p_grid , prob=posterior , size=1e4 , replace=TRUE )
```

The workhorse here is `sample`, which randomly pulls values from a vector. The vector in this case is `p_grid`, the grid of parameter values. The probability of each value is given by `posterior`, which you computed just above.

The resulting samples are displayed in [FIGURE 3.1](#). On the left, all 10,000 (1e4) random samples are shown sequentially.

```
R code 3.4
plot( samples )
```

In this plot, it's as if you are flying over the posterior distribution, looking down on it. There are many more samples from the dense region near 0.6 and very few samples below 0.25. On the right, the plot shows the *density estimate* computed from these samples.

```
R code 3.5
library(rethinking)
dens( samples )
```

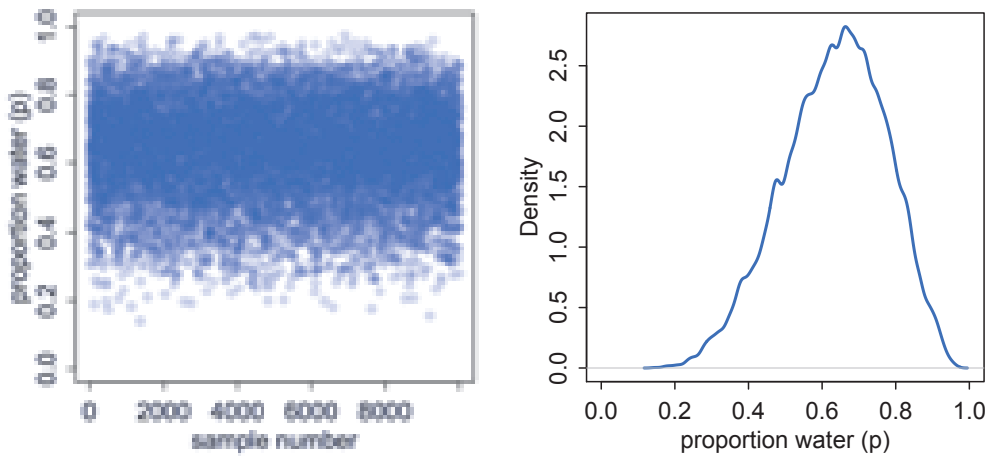


FIGURE 3.1. Sampling parameter values from the posterior distribution. Left: 10,000 samples from the posterior implied by the globe tossing data and model. Right: The density of samples (vertical) at each parameter value (horizontal).

You can see that the estimated density is very similar to ideal posterior you computed via grid approximation. If you draw even more samples, maybe $1e5$ or $1e6$, the density estimate will get more and more similar to the ideal.

All you've done so far is crudely replicate the posterior density you had already computed. So next it is time to use these samples to describe and understand the posterior.

3.2. Sampling to summarize

Once your model produces a posterior distribution, the model's work is done. But your work has just begun. It is necessary to summarize and interpret the posterior distribution. Exactly how it is summarized depends upon your purpose. But common questions include:

- How much posterior probability lies below some parameter value?
- How much posterior probability lies between two parameter values?
- Which parameter value marks the lower 5% of the posterior probability?
- Which range of parameter values contains 90% of the posterior probability?
- Which parameter value has highest posterior probability?

These simple questions can be usefully divided into questions about (1) intervals of *defined boundaries*, (2) questions about intervals of *defined probability mass*, and (3) questions about *point estimates*. We'll see how to approach these questions using samples from the posterior.

3.2.1. Intervals of defined boundaries. Suppose I ask you for the posterior probability that the proportion of water is less than 0.5. Using the grid-approximate posterior, you can just add up all of the probabilities, where the corresponding parameter value is less than 0.5:

```
# add up posterior probability where p < 0.5
sum( posterior[ p_grid < 0.5 ] )
```

R code
3.6

```
[1] 0.1718746
```

So about 17% of the posterior probability is below 0.5. Couldn't be easier. But since grid approximation isn't practical in general, it won't always be so easy. Once there is more than one parameter in the posterior distribution (wait until the next chapter for that complication), even this simple sum is no longer very simple.

So let's see how to perform the same calculation, using samples from the posterior. This approach does generalize to complex models with many parameters, and so you can use it everywhere. All you have to do is similarly add up all of the samples below 0.5, but also divide the resulting count by the total number of samples. In other words, find the frequency of parameter values below 0.5:

```
R code 3.7 sum( samples < 0.5 ) / 1e4
```

```
[1] 0.1726
```

And that's nearly the same answer as the grid approximation provided, although your answer will not be exactly the same, because the exact samples you drew from the posterior will be different. This region is shown in the upper-left plot in [FIGURE 3.2](#). Using the same approach, you can ask how much posterior probability lies between 0.5 and 0.75:

```
R code 3.8 sum( samples > 0.5 & samples < 0.75 ) / 1e4
```

```
[1] 0.6059
```

So about 61% of the posterior probability lies between 0.5 and 0.75. This region is shown in the upper-right plot of [FIGURE 3.2](#).

Overthinking: Counting with `sum`. In the R code examples just above, I used the function `sum` to effectively count up how many samples fulfill a logical criterion. Why does this work? It works because R internally converts a logical expression, like `samples < 0.5`, to a vector of TRUE and FALSE results, one for each element of `samples`, saying whether or not each element matches the criterion. Go ahead and enter `samples < 0.5` on the R prompt, to see this for yourself. Then when you `sum` this vector of TRUE and FALSE, R counts each TRUE as 1 and each FALSE as 0. So it ends up counting how many TRUE values are in the vector, which is the same as the number of elements in `samples` that match the logical criterion.

3.2.2. Intervals of defined mass. It is more common to see scientific journals reporting an interval of defined mass, usually known as a **CONFIDENCE INTERVAL**. An interval of posterior probability, such as the ones we are working with, may instead be called a **CREDIBLE INTERVAL**, although the terms may also be used interchangeably, in the usual polysemy that arises when commonplace words are used in technical definitions. It's easy to keep track of what's being summarized, however, as long as you pay attention to how the model is defined.

These posterior intervals report two parameter values that contain between them a specified amount of posterior probability, a probability mass. For this type of interval, it is easier to find the answer by using samples from the posterior than by using a grid approximation. Suppose for example you want to know the boundaries of the lower 80% posterior probability. You know this interval starts at $p = 0$. To find out where it stops, think of the samples as data and ask where the 80th percentile lies:

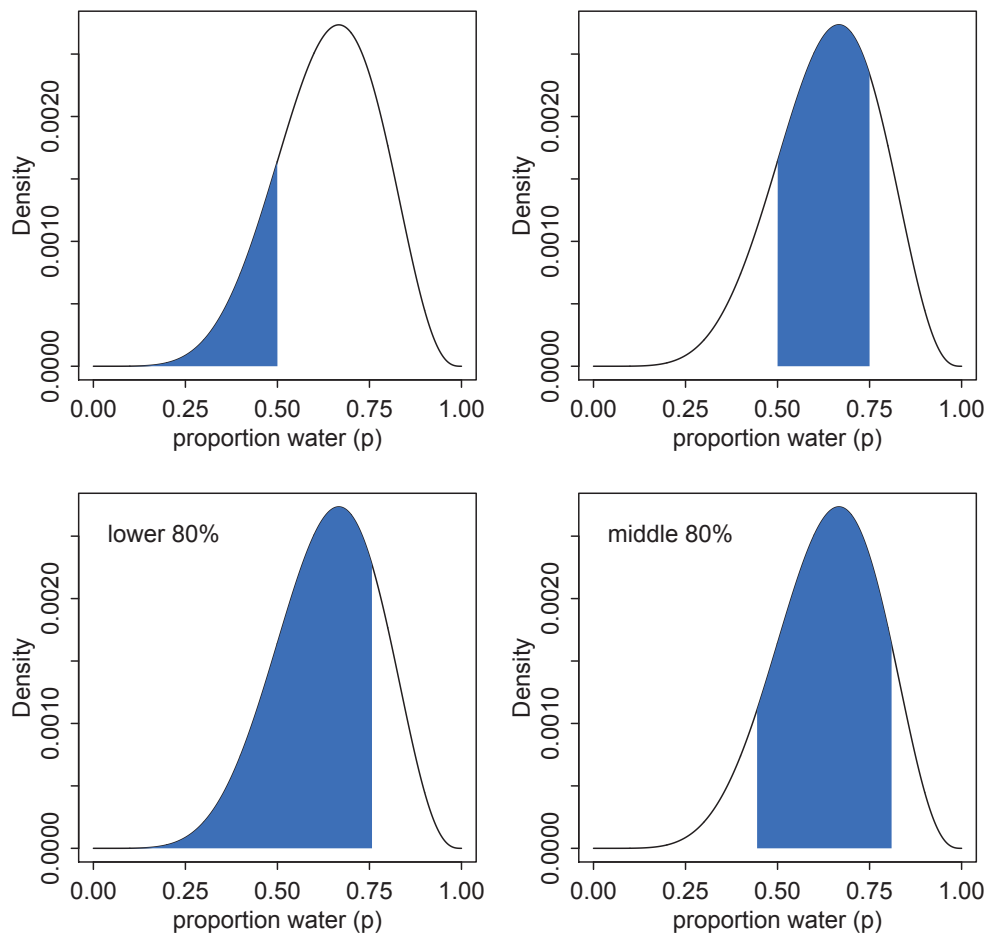


FIGURE 3.2. Two kinds of posterior interval. Top row: Intervals of defined boundaries. Top-left: The blue area is the posterior probability below a parameter value of 0.5. Top-right: The posterior probability between 0.5 and 0.75. Bottom row: Intervals of defined mass. Bottom-left: Lower 80% posterior probability exists below a parameter value of about 0.75. Bottom-right: Middle 80% posterior probability lies between the 10% and 90% quantiles.

```
quantile( samples , 0.8 )
```

R code
3.9

```
80%  
0.7607608
```

This region is shown in the bottom-left plot in [FIGURE 3.2](#). Similarly, the middle 80% interval lies between the 10th percentile and the 90th percentile. These boundaries are found using the same approach:

```
R code
3.10 quantile( samples , c( 0.1 , 0.9 ) )
```

```
      10%      90%
0.4464464 0.8118118
```

This region is shown in the bottom-right plot in [FIGURE 3.2](#).

Intervals of this sort, which assign equal probability mass to each tail, are very common in the scientific literature. We'll call them **PERCENTILE INTERVALS** (PI). These intervals do a good job of communicating the shape of a distribution, as long as the distribution isn't too asymmetrical. But in terms of supporting inferences about which parameters are consistent with the data, they are not perfect. Consider the posterior distribution and different intervals in [FIGURE 3.3](#). This posterior is consistent with observing three waters in three tosses and a uniform (flat) prior. It is highly skewed, having its maximum value at the boundary, $p = 1$. You can compute it, via grid approximation, with:

```
R code
3.11 p_grid <- seq( from=0 , to=1 , length.out=1000 )
prior <- rep(1,1000)
likelihood <- dbinom( 3 , size=3 , prob=p_grid )
posterior <- likelihood * prior
posterior <- posterior / sum(posterior)
samples <- sample( p_grid , size=1e4 , replace=TRUE , prob=posterior )
```

This code also goes ahead to sample from the posterior. Now, on the left of [FIGURE 3.3](#), the 50% percentile confidence interval is shaded. You can conveniently compute this from the samples with PI (part of *rethinking*):

```
R code
3.12 PI( samples , prob=0.5 )
```

```
      25%      75%
0.7037037 0.9329329
```

This interval assigns 25% of the probability mass above and below the interval. So it provides the central 50% probability. But in this example, it ends up excluding the most probable parameter values, near $p = 1$. So in terms of describing the shape of the posterior distribution—which is really all these intervals are asked to do—the percentile interval can be misleading.

In contrast, the right-hand plot in [FIGURE 3.3](#) displays the 50% **HIGHEST POSTERIOR DENSITY INTERVAL** (HPDI).⁵¹ The HPDI is the narrowest interval containing the specified probability mass. If you think about it, there must be an infinite number of posterior intervals with the same mass. But if you want an interval that best represents the parameter values most consistent with the data, then you want the densest of these intervals. That's what the HPDI is. Compute it from the samples with HPDI (also part of *rethinking*):

```
R code
3.13 HPDI( samples , prob=0.5 )
```

```
      |0.5      0.5|
0.8408408 1.0000000
```

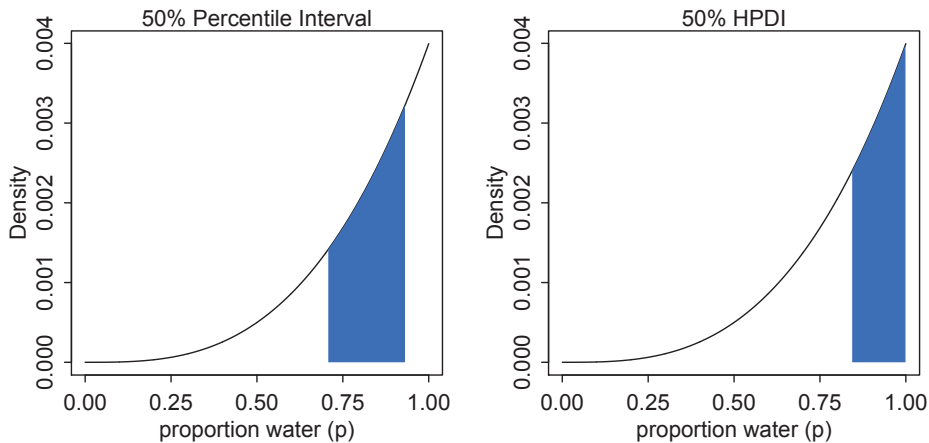


FIGURE 3.3. The difference between percentile and highest posterior density confidence intervals. The posterior density here corresponds to a flat prior and observing three water samples in three total tosses of the globe. Left: 50% percentile interval. This interval assigns equal mass (25%) to both the left and right tail. As a result, it omits the most probable parameter value, $p = 1$. Right: 50% highest posterior density interval, HPDI. This interval finds the narrowest region with 50% of the posterior probability. Such a region always includes the most probable parameter value.

This interval captures the parameters with highest posterior probability, as well as being noticeably narrower: 0.16 in width rather than 0.23 for the percentile interval.

So the HPDI has some advantages over the PI. But in most cases, these two types of interval are very similar.⁵² They only look so different in this case because the posterior distribution is highly skewed. If we instead used samples from the posterior distribution for six waters in nine tosses, these intervals would be nearly identical. Try it for yourself, using different probability masses, such as `prob=0.8` and `prob=0.95`. When the posterior is bell shaped, it hardly matters which type of interval you use. Remember, we're not launching rockets or calibrating atom smashers, so fetishizing precision to the 5th decimal place will not improve your science.

The HPDI also has some disadvantages. HPDI is more computationally intensive than PI and suffers from greater *simulation variance*, which is a fancy way of saying that it is sensitive to how many samples you draw from the posterior. It is also harder to understand and many scientific audiences will not appreciate its features, while they will immediately understand a percentile interval, as ordinary non-Bayesian intervals are nearly always percentile intervals (although of sampling distributions, not posterior distributions).

Overall, if the choice of interval type makes a big difference, then you shouldn't be using intervals to summarize the posterior. Remember, the entire posterior distribution is the Bayesian estimate. It summarizes the relative plausibilities of each possible value of the parameter. Intervals of the distribution are just helpful for summarizing it. If choice of interval leads to different inferences, then you'd be better off just plotting the entire posterior distribution.

Rethinking: Why 95%? The most common interval mass in the natural and social sciences is the 95% interval. This interval leaves 5% of the probability outside, corresponding to a 5% chance of the parameter not lying within the interval (although see below). This customary interval also reflects the customary threshold for *statistical significance*, which is 5% or $p < 0.05$. It is not easy to defend the choice of 95% (5%), outside of pleas to convention. Ronald Fisher is sometimes blamed for this choice, but his widely cited 1925 invocation of it was not enthusiastic:

“The [number of standard deviations] for which $P = .05$, or 1 in 20, is 1.96 or nearly 2; it is convenient to take this point as a limit in judging whether a deviation is to be considered significant or not.”⁵³

Most people don’t think of convenience as a serious criterion. Later in his career, Fisher actively advised against always using the same threshold for significance.⁵⁴

So what are you supposed to do then? There is no consensus, but thinking is always a good idea. If you are trying to say that an interval doesn’t include some value, then you might use the widest interval that excludes the value. Often, all confidence intervals do is communicate the shape of a distribution. In that case, a series of nested intervals may be more useful than any one interval. For example, why not present 67%, 89%, and 97% intervals, along with the median? Why these values? No reason. They are prime numbers, which makes them easy to remember. But all that matters is they be spaced enough to illustrate the shape of the posterior. And these values avoid 95%, since conventional 95% intervals encourage many readers to conduct unconscious hypothesis tests.

Rethinking: What do confidence intervals mean? It is common to hear that a 95% confidence interval means that there is a probability 0.95 that the true parameter value lies within the interval. In strict non-Bayesian statistical inference, such a statement is never correct, because strict non-Bayesian inference forbids using probability to measure uncertainty about parameters. Instead, one should say that if we repeated the study and analysis a very large number of times, then 95% of the computed intervals would contain the true parameter value. If the distinction is not entirely clear to you, then you are in good company. Most scientists find the definition of a confidence interval to be bewildering, and many of them slip unconsciously into a Bayesian interpretation.

But whether you use a Bayesian interpretation or not, a 95% interval does not contain the true value 95% of the time. The history of science teaches us that confidence intervals exhibit chronic overconfidence.⁵⁵ The word *true* should set off alarms that something is wrong with a statement like “contains the true value.” The 95% is a *small world* number (see the introduction to Chapter 2), only true in the model’s logical world. So it will never apply exactly to the real or *large world*. It is what the golem believes, but you are free to believe something else. Regardless, the width of the interval, and the values it covers, can provide valuable advice.

3.2.3. Point estimates. The third and final common summary task for the posterior is to produce point estimates of some kind. Given the entire posterior distribution, what value should you report? This seems like an innocent question, but it is difficult to answer. The Bayesian parameter estimate is precisely the entire posterior distribution, which is not a single number, but instead a function that maps each unique parameter value onto a plausibility value. So really the most important thing to note is that you don’t have to choose a point estimate. It’s hardly ever necessary.

But if you must produce a point estimate from the posterior, you’ll have to ask and answer more questions. Consider the following example. Suppose again the globe tossing experiment in which we observe 3 waters out of 3 tosses, as in [FIGURE 3.3](#). Let’s consider three alternative point estimates. First, it is very common for scientists to report the parameter

value with highest posterior probability, a *maximum a posteriori* (MAP) estimate. You can easily compute the MAP in this example:

```
p_grid[ which.max(posterior) ]
```

R code
3.14

```
[1] 1
```

Or if you instead have samples from the posterior, you can still approximate the same point:

```
chainmode( samples , adj=0.01 )
```

R code
3.15

```
[1] 0.9985486
```

But why is this point, the mode, interesting? Why not report the posterior mean or median?

```
mean( samples )  
median( samples )
```

R code
3.16

```
[1] 0.8005558
```

```
[1] 0.8408408
```

These are also point estimates, and they also summarize the posterior. But all three—the mode (MAP), mean, and median—are different in this case. How can we choose among them? [FIGURE 3.4](#) shows this posterior distribution and the locations of these point summaries.

One principled way to go beyond using the entire posterior as the estimate is to choose a **LOSS FUNCTION**. A loss function is a rule that tells you the cost associated with using any particular point estimate. While statisticians and game theorists have long been interested in loss functions, and how Bayesian inference supports them, scientists hardly ever use them explicitly. The key insight is that *different loss functions imply different point estimates*.

Here's an example to help us work through the procedure. Suppose I offer you a bet. Tell me which value of p , the proportion of water on the Earth, you think is correct. I will pay you \$100, if you get it exactly right. But I will subtract money from your gain, proportional to the distance of your decision from the correct value. Precisely, your loss is proportional to the absolute value of $d - p$, where d is your decision and p is the correct answer. We could change the precise dollar values involved, without changing the important aspects of this problem. What matters is that the loss is proportional to the distance of your decision from the true value.

Now once you have the posterior distribution in hand, how should you use it to maximize your expected winnings? It turns out that the parameter value that maximizes expected winnings (minimizes expected loss) is the median of the posterior distribution. Let's calculate that fact, without using a mathematical proof. Those interested in the proof should follow the endnote.⁵⁶

Calculating expected loss for any given decision means using the posterior to average over our uncertainty in the true value. Of course we don't know the true value, in most cases. But if we are going to use our model's information about the parameter, that means using the entire posterior distribution. So suppose we decide $p = 0.5$ will be our decision. Then the expected loss will be:

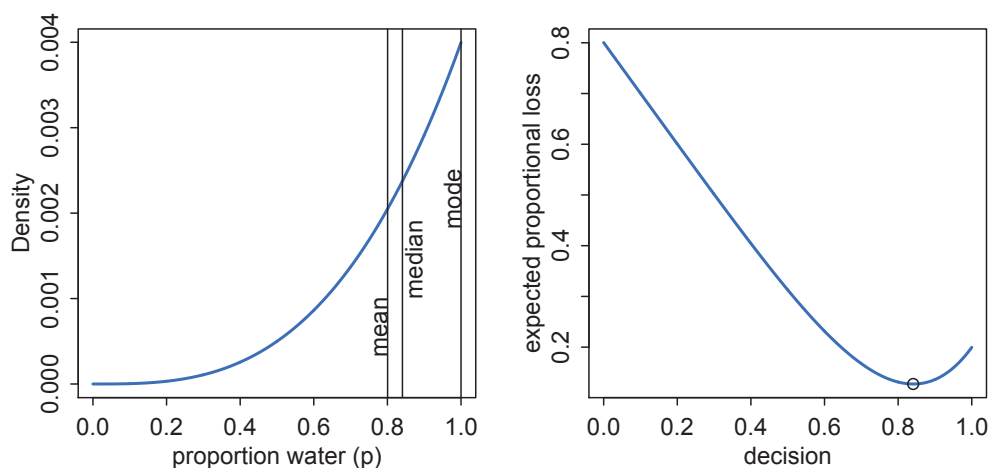


FIGURE 3.4. Point estimates and loss functions. Left: Posterior distribution (blue) after observing 3 water in 3 tosses of the globe. Vertical lines show the locations of the mode, median, and mean. Each point implies a different loss function. Right: Expected loss under the rule that loss is proportional to absolute distance of decision (horizontal axis) from the true value. The point marks the value of p that minimizes the expected loss, the posterior median.

```
R code
3.17 sum( posterior*abs( 0.5 - p_grid ) )
```

```
[1] 0.3128752
```

The symbols `posterior` and `p_grid` are the same ones we've been using throughout this chapter, containing the posterior probabilities and the parameter values, respectively. All the code above does is compute the weighted average loss, where each loss is weighted by its corresponding posterior probability. There's a trick for repeating this calculation for every possible decision, using the function `sapply`.

```
R code
3.18 loss <- sapply( p_grid , function(d) sum( posterior*abs( d - p_grid ) ) )
```

Now the symbol `loss` contains a list of loss values, one for each possible decision, corresponding the values in `p_grid`. From here, it's easy to find the parameter value that minimizes the loss:

```
R code
3.19 p_grid[ which.min(loss) ]
```

```
[1] 0.8408408
```

And this is actually the posterior median, the parameter value that splits the posterior density such that half of the mass is above it and half below it. Try `median(samples)` for comparison. It may not be exactly the same value, due to sampling variation, but it will be close.

So what are we to learn from all of this? In order to decide upon a *point estimate*, a single-value summary of the posterior distribution, we need to pick a loss function. Different loss functions nominate different point estimates. The two most common examples are the absolute loss as above, which leads to the median as the point estimate, and the quadratic loss $(d - p)^2$, which leads to the posterior mean (`mean(samples)`) as the point estimate. When the posterior distribution is symmetrical and normal-looking, then the median and mean converge to the same point, which relaxes some anxiety we might have about choosing a loss function. For the original globe tossing data (6 waters in 9 tosses), for example, the mean and median are barely different.

In principle, though, the details of the applied context may demand a rather unique loss function. Consider a practical example like deciding whether or not to order an evacuation, based upon an estimate of hurricane wind speed. Damage to life and property increases very rapidly as wind speed increases. There are also costs to ordering an evacuation when none is needed, but these are much smaller. Therefore the implied loss function is highly asymmetric, rising sharply as true wind speed exceeds our guess, but rising only slowly as true wind speed falls below our guess. In this context, the optimal point estimate would tend to be larger than posterior mean or median. Moreover, the real issue is whether or not to order an evacuation, and so producing a point estimate of wind speed may not be necessary at all.

Usually, research scientists don't think about loss functions. And so any point estimate like the mean or MAP that they may report isn't intended to support any particular decision, but rather to describe the shape of the posterior. You might argue that the decision to make is whether or not to accept an hypothesis. But the challenge then is to say what the relevant costs and benefits would be, in terms of the knowledge gained or lost.⁵⁷ Usually it's better to communicate as much as you can about the posterior distribution, as well as the data and the model itself, so that others can build upon your work. Premature decisions to accept or reject hypotheses can cost lives.⁵⁸

It's healthy to keep these issues in mind, if only because they remind us that many of the routine questions in statistical inference can only be answered under consideration of a particular empirical context and applied purpose. Statisticians can provide general outlines and standard answers, but a motivated and attentive scientist will always be able to improve upon such general advice.

3.3. Sampling to simulate prediction

Another common job for samples from the posterior is to ease **SIMULATION** of the model's implied observations. Generating implied observations from a model is useful for at least four distinct reasons.

- (1) *Model checking.* After a model is fit to real data, it is worth simulating implied observations, to check both whether the fit worked correctly and to investigate model behavior.
- (2) *Software validation.* In order to be sure that our model fitting software is working, it helps to simulate observations under a known model and then attempt to recover the values of the parameters the data were simulated under.
- (3) *Research design.* If you can simulate observations from your hypothesis, then you can evaluate whether the research design can be effective. In a narrow sense, this means doing *power analysis*, but the possibilities are much broader.

- (4) *Forecasting*. Estimates can be used to simulate new predictions, for new cases and future observations. These forecasts can be useful as applied prediction, but also for model criticism and revision.

In this final section of the chapter, we'll look at how to produce simulated observations and how to perform some simple model checks.

3.3.1. Dummy data. Let's summarize the globe tossing model that you've been working with for two chapters now. A fixed true proportion of water p exists, and that is the target of our inference. Tossing the globe in the air and catching it produces observations of "water" and "land" that appear in proportion to p and $1 - p$, respectively.

Now note that these assumptions not only allow us to infer the plausibility of each possible value of p , after observation. That's what you did in the previous chapter. These assumptions also allow us to simulate the observations that the model implies. They allow this, because likelihood functions work in both directions. Given a realized observation, the likelihood function says how plausible the observation is. And given only the parameters, the likelihood defines a distribution of possible observations that we can sample from, to simulate observation. In this way, Bayesian models are always *generative*, capable of simulating predictions. Many non-Bayesian models are also generative, but many are not.

We will call such simulated data **DUMMY DATA**, to indicate that it is a stand-in for actual data. With the globe tossing model, the dummy data arises from a binomial likelihood:

$$\Pr(w|n, p) = \frac{n!}{w!(n-w)!} p^w (1-p)^{n-w}$$

where w is an observed count of "water" and n is the number of tosses. Suppose $n = 2$, two tosses of the globe. Then there are only three possible observations: 0 water, 1 water, 2 water. You can quickly compute the likelihood of each, for any given value of p . Let's use $p = 0.7$, which is just about the true proportion of water on the Earth:

```
R code 3.20 dbinom( 0:2 , size=2 , prob=0.7 )
```

```
[1] 0.09 0.42 0.49
```

This means that there's a 9% chance of observing $w = 0$, a 42% chance of $w = 1$, and a 49% chance of $w = 2$. If you change the value of p , you'll get a different distribution of implied observations.

Now we're going to simulate observations, using these likelihoods. This is done by sampling from the distribution just described above. You could use `sample` to do this, but R provides convenient sampling functions for all the ordinary probability distributions, like the binomial. So a single dummy data observation of w can be sampled with:

```
R code 3.21 rbinom( 1 , size=2 , prob=0.7 )
```

```
[1] 1
```

The "r" in `rbinom` stands for "random." It can also generate more than one simulation at a time. A set of 10 simulations can be made by:

```
R code 3.22 rbinom( 10 , size=2 , prob=0.7 )
```

```
[1] 2 2 2 1 2 1 1 1 0 2
```

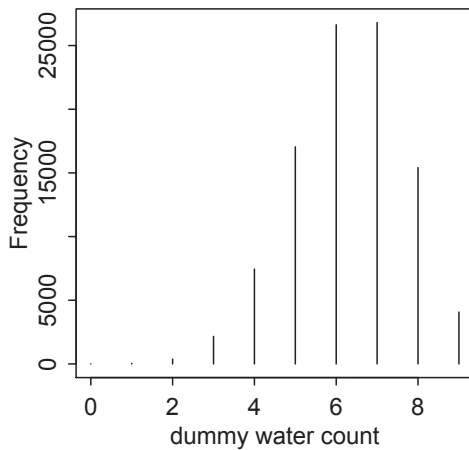


FIGURE 3.5. Distribution of simulated sample observations from 9 tosses of the globe. These samples assume the proportion of water is 0.7.

Let's generate 100,000 dummy observations, just to verify that each value (0, 1, or 2) appears in proportion to its likelihood:

```
dummy_w <- rbinom( 1e5 , size=2 , prob=0.7 )
table(dummy_w)/1e5
```

R code
3.23

```
dummy_w
      0      1      2
0.08904 0.41948 0.49148
```

And those values are very close to the analytically calculated likelihoods further up. You will see slightly different values, due to simulation variance. Execute the code above multiple times, to see how the exact realized frequencies fluctuate from simulation to simulation.

Only two tosses of the globe isn't much of a sample, though. So now let's simulate the same sample size as before, 9 tosses.

```
dummy_w <- rbinom( 1e5 , size=9 , prob=0.7 )
simplehist( dummy_w , xlab="dummy water count" )
```

R code
3.24

The resulting plot is shown in [FIGURE 3.5](#). Notice that most of the time the expected observation does not contain water in its true proportion, 0.7. That's the nature of observation: There is a one-to-many relationship between data and data-generating processes. You should experiment with sample size, the `size` input in the code above, as well as the `prob`, to see how the distribution of simulated samples changes shape and location.

So that's how to perform a basic simulation of observations. What good is this? There are many useful jobs for these samples. In this chapter, we'll put them to use in examining the implied predictions of a model. But to do that, we'll have to combine them with samples from the posterior distribution. That's next.

Rethinking: Sampling distributions. Many readers will already have seen simulated observations. **SAMPLING DISTRIBUTIONS** are the foundation of common non-Bayesian statistical traditions. In those approaches, inference about parameters is made through the sampling distribution. In this

book, inference about parameters is never done directly through a sampling distribution. The posterior distribution is not sampled, but deduced logically. Then samples can be drawn from the posterior, as earlier in this chapter, to aid in inference. In neither case is “sampling” a physical act. In both cases, it’s just a mathematical device and produces only *small world* (Chapter 2) numbers.

3.3.2. Model checking. **MODEL CHECKING** means (1) ensuring the model fitting worked correctly and (2) evaluating the adequacy of a model for some purpose. Since Bayesian models are always *generative*, able to simulate observations as well as estimate parameters from observations, once you condition a model on data, you can simulate to examine the model’s empirical expectations.

3.3.2.1. Did the software work? In the simplest case, we can check whether the software worked by checking for correspondence between implied predictions and the data used to fit the model. You might also call these implied predictions *retrodictions*, as they ask how well the model reproduces the data used to educate it. An exact match is neither expected nor desired. But when there is no correspondence at all, it probably means the software did something wrong.

There is no way to really be sure that software works correctly. Even when the retrodictions correspond to the observed data, there may be subtle mistakes. And when you start working with multilevel models, you’ll have to expect a certain pattern of lack of correspondence between retrodictions and observations. Despite there being no perfect way to ensure software has worked, the simple check I’m encouraging here often catches silly mistakes, mistakes of the kind everyone makes from time to time.

In the case of the globe tossing analysis, the software implementation is simple enough that it can be checked against analytical results. So instead let’s move directly to considering the model’s adequacy.

3.3.2.2. Is the model adequate? After assessing whether the posterior distribution is the correct one, because the software worked correctly, it’s useful to also look for aspects of the data that are not well described by the model’s expectations. The goal is not to test whether the model’s assumptions are “true,” because all models are false. Rather, the goal is to assess exactly how the model fails to describe the data, as a path towards model comprehension, revision, and improvement.

All models fail in some respect, so you have to use your judgment—as well as the judgments of your colleagues—to decide whether any particular failure is or is not important. Few scientists want to produce models that do nothing more than re-describe existing samples. So imperfect prediction (retrodiction) is not a bad thing. Typically we hope to either predict future observations or understand enough that we might usefully tinker with the world. We’ll consider these problems again, in Chapter 6.

For now, we need to learn how to combine sampling of simulated observations, as in the previous section, with sampling parameters from the posterior distribution. We expect to do better when we use the entire posterior distribution, not just some point estimate derived from it. Why? Because there is a lot of information about uncertainty in the entire posterior distribution. We lose this information when we pluck out a single parameter value and then perform calculations with it. This loss of information leads to overconfidence.

Let’s do some basic model checks, using simulated observations for the globe tossing model. The observations in our example case are counts of water, over tosses of the globe.

The implied predictions of the model are uncertain in two ways, and it's important to be aware of both.

First, there is observation uncertainty. For any unique value of the parameter p , there is a unique implied pattern of observations that the model expects. These patterns of observations are the same gardens of forking data that you explored in the previous chapter. These patterns are also what you sampled in the previous section. There is uncertainty in the predicted observations, because even if you know p with certainty, you won't know the next globe toss with certainty (unless $p = 0$ or $p = 1$).

Second, there is uncertainty about p . The posterior distribution over p embodies this uncertainty. And since there is uncertainty about p , there is uncertainty about everything that depends upon p . The uncertainty in p will interact with the sampling variation, when we try to assess what the model tells us about outcomes.

We'd like to *propagate* the parameter uncertainty—carry it forward—as we evaluate the implied predictions. All that is required is averaging over the posterior density for p , while computing the predictions. For each possible value of the parameter p , there is an implied distribution of outcomes. So if you were to compute the sampling distribution of outcomes at each value of p , then you could average all of these prediction distributions together, using the posterior probabilities of each value of p , to get a **POSTERIOR PREDICTIVE DISTRIBUTION**.

FIGURE 3.6 illustrates this averaging. At the top, the posterior distribution is shown, with 10 unique parameter values highlighted by the vertical lines. The implied distribution of observations specific to each of these parameter values is shown in the middle row of plots. Observations are never certain for any value of p , but they do shift around in response to it. Finally, at the bottom, the sampling distributions for all values of p are combined, using the posterior probabilities to compute the weighted average frequency of each possible observation, zero to nine water samples.

The resulting distribution is for predictions, but it incorporates all of the uncertainty embodied in the posterior distribution for the parameter p . As a result, it is honest. While the model does a good job of predicting the data—the most likely observation is indeed the observed data—predictions are still quite spread out. If instead you were to use only a single parameter value to compute implied predictions, say the most probable value at the peak of posterior distribution, you'd produce an overconfident distribution of predictions, narrower than the posterior predictive distribution in **FIGURE 3.6** and more like the sampling distribution shown for $p = 0.6$ in the middle row. The usual effect of this overconfidence will be to lead you to believe that the model is more consistent with the data than it really is—the predictions will cluster around the observations more tightly. This illusion arises from tossing away uncertainty about the parameters.

So how do you actually do the calculations? To simulate predicted observations for a single value of p , say $p = 0.6$, you can use `rbinom` to generate random binomial samples:

```
w <- rbinom( 1e4 , size=9 , prob=0.6 )
```

R code
3.25

This generates 10,000 (1e4) simulated predictions of 9 globe tosses (`size=9`), assuming $p = 0.6$. The predictions are stored as counts of water, so the theoretical minimum is zero and the theoretical maximum is nine. You can use `simplehist(w)` (in the `rethinking` package) to get a clean histogram of your simulated outcomes.

All you need to propagate parameter uncertainty into these predictions is replace the value 0.6 with samples from the posterior:

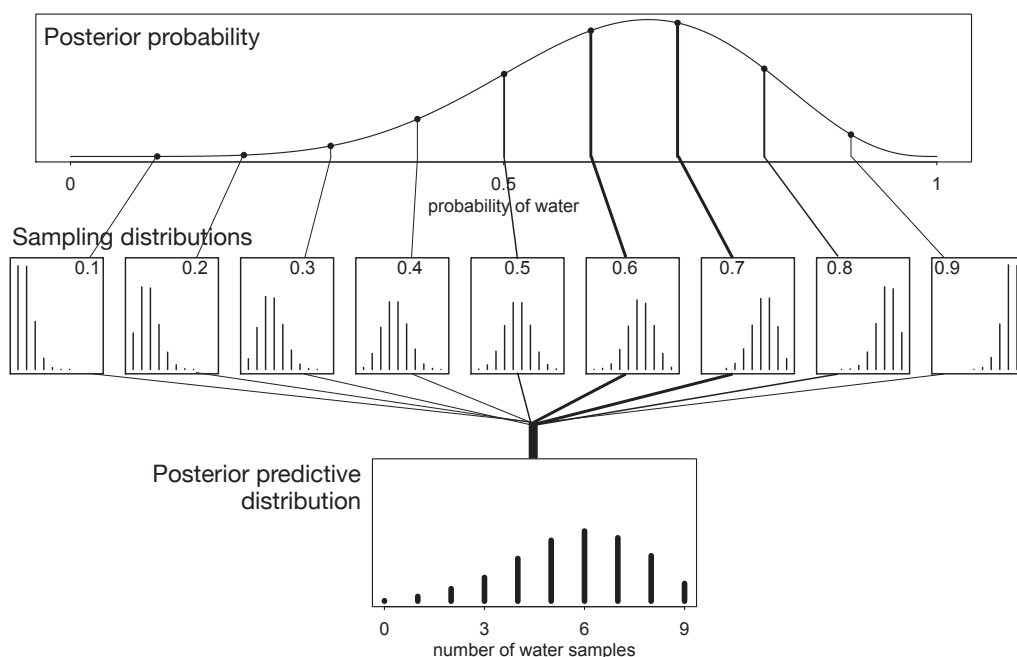


FIGURE 3.6. Simulating predictions from the total posterior. Top: The familiar posterior distribution for the globe tossing data. Ten example parameter values are marked by the vertical lines. Values with greater posterior probability indicated by thicker lines. Middle row: Each of the ten parameter values implies a unique sampling distribution of predictions. Bottom: Combining simulated observation distributions for all parameter values (not just the ten shown), each weighted by its posterior probability, produces the posterior predictive distribution. This distribution propagates uncertainty about parameter to uncertainty about prediction.

R code
3.26 `w <- rbinom( 1e4 , size=9 , prob=samples )`

The symbol `samples` above is the same list of random samples from the posterior distribution that you've used in previous sections. For each sampled value, a random binomial observation is generated. Since the sampled values appear in proportion to their posterior probabilities, the resulting simulated observations are averaged over the posterior. You can manipulate these simulated observations just like you manipulate samples from the posterior—you can compute intervals and point statistics using the same procedures. If you plot these samples, you'll see the distribution shown in the right-hand plot in [FIGURE 3.6](#).

The simulated model predictions are quite consistent with the observed data in this case—the actual count of 6 lies right in the middle of the simulated distribution. There is quite a lot of spread to the predictions, but a lot of this spread arises from the binomial process itself, not uncertainty about p . Still, it'd be premature to conclude that the model is perfect. So far, we've only viewed the data just as the model views it: Each toss of the globe is completely independent of the others. This assumption is questionable. Unless the person

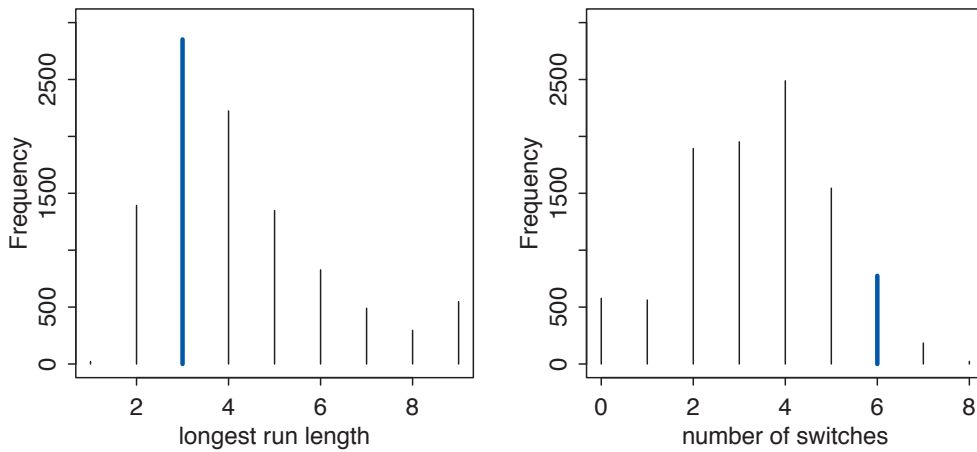


FIGURE 3.7. Alternative views of the same posterior predictive distribution (see [FIGURE 3.6](#)). Instead of considering the data as the model saw it, as a sum of water samples, now we view the data as both the length of the maximum run of water or land (left) and the number of switches between water and land samples (right). Observed values highlighted in blue. While the simulated predictions are consistent with the run length (3 water in a row), they are much less consistent with the frequent switches (6 switches in 9 tosses).

tossing the globe is careful, it is easy to induce correlations and therefore patterns among the sequential tosses. Consider for example that about half of the globe (and planet) is covered by the Pacific Ocean. As a result, water and land are not uniformly distributed on the globe, and therefore unless the globe spins and rotates enough while in the air, the position when tossed could easily influence the sample once it lands. The same problem arises in coin tosses, and indeed skilled individuals can influence the outcome of a coin toss, by exploiting the physics of it.⁵⁹

So with the goal of seeking out aspects of prediction in which the model fails, let's look at the data in two different ways. Recall that the sequence of nine tosses was W L W W L W L W. First, consider the length of the longest run of either water or land. This will provide a crude measure of correlation between tosses. So in the observed data, the longest run is 3 W's. Second, consider the number of times in the data that the sample switches from water to land or from land to water. This is another measure of correlation between samples. In the observed data, the number of switches is 6. There is nothing special about these two new ways of describing the data. They just serve to inspect the data in new ways. In your own modeling, you'll have to imagine aspects of the data that are relevant in your context, for your purposes.

[FIGURE 3.7](#) shows the simulated predictions, viewed in these two new ways. On the left, the length of the longest run of water or land is plotted, with the observed value of 3 highlighted by the bold line. Again, the true observation is the most common simulated observation, but with a lot of spread around it. On the right, the number of switches from water to land and land to water is shown, with the observed value of 6 highlighted in bold. Now

the simulated predictions appear less consistent with the data, as the majority of simulated observations have fewer switches than were observed in the actual sample. This is consistent with lack of independence between tosses of the globe, in which each toss is negatively correlated with the last.

Does this mean that the model is bad? That depends. The model will always be wrong in some sense, be *mis-specified*. But whether or not the mis-specification should lead us to try other models will depend upon our specific interests. In this case, if tosses do tend to switch from W to L and L to W, then each toss will provide less information about the true coverage of water on the globe. In the long run, even the wrong model we've used throughout the chapter will converge on the correct proportion. But it will do so more slowly than the posterior distribution may lead us to believe.

Rethinking: What does more extreme mean? A common way of measuring deviation of observation from model is to count up the tail area that includes the observed data and any more extreme data. Ordinary p -values are an example of such a tail-area probability. When comparing observations to distributions of simulated predictions, as in [FIGURE 3.6](#) and [FIGURE 3.7](#), we might wonder how far out in the tail the observed data must be before we conclude that the model is a poor one. Because statistical contexts vary so much, it's impossible to give a universally useful answer.

But more importantly, there are usually very many ways to view data and define "extreme." Ordinary p -values view the data in just the way the model expects it, and so provide a very weak form of model checking. For example, the far-right plot in [FIGURE 3.6](#) evaluates model fit in the best way for the model. Alternative ways of defining "extreme" may provide a more serious challenge to a model. The different definitions of extreme in [FIGURE 3.7](#) can more easily embarrass it.

Model fitting remains an objective procedure—everyone and every golem conducts Bayesian updating in a way that doesn't depend upon personal preferences. But model checking is inherently subjective, and this actually allows it to be quite powerful, since subjective knowledge of an empirical domain provides expertise. Expertise in turn allows for imaginative checks of model performance. Since golems have terrible imaginations, we need the freedom to engage our own imaginations. In this way, the objective and subjective work together.⁶⁰

3.4. Summary

This chapter introduced the basic procedures for manipulating posterior distributions. Our fundamental tool is samples of parameter values drawn from the posterior distribution. Working with samples transforms a problem of integral calculus into a problem of data summary. These samples can be used to produce intervals, point estimates, posterior predictive checks, as well as other kinds of simulations.

Posterior predictive checks combine uncertainty about parameters, as described by the posterior distribution, with uncertainty about outcomes, as described by the assumed likelihood function. These checks are useful for verifying that your software worked correctly. They are also useful for prospecting for ways in which your models are inadequate.

Once models become more complex, posterior predictive simulations will be used for a broader range of applications. Even understanding a model often requires simulating implied observations. We'll keep working with samples from the posterior, to make these tasks as easy and customizable as possible.

3.5. Practice

Easy. These problems use the samples from the posterior distribution for the globe tossing example. This code will give you a specific set of samples, so that you can check your answers exactly.

```
p_grid <- seq( from=0 , to=1 , length.out=1000 )
prior <- rep( 1 , 1000 )
likelihood <- dbinom( 6 , size=9 , prob=p_grid )
posterior <- likelihood * prior
posterior <- posterior / sum(posterior)
set.seed(100)
samples <- sample( p_grid , prob=posterior , size=1e4 , replace=TRUE )
```

R code
3.27

Use the values in samples to answer the questions that follow.

- 3E1. How much posterior probability lies below $p = 0.2$?
- 3E2. How much posterior probability lies above $p = 0.8$?
- 3E3. How much posterior probability lies between $p = 0.2$ and $p = 0.8$?
- 3E4. 20% of the posterior probability lies below which value of p ?
- 3E5. 20% of the posterior probability lies above which value of p ?
- 3E6. Which values of p contain the narrowest interval equal to 66% of the posterior probability?
- 3E7. Which values of p contain 66% of the posterior probability, assuming equal posterior probability both below and above the interval?

Medium.

- 3M1. Suppose the globe tossing data had turned out to be 8 water in 15 tosses. Construct the posterior distribution, using grid approximation. Use the same flat prior as before.
- 3M2. Draw 10,000 samples from the grid approximation from above. Then use the samples to calculate the 90% HPDI for p .
- 3M3. Construct a posterior predictive check for this model and data. This means simulate the distribution of samples, averaging over the posterior uncertainty in p . What is the probability of observing 8 water in 15 tosses?
- 3M4. Using the posterior distribution constructed from the new (8/15) data, now calculate the probability of observing 6 water in 9 tosses.
- 3M5. Start over at 3M1, but now use a prior that is zero below $p = 0.5$ and a constant above $p = 0.5$. This corresponds to prior information that a majority of the Earth's surface is water. Repeat each problem above and compare the inferences. What difference does the better prior make? If it helps, compare inferences (using both priors) to the true value $p = 0.7$.

Hard.

Introduction. The practice problems here all use the data below. These data indicate the gender (male=1, female=0) of officially reported first and second born children in 100 two-child families.

R code
3.28

```
birth1 <- c(1,0,0,0,1,1,0,1,0,1,0,0,1,1,0,1,1,0,0,0,1,0,0,0,1,0,
0,0,0,1,1,1,0,1,0,1,1,1,0,1,0,1,1,0,0,1,1,0,1,0,0,0,0,0,0,
1,1,0,1,0,0,1,0,0,0,1,0,0,1,1,1,1,0,1,1,1,1,0,0,1,0,1,1,0,
1,0,1,1,1,0,1,1,1,1)
birth2 <- c(0,1,0,1,0,1,1,1,0,0,1,1,1,1,0,0,1,1,1,0,0,1,1,1,0,
1,1,1,0,1,1,1,0,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,1,1,1,1,1,
1,1,1,0,1,1,0,1,1,0,1,1,1,0,0,0,0,0,1,0,0,0,1,1,0,0,1,0,1,1,
0,0,0,1,1,1,0,0,0,0)
```

So for example, the first family in the data reported a boy (1) and then a girl (0). The second family reported a girl (0) and then a boy (1). The third family reported two girls. You can load these two vectors into R's memory by typing:

R code
3.29

```
library(rethinking)
data(homeworkch3)
```

Use these vectors as data. So for example to compute the total number of boys born across all of these births, you could use:

R code
3.30

```
sum(birth1) + sum(birth2)
```

```
[1] 111
```

3H1. Using grid approximation, compute the posterior distribution for the probability of a birth being a boy. Assume a uniform prior probability. Which parameter value maximizes the posterior probability?

3H2. Using the `sample` function, draw 10,000 random parameter values from the posterior distribution you calculated above. Use these samples to estimate the 50%, 89%, and 97% highest posterior density intervals.

3H3. Use `rbinom` to simulate 10,000 replicates of 200 births. You should end up with 10,000 numbers, each one a count of boys out of 200 births. Compare the distribution of predicted numbers of boys to the actual count in the data (111 boys out of 200 births). There are many good ways to visualize the simulations, but the `dens` command (part of the `rethinking` package) is probably the easiest way in this case. Does it look like the model fits the data well? That is, does the distribution of predictions include the actual observation as a central, likely outcome?

3H4. Now compare 10,000 counts of boys from 100 simulated first borns only to the number of boys in the first births, `birth1`. How does the model look in this light?

3H5. The model assumes that sex of first and second births are independent. To check this assumption, focus now on second births that followed female first borns. Compare 10,000 simulated counts of boys to only those second births that followed girls. To do this correctly, you need to count the number of first borns who were girls and simulate that many births, 10,000 times. Compare the counts of boys in your simulations to the actual observed count of boys following girls. How does the model look in this light? Any guesses what is going on in these data?

4 Linear Models

History has been unkind to Ptolemy. Claudius Ptolemy (born 90 CE, died 168 CE) was an Egyptian mathematician and astronomer, most famous for his geocentric model of the solar system. These days, when scientists wish to mock someone, they might compare him to a supporter of the geocentric model. But Ptolemy was a genius. His mathematical model of the motions of the planets (FIGURE 4.1) was extremely accurate. To achieve its accuracy, it employed a device known as an *epicycle*, a circle on a circle. It is even possible to have epicycles, circles on circles on circles. With enough epicycles in the right places, Ptolemy's model could predict with accuracy greater than anyone had achieved before him. And so the model was utilized for over a thousand years. And Ptolemy and people like him, toiling over centuries, worked it all out without the aid of a computer. Anyone should be flattered to be compared to Ptolemy.

The trouble of course is that the geocentric model is wrong, in many respects. If you used it to plot the path of your Mars probe, you'd miss the red planet by quite a distance. But for spotting Mars in the night sky, it remains an excellent model. It would have to be re-calibrated every century or so, depending upon which heavenly body you wish to locate. But the geocentric model continues to make useful predictions, provided those predictions remain within a narrow domain of questioning.

The strategy of using epicycles might seem crazy, once you know the correct structure of the solar system. But it turns out that the ancients had hit upon a generalized system of approximation. Given enough circles embedded in enough places, the Ptolemaic strategy is the same as a *Fourier series*, a way of decomposing a periodic function (like an orbit) into a series of sine and cosine functions. So no matter the actual arrangement of planets and moons, a geocentric model can be built to describe their paths against the night sky.

LINEAR REGRESSION is the geocentric model of applied statistics. By "linear regression," we will mean a family of simple statistical golems that attempt to learn about the mean and variance of some measurement, using an additive combination of other measurements. Like geocentrism, linear regression can usefully describe a very large variety of natural phenomena. Like geocentrism, linear regression is a descriptive model that corresponds to many different process models. If we read its structure too literally, we're likely to make mistakes. But used wisely, these little linear golems continue to be useful.

This chapter introduces linear regression as a Bayesian procedure. Under a probability interpretation, which is necessary for Bayesian work, linear regression uses a Gaussian (normal) distribution to describe our golem's uncertainty about some measurement of interest. This type of model is simple, flexible, and commonplace. Like all statistical models, it is not universally useful. But linear regression has a strong claim to being foundational, in the

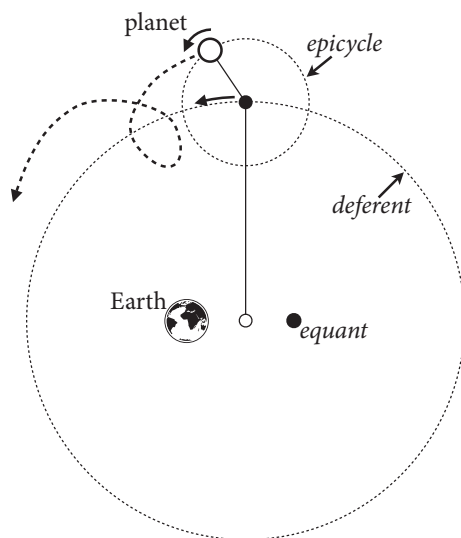


FIGURE 4.1. The Ptolemaic Universe, in which complex motion of the planets in the night sky was explained by orbits within orbits, called *epicycles*. The model is incredibly wrong, yet makes quite good predictions.

sense that once you learn to build and interpret linear regression models, you can quickly move on to other types of regression which are less normal.

4.1. Why normal distributions are normal

Suppose you and a thousand of your closest friends line up on the halfway line of a soccer field (football pitch). Each of you has a coin in your hand. At the sound of the whistle, you begin flipping the coins. Each time a coin comes up heads, that person moves one step towards the left-hand goal. Each time a coin comes up tails, that person moves one step towards the right-hand goal. Each person flips the coin 16 times, follows the implied moves, and then stands still. Now we measure the distance of each person from the halfway line. Can you predict what proportion of the thousand people who are standing on the halfway line? How about the proportion 5 yards left of the line?

It's hard to say where any individual person will end up, but you can say with great confidence what the collection of positions will be. The distances will be distributed in approximately normal, or Gaussian, fashion. This is true even though the underlying distribution is binomial. It does this because there are so many more possible ways to realize a sequence of left-right steps that sums to zero. There are slightly fewer ways to realize a sequence that ends up one step left or right of zero, and so on, with the number of possible sequences declining in the characteristic bell curve of the normal distribution.

4.1.1. Normal by addition. Let's see this result, by simulating this experiment in R. To show that there's nothing special about the underlying coin flip, assume instead that each step is different from all the others, a random distance between zero and one yard. Thus a coin is flipped, a distance between zero and one yard is taken in the indicated direction, and the process repeats. To simulate this, we generate for each person a list of 16 random numbers between -1 and 1 . These are the individual steps. Then we add these steps together to get the position after 16 steps. Then we need to replicate this procedure 1000 times. This is the sort of task that would be harrowing in a point-and-click interface, but it is made trivial by the command line. Here's a single line to do the whole thing:

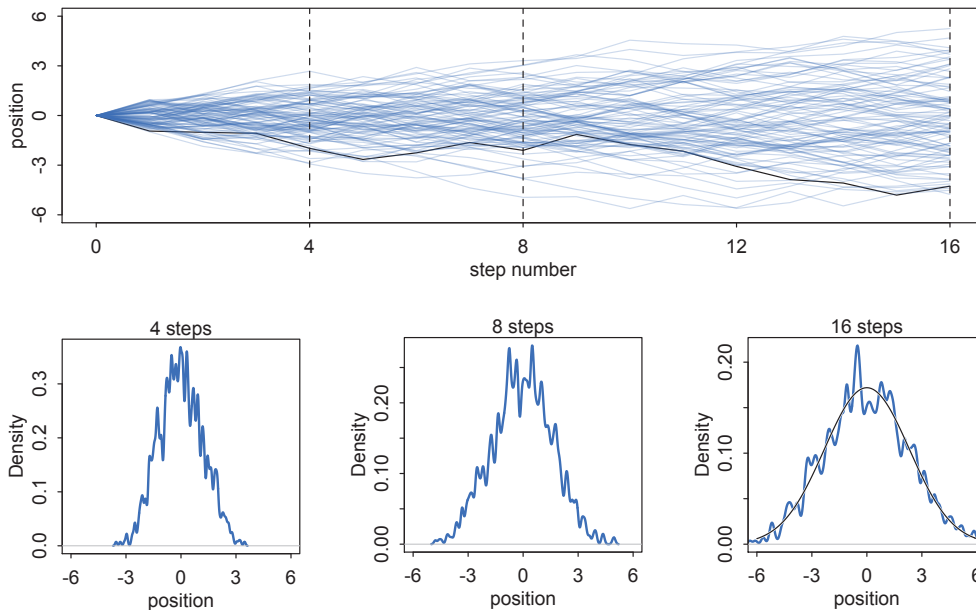


FIGURE 4.2. Random walks on the soccer field converge to a normal distribution. The more steps are taken, the closer the match between the real empirical distribution of positions and the ideal normal distribution, superimposed in the last plot in the bottom panel.

```
pos <- replicate( 1000 , sum( runif(16,-1,1) ) )
```

R code
4.1

You can plot the distribution of final positions in a number of different ways, including `hist(pos)` and `plot(density(pos))`. In [FIGURE 4.2](#), I show the result of these random walks and how their distribution evolves as the number of steps increases. The top panel plots 100 different, independent random walks, with one highlighted in black. The vertical dashes indicate the locations corresponding to the distribution plots underneath, measured after 4, 8, and 16 steps. Although the distribution of positions starts off seemingly idiosyncratic, after 16 steps, it has already taken on a familiar outline. The familiar “bell” curve of the Gaussian distribution is emerging from the randomness. Go ahead and experiment with even larger numbers of steps to verify for yourself that the distribution of positions is stabilizing on the Gaussian. You can square the step sizes and transform them in a number of arbitrary ways, without changing the result: Normality emerges. Where does it come from?

Any process that adds together random values from the same distribution converges to a normal. But it’s not easy to grasp why addition should result in a bell curve of sums.⁶¹ Here’s a conceptual way to think of the process. Whatever the average value of the source distribution, each sample from it can be thought of as a fluctuation from that average value. When we begin to add these fluctuations together, they also begin to cancel one another out. A large positive fluctuation will cancel a large negative one. The more terms in the sum, the more chances for each fluctuation to be canceled by another, or by a series of smaller ones in the opposite direction. So eventually the most likely sum, in the sense that there are the

most ways to realize it, will be a sum in which every fluctuation is canceled by another, a sum of zero (relative to the mean).⁶²

It doesn't matter what shape the underlying distribution possesses. It could be uniform, like in our example above, or it could be (nearly) anything else.⁶³ Depending upon the underlying distribution, the convergence might be slow, but it will be inevitable. Often, as in this example, convergence is rapid.

4.1.2. Normal by multiplication. Here's another way to get a normal distribution. Suppose the growth rate of an organism is influenced by a dozen loci, each with several alleles that code for more growth. Suppose also that all of these loci interact with one another, such that each increase growth by a percentage. This means that their effects multiply, rather than add. For example, we can sample a random growth rate for this example with this line of code:

R code
4.2 `prod( 1 + runif(12,0,0.1) )`

This code just samples 12 random numbers between 1.0 and 1.1, each representing a proportional increase in growth. Thus 1.0 means no additional growth and 1.1 means a 10% increase. The product of all 12 is computed and returned as output. Now what distribution do you think these random products will take? Let's generate 10,000 of them and see:

R code
4.3 `growth <- replicate(10000 , prod(1 + runif(12,0,0.1)))
dens(growth , norm.comp=TRUE)`

The reader should execute this code in R and see that the distribution is approximately normal again. I said normal distributions arise from summing random fluctuations, which is true. But the effect at each locus was multiplied by the effects at all the others, not added. So what's going on here?

We again get convergence towards a normal distribution, because the effect at each locus is quite small. Multiplying small numbers is approximately the same as addition. For example, if there are two loci with alleles increasing growth by 10% each, the product is:

$$1.1 \times 1.1 = 1.21$$

We could also approximate this product by just adding the increases, and be off by only 0.01:

$$1.1 \times 1.1 = (1 + 0.1)(1 + 0.1) = 1 + 0.2 + 0.01 \approx 1.2$$

The smaller the effect of each locus, the better this additive approximation will be. In this way, small effects that multiply together are approximately additive, and so they also tend to stabilize on Gaussian distributions. Verify this for yourself by comparing:

R code
4.4 `big <- replicate(10000 , prod(1 + runif(12,0,0.5)))
small <- replicate(10000 , prod(1 + runif(12,0,0.01)))`

The interacting growth deviations, as long as they are sufficiently small, converge to a Gaussian distribution. In this way, the range of casual forces that tend towards Gaussian distributions extends well beyond purely additive interactions.

4.1.3. Normal by log-multiplication. But wait, there's more. Large deviates that are multiplied together do not produce Gaussian distributions, but they do tend to produce Gaussian distributions on the log scale. For example:

```
log.big <- replicate( 10000 , log(prod(1 + runif(12,0,0.5))) )
```

R code
4.5

Yet another Gaussian distribution. We get the Gaussian distribution back, because adding logs is equivalent to multiplying the original numbers. So even multiplicative interactions of large deviations can produce Gaussian distributions, once we measure the outcomes on the log scale. Since measurement scales are arbitrary, there's nothing suspicious about this transformation. After all, it's natural to measure sound and earthquakes and even information (Chapter 6) on a log scale.

4.1.4. Using Gaussian distributions. We're going to spend the rest of this chapter using the Gaussian distribution as a skeleton for our hypotheses, building up models of measurements as aggregations of normal distributions. The justifications for using the Gaussian distribution fall into two broad categories: (1) ontological and (2) epistemological.

4.1.4.1. Ontological justification. The world is full of Gaussian distributions, approximately. As a mathematical idealization, we're never going to experience a perfect Gaussian distribution. But it is a widespread pattern, appearing again and again at different scales and in different domains. Measurement errors, variations in growth, and the velocities of molecules all tend towards Gaussian distributions. These processes do this because at their heart, these processes add together fluctuations. And repeatedly adding finite fluctuations results in a distribution of sums that have shed all information about the underlying process, aside from mean and spread.

One consequence of this is that statistical models based on Gaussian distributions cannot reliably identify micro-process. This recalls the modeling philosophy from Chapter 1 (page 6). But it also means that these models can do useful work, even when they cannot identify process. If we had to know the development biology of height before we could build a statistical model of height, human biology would be sunk.

There are many other patterns in nature, so make no mistake in assuming that the Gaussian pattern is universal. In later chapters, we'll see how other useful and common patterns, like the exponential and gamma and Poisson, also arise from natural processes. The Gaussian is a member of a family of fundamental natural distributions known as the **EXPONENTIAL FAMILY**. All of the members of this family are important for working science, because they populate our world.

4.1.4.2. Epistemological justification. But the natural occurrence of the Gaussian distribution is only one reason to build models around it. Another route to justifying the Gaussian as our choice of skeleton, and a route that will help us appreciate later why it is often a poor choice, is that it represents a particular state of ignorance. When all we know or are willing to say about a distribution of measures (measures are continuous values on the real number line) is their mean and variance, then the Gaussian distribution arises as the most consistent with our assumptions.

That is to say that the Gaussian distribution is the most natural expression of our state of ignorance, because if all we are willing to assume is that a measure has finite variance, the Gaussian distribution is the shape that can be realized in the largest number of ways and does not introduce any new assumptions. It is the least surprising and least informative

assumption to make. In this way, the Gaussian is the distribution most consistent with our assumptions. Or rather, it is the most consistent with our golem's assumptions. If you don't think the distribution should be Gaussian, then that implies that you know something else that you should tell your golem about, something that would improve inference.

This epistemological justification is premised on **INFORMATION THEORY** and **MAXIMUM ENTROPY**. We'll dwell on information theory in Chapter 6 and maximum entropy in Chapter 9. Then in later chapters, other common and useful distributions will be used to build *generalized linear models* (GLMs). When these other distributions are introduced, you'll learn the constraints that make them the uniquely most appropriate (consistent with our assumptions) distributions.

For now, let's take the ontological and epistemological justifications of just the Gaussian distribution as reasons to start building models of measures around it. Throughout all of this modeling, keep in mind that using a model is not equivalent to swearing an oath to it. The golem is your servant, not the other way around. And so the golem's beliefs are not your own. There is no contract that forces us to believe the assumptions of our models. They just need to be useful robots.

Overthinking: Gaussian distribution. You don't have to memorize the Gaussian probability distribution formula to make good use of it. Your computer already knows it. But a little knowledge of its form can help demystify it. The probability *density* (see below) of some value y , given a Gaussian (normal) distribution with mean μ and standard deviation σ , is:

$$p(y|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

This looks monstrous. But the important bit is just the $(y - \mu)^2$ bit. This is the part that gives the normal distribution its fundamental shape, a quadratic shape. Once you exponentiate the quadratic shape, you get the classic bell curve. The rest of it just scales and standardizes the distribution so that it sums to one, as all probability distributions must. But an expression as simple as $\exp(-y^2)$ yields the Gaussian prototype.

The Gaussian is also a continuous distribution, unlike the binomial probabilities of earlier chapters. This means that the value y in the Gaussian distribution can be any continuous value. The binomial, in contrast, requires integers. Probability distributions with only discrete outcomes, like the binomial, are usually called *probability mass* functions and denoted Pr . Continuous ones like the Gaussian are called *probability density* functions, denoted with p or just plain old f , depending upon author and tradition. For mathematical reasons, probability densities, but not masses, can be greater than 1. Try `dnorm(0, 0, 0.1)`, for example, which is the way to make R calculate $p(0|0, 0.1)$. The answer, about 4, is no mistake. Probability *density* is the rate of change in cumulative probability. So where cumulative probability is increasing rapidly, density can easily exceed 1. But if we calculate the area under the density function, it will never exceed 1. Such areas are also called *probability mass*.

You can usually ignore all these density/mass details while doing computational work. In the conceptual parts of this book, I'll treat them identically. But it's good to be aware of the distinction. Sometimes the difference matters.

The Gaussian distribution is routinely seen without σ but with another parameter, τ . The parameter τ in this context is usually called *precision* and defined as $\tau = 1/\sigma^2$. This change of parameters gives us the equivalent formula (just substitute $\sigma = 1/\sqrt{\tau}$):

$$p(y|\mu, \tau) = \sqrt{\frac{\tau}{2\pi}} \exp\left(-\frac{1}{2}\tau(y - \mu)^2\right)$$

This form is common in Bayesian data analysis, and Bayesian model fitting software, such as BUGS or JAGS, sometimes requires using τ rather than σ .

4.2. A language for describing models

This book adopts a standard language for describing and coding statistical models. You find this language in many statistical texts and in nearly all statistical journals, as it is general to both Bayesian and non-Bayesian modeling. Scientists increasingly use this same language to describe their statistical methods, as well. So learning this language is an investment, no matter where you are headed next.

Here's the approach, in abstract. There will be many examples later. These numbered items describe the choices that will be encoded in your model description.

- (1) First, we recognize a set of measurements that we hope to predict or understand, the *outcome* variable or variables.
- (2) For each of these outcome variables, we define a likelihood distribution that defines the plausibility of individual observations. In linear regression, this distribution is always Gaussian.
- (3) Then we recognize a set of other measurements that we hope to use to predict or understand the outcome. Call these *predictor* variables.
- (4) We relate the exact shape of the likelihood distribution—its precise location and variance and other aspects of its shape, if it has them—to the predictor variables. In choosing a way to relate the predictors to the outcomes, we are forced to name and define all of the parameters of the model.
- (5) Finally, we choose priors for all of the parameters in the model. These priors define the initial information state of the model, before seeing the data.

After all these decisions are made—and most of them will come to seem automatic to you before long—we summarize the model with something mathy like:

$$\begin{aligned}\text{outcome}_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \beta \times \text{predictor}_i \\ \beta &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{HalfCauchy}(0, 1)\end{aligned}$$

If that doesn't make much sense, good. That indicates that you are holding the right textbook, since this book teaches you how to read and write these mathematical model descriptions. We won't do any mathematical manipulation of them. Instead, they provide an unambiguous way to define and communicate our models. Once you get comfortable with their grammar, when you start reading these mathematical descriptions in other books or in scientific journals, you'll find them less obtuse.

The approach above surely isn't the only way to describe statistical modeling, but it is a widespread and productive language. Once a scientist learns this language, it becomes easier to communicate the assumptions of our models. We no longer have to remember seemingly arbitrary lists of bizarre conditions like *homoscedasticity* (constant variance), because we can just read these conditions from the model definitions. We will also be able to see natural ways to change these assumptions, instead of feeling trapped within some procrustean model type, like regression or multiple regression or ANOVA or ANCOVA or such. These are all the same kind of model, and that fact becomes obvious once we know how to talk about models as mappings of one set of variables through a probability distribution onto another set of variables. Fundamentally, these models define the ways values of some variables can arise, given values of other variables (Chapter 2).

4.2.1. Re-describing the globe tossing model. It's good to work with examples. Recall the proportion of water problem from previous chapters. The model in that case was always:

$$\begin{aligned}w &\sim \text{Binomial}(n, p) \\ p &\sim \text{Uniform}(0, 1)\end{aligned}$$

where w was the observed count of water samples, n was the total number of samples, and p was the proportion of water on the actual globe. Read the above statement as:

The count w is distributed binomially with sample size n and probability p .

The prior for p is assumed to be uniform between zero and one.

Once we know the model in this way, we automatically know all of its assumptions. We know the binomial distribution assumes that each sample (globe toss) is independent of the others, and so we also know that the model assumes that sample points are independent of one another.

For now, we'll focus on simple models like the above. In these models, the first line defines the likelihood function used in Bayes' theorem. The other lines define priors. Both of the lines in this model are **STOCHASTIC**, as indicated by the $\sim$ symbol. A stochastic relationship is just a mapping of a variable or parameter onto a distribution. It is *stochastic* because no single instance of the variable on the left is known with certainty. Instead, the mapping is probabilistic: Some values are more plausible than others, but very many different values are plausible under any model. Later, we'll have models with deterministic definitions in them as well.

Overthinking: From model definition to Bayes' theorem. To relate the mathematical format above to Bayes' theorem, you could use the model definition to define the posterior distribution:

$$\Pr(p|w, n) = \frac{\text{Binomial}(w|n, p)\text{Uniform}(p|0, 1)}{\int \text{Binomial}(w|n, p)\text{Uniform}(p|0, 1)dp}$$

That monstrous denominator is just the average likelihood again. It standardizes the posterior to sum to 1. The action is in the numerator, where the posterior probability of any particular value of p is seen again to be proportional to the product of the likelihood and prior. In R code form, this is the same grid approximation calculation you've been using all along. In a form recognizable as the above expression:

```
R code
4.6 w <- 6; n <- 9;
    p_grid <- seq(from=0, to=1, length.out=100)
    posterior <- dbinom(w, n, p_grid) * dunif(p_grid, 0, 1)
    posterior <- posterior / sum(posterior)
```

Compare to the calculations in earlier chapters.

4.3. A Gaussian model of height

Let's build a linear regression model now. Well, it'll be a "regression" once we have a predictor variable in it. For now, we'll get the scaffold in place and construct the predictor variable in the next section.

We'll work through this material by using real sets of data. In this case, we want a single measurement variable to model as a Gaussian distribution. There will be two parameters

describing the distribution's shape, the mean μ and the standard deviation σ . Bayesian updating will allow us to consider every possible combination of values for μ and σ and to score each combination by its relative plausibility, in light of the data. These relative plausibilities are the posterior probabilities of each combination of values μ, σ .

Another way to say the above is this. There are an infinite number of possible Gaussian distributions. Some have small means. Others have large means. Some are wide, with a large σ . Others are narrow. We want our Bayesian machine to consider every possible distribution, each defined by a combination of μ and σ , and rank them by posterior plausibility. Posterior plausibility provides a measure of the logical compatibility of each possible distribution with the data and model.

In practice we'll use approximations to the formal analysis. So we won't really consider every possible value of μ and σ . But that won't cost us anything in most cases. Instead the thing to worry about is keeping in mind that the "estimate" here will be the entire posterior distribution, not any point within it. And as a result, the posterior distribution will be a distribution of Gaussian distributions. Yes, a distribution of distributions. If that doesn't make sense yet, then that just means you are being honest with yourself. Hold on, work hard, and it will make plenty of sense before long.

4.3.1. The data. The data contained in `data(Howell1)` are partial census data for the Dobe area !Kung San, compiled from interviews conducted by Nancy Howell in the late 1960s.⁶⁴ For the non-anthropologists reading along, the !Kung San are the most famous foraging population of the 20th century, largely because of detailed quantitative studies by people like Howell.

Load the data and place them into a convenient object with:

```
library(rethinking)
data(Howell1)
d <- Howell1
```

R code
4.7

What you have now is a *data frame* named simply `d`. I use the name `d` over and over again in this book to refer to the data frame we are working with at the moment. I keep its name short to save you typing. A *data frame* is a special kind of object in R. It is a table with named columns, corresponding to variables, and numbered rows, corresponding to individual cases. In this example, the cases are individuals. Inspect the structure of the data frame, the same way you'd inspect the structure of any symbol in R:

```
str( d )
```

R code
4.8

```
'data.frame': 544 obs. of 4 variables:
 $ height: num  152 140 137 157 145 ...
 $ weight: num  47.8 36.5 31.9 53 41.3 ...
 $ age : num  63 63 65 41 51 35 32 27 19 54 ...
 $ male : int  1 0 0 1 0 1 0 1 0 1 ...
```

This data frame contains four columns. Each column has 544 entries, so there are 544 individuals in these data. Each individual has a recorded height (centimeters), weight (kilograms), age (years), and "maleness" (0 indicating female and 1 indicating male).

We're going to work with just the `height` column, for the moment. The column containing the heights is really just a regular old R *vector*, the kind of list we have been working with in many of the code examples. You can access this vector by using its name:

R code
4.9 `d$height`

Read the symbol `$` as *extract*, as in *extract the column named `height` from the data frame `d`*. All it does is give you the column that follows it.

Overthinking: Data frames. It might seem like this whole data frame thing is kind of annoying, right now. If we're working with only one column here, why bother with this `d` thing at all? You don't have to use a data frame, as you can just pass raw vectors to every command we'll use in this book. But keeping related variables in the same data frame is a huge convenience. Once we have more than one variable, and we wish to model one as a function of the others, you'll better see the value of the data frame. You won't have to wait long.

More technically, a data frame is a special kind of `list` in R. So you access the individual variables with the usual list “double bracket” notation, like `d[[1]]` for the first variable or `d[['x']]` for the variable named `x`. Unlike regular lists, however, data frames force all variables to have the same length. That isn't always a good thing. And that's why some statistical packages, like the powerful Stan Markov chain sampler (`mc-stan.org`), accept plain lists of data, rather than proper data frames.

All we want for now are heights of adults in the sample. The reason to filter out non-adults for now is that height is strongly correlated with age, before adulthood. Later in the chapter, I'll ask you to tackle the age problem. But for now, better to postpone it. You can filter the data frame down to individuals of age 18 or greater with:

R code
4.10 `d2 <- d[ d$age >= 18 , ]`

We'll be working with the data frame `d2` now. It should have 352 rows (individuals) in it.

Overthinking: Index magic. The square bracket notation used in the code above is *index* notation. It is very powerful, but also quite compact and confusing. The data frame `d` is a matrix, a rectangular grid of values. You can access any value in the matrix with `d[row, col]`, replacing `row` and `col` with row and column numbers. If `row` or `col` are lists of numbers, then you get more than one row or column. If you leave the spot for `row` or `col` blank, then you get all of whatever you leave blank. For example, `d[ 3 , ]` gives all columns at row 3. Typing `d[ , ]` just gives you the entire matrix, because it returns all rows and all columns.

So what `d[ d$age >= 18 , ]` does is give you all of the rows in which `d$age` is greater-than-or-equal-to 18. It also gives you all of the columns, because the spot after the comma is blank. The result is stored in `d2`, the new data frame containing only adults. With a little practice, you can use this square bracket index notion to perform custom searches of your data, much like performing a database query.

4.3.2. The model. Our goal is to model these values using a Gaussian distribution. First, go ahead and plot the distribution of heights, with `dens(d2$height)`. These data look rather Gaussian in shape, as is typical of height data. This may be because height is a sum of many small growth factors. As you saw at the start of the chapter, a distribution of sums tends to

converge to a Gaussian distribution. Whatever the reason, adult heights are nearly always approximately normal.

So it's reasonable for the moment to adopt the stance that the model's likelihood should be Gaussian. But be careful about choosing the Gaussian distribution only when the plotted outcome variable looks Gaussian to you. Gawking at the raw data, to try to decide how to model them, is usually not a good idea. The data could be a mixture of different Gaussian distributions, for example, and in that case you won't be able to detect the underlying normality just by eyeballing the outcome distribution. Furthermore, as mentioned earlier in this chapter, the empirical distribution needn't be actually Gaussian in order to justify using a Gaussian likelihood.

So which Gaussian distribution? There are an infinite number of them, with an infinite number of different means and standard deviations. We're ready to write down the general model and compute the plausibility of each combination of μ and σ . To define the heights as normally distributed with a mean μ and standard deviation σ , we write:

$$h_i \sim \text{Normal}(\mu, \sigma)$$

In many books you'll see the same model written as $h_i \sim \mathcal{N}(\mu, \sigma)$, which means the same thing. The symbol h refers to the list of heights, and the subscript i means *each individual element of this list*. It is conventional to use i because it stands for *index*. The index i takes on row numbers, and so in this example can take any value from 1 to 352 (the number of heights in `d2$height`). As such, the model above is saying that all the golem knows about each height measurement is defined by the same normal distribution, with mean μ and standard deviation σ . Before long, those little i 's are going to show up on the right-hand side of the model definition, and you'll be able to see why we must bother with them. So don't ignore the i , even if it seems like useless ornamentation right now.

Rethinking: Independent and identically distributed. The short model above is sometimes described as assuming that the values h_i are *independent and identically distributed*, which may be abbreviated i.i.d., iid, or IID. You might even see the same model written:

$$h_i \stackrel{\text{iid}}{\sim} \text{Normal}(\mu, \sigma).$$

"iid" indicates that each value h_i has the same probability function, independent of the other h values and using the same parameters. A moment's reflection tells us that this is hardly ever true, in a physical sense. Whether measuring the same distance repeatedly or studying a population of heights, it is hard to argue that every measurement is independent of the others. For example, heights within families are correlated because of alleles shared through recent shared ancestry.

The i.i.d. assumption doesn't have to seem awkward, however, as long as you remember that probability is inside the golem, not outside in the world. The i.i.d. assumption is about how the golem represents its uncertainty. It is an *epistemological* assumption. It is not a physical assumption about the world, an *ontological* one, unless you insist that it is. E. T. Jaynes (1922–1998) called this the *mind projection fallacy*, the mistake of confusing epistemological claims with ontological claims.⁶⁵

The point isn't to say epistemology trumps reality, but rather that in ignorance of such correlations the most conservative distribution to use is i.i.d.⁶⁶ This issue will return in Chapter 9. Furthermore, there is a mathematical result known as *de Finetti's theorem* that tells us that values which are **EXCHANGEABLE** can be approximated by mixtures of i.i.d. distributions. Colloquially, exchangeable values can be reordered. The practical impact of this is that "i.i.d." as an assumption cannot be read too literally, as different process models again correspond to the same statistical model (as argued in

Chapter 1). Even furthermore, there are many types of correlation that do little or nothing to the overall shape of a distribution, but only affect the precise sequence in which values appear. For example, pairs of sisters have highly correlated heights. But the overall distribution of female height remains almost perfectly normal. In such cases, i.i.d. remains perfectly useful, despite ignoring the correlations. Consider for example that Markov chain Monte Carlo (Chapter 8) can use highly correlated sequential samples to estimate most any iid distribution we like.

To complete the model, we're going to need some priors. The parameters to be estimated are both μ and σ , so we need a prior $\Pr(\mu, \sigma)$, the joint prior probability for all parameters. In most cases, priors are specified independently for each parameter, which amounts to assuming $\Pr(\mu, \sigma) = \Pr(\mu) \Pr(\sigma)$. Then we can write:

$$\begin{array}{ll} h_i \sim \text{Normal}(\mu, \sigma) & [\text{likelihood}] \\ \mu \sim \text{Normal}(178, 20) & [\mu \text{ prior}] \\ \sigma \sim \text{Uniform}(0, 50) & [\sigma \text{ prior}] \end{array}$$

The labels on the right are not part of the model, but instead just notes to help you keep track of the purpose of each line. The prior for μ is a broad Gaussian prior, centered on 178cm, with 95% of probability between 178 ± 40 .

Why 178 cm? Your author is 178 cm tall. And the range from 138 cm to 218 cm encompasses a huge range of plausible mean heights for human populations. So domain-specific information has gone into this prior. Everyone knows something about human height and can set a reasonable and vague prior of this kind. But in many regression problems, as you'll see later, using prior information is more subtle, because parameters don't always have such clear physical meaning.

Whatever the prior, it's a very good idea to plot your priors, so you have a sense of the assumption they build into the model. In this case:

```
R code
4.11 curve( dnorm( x , 178 , 20 ) , from=100 , to=250 )
```

Execute that code yourself, to see that the golem is assuming that the average height (not each individual height) is almost certainly between 140 cm and 220 cm. So this prior carries a little information, but not a lot. The σ prior is a truly flat prior, a uniform one, that functions just to constrain σ to have positive probability between zero and 50cm. View it with:

```
R code
4.12 curve( dunif( x , 0 , 50 ) , from=-10 , to=60 )
```

A standard deviation like σ must be positive, so bounding it at zero makes sense. How should we pick the upper bound? In this case, a standard deviation of 50cm would imply that 95% of individual heights lie within 100cm of the average height. That's a very large range.

All this talk is nice, but it'll help to really see what these priors imply about the distribution of individual heights. You didn't specify a prior probability distribution of heights directly, but once you've chosen priors for μ and σ , these imply a prior distribution of individual heights. You can quickly simulate heights by sampling from the prior, like you sampled from the posterior back in Chapter 3. Remember, every posterior is also potentially a prior for a subsequent analysis, so you can process priors just like posteriors.

```
sample_mu <- rnorm( 1e4 , 178 , 20 )
sample_sigma <- runif( 1e4 , 0 , 50 )
prior_h <- rnorm( 1e4 , sample_mu , sample_sigma )
dens( prior_h )
```

R code
4.13

The density plot you get shows a vaguely bell-shaped density with thick tails. It is the expected distribution of heights, averaged over the prior. Notice that the prior probability distribution of height is not itself Gaussian. This is okay. The distribution you see is not an empirical expectation, but rather the distribution of relative plausibilities of different heights, before seeing the data.

Play around with the numbers in the priors above, to explore their effects on the prior probability density of heights.

Rethinking: A farewell to epsilon. Some readers will have already met an alternative notation for a Gaussian linear model:

$$h_i = \mu + \epsilon_i$$

$$\epsilon_i \sim \text{Normal}(0, \sigma)$$

This is equivalent to the $h_i \sim \text{Normal}(\mu, \sigma)$ form, with the ϵ standing in for the Gaussian density. But this ϵ form is poor form. The reason is that it does not usually generalize to other types of models. This means it won't be possible to express non-Gaussian models using tricks like ϵ . Better to learn one system that does generalize.

Overthinking: Model definition to Bayes' theorem again. It can help to see how the model definition on the previous page allows us to build up the posterior distribution. The height model, with its priors for μ and σ , defines this posterior distribution:

$$\Pr(\mu, \sigma | h) = \frac{\prod_i \text{Normal}(h_i | \mu, \sigma) \text{Normal}(\mu | 178, 20) \text{Uniform}(\sigma | 0, 50)}{\int \int \prod_i \text{Normal}(h_i | \mu, \sigma) \text{Normal}(\mu | 178, 20) \text{Uniform}(\sigma | 0, 50) d\mu d\sigma}$$

This looks monstrous, but it's the same creature as before. There are two new things that make it seem complicated. The first is that there is more than one observation in h , so to get the joint likelihood across all the data, we have to compute the probability for each h_i and then multiply all these likelihoods together. The product on the right-hand side takes care of that. The second complication is the two priors, one for μ and one for σ . But these just stack up. In the grid approximation code in the section to follow, you'll see the implications of this definition in the R code. Everything will be calculated on the log scale, so multiplication will become addition. But otherwise it's just a matter of executing Bayes' theorem.

4.3.3. Grid approximation of the posterior distribution. Since this is the first Gaussian model in the book, and indeed the first model with more than one parameter, it's worth quickly mapping out the posterior distribution through brute force calculations. This isn't the approach I encourage in any other place, because it is laborious and computationally expensive. Indeed, it is usually so impractical as to be essentially impossible. But as always, it is worth knowing what the target actually looks like, before you start accepting approximations of it. A little later in this chapter, you'll use quadratic approximation to estimate the posterior distribution, and that's the approach you'll use for several chapters more. Once you

have the samples you'll produce in this subsection, you can compare them to the quadratic approximation in the next.

Unfortunately, doing the calculations here requires some technical tricks that add little, if any, conceptual insight. So I'm going to present the code here without explanation. You can execute it and keep going for now, but later return and follow the endnote for an explanation of the algorithm.⁶⁷ For now, here are the guts of the golem:

```
R code
4.14 mu.list <- seq( from=140, to=160 , length.out=200 )
      sigma.list <- seq( from=4 , to=9 , length.out=200 )
      post <- expand.grid( mu=mu.list , sigma=sigma.list )
      post$LL <- sapply( 1:nrow(post) , function(i) sum( dnorm(
                                d2$height ,
                                mean=post$mu[i] ,
                                sd=post$sigma[i] ,
                                log=TRUE ) ) )
      post$prod <- post$LL + dnorm( post$mu , 178 , 20 , TRUE ) +
                    dunif( post$sigma , 0 , 50 , TRUE )
      post$prob <- exp( post$prod - max(post$prod) )
```

You can inspect this posterior distribution, now residing in `post$prob`, using a variety of plotting commands. You can get a simple contour plot with:

```
R code
4.15 contour_xyz( post$mu , post$sigma , post$prob )
```

Or you can plot a simple heat map with:

```
R code
4.16 image_xyz( post$mu , post$sigma , post$prob )
```

The functions `contour_xyz` and `image_xyz` are both in the `rethinking` package.

4.3.4. Sampling from the posterior. To study this posterior distribution in more detail, again I'll push the flexible approach of sampling parameter values from it. This works just like it did in Chapter 3, when you sampled values of p from the posterior distribution for the globe tossing example. The only new trick is that since there are two parameters, and we want to sample combinations of them, we first randomly sample row numbers in `post` in proportion to the values in `post$prob`. Then we pull out the parameter values on those randomly sampled rows. This code will do it:

```
R code
4.17 sample.rows <- sample( 1:nrow(post) , size=1e4 , replace=TRUE ,
                          prob=post$prob )
      sample.mu <- post$mu[ sample.rows ]
      sample.sigma <- post$sigma[ sample.rows ]
```

You end up with 10,000 samples, with replacement, from the posterior for the height data. Take a look at these samples:

```
R code
4.18 plot( sample.mu , sample.sigma , cex=0.5 , pch=16 , col=col.alpha(rangi2,0.1) )
```

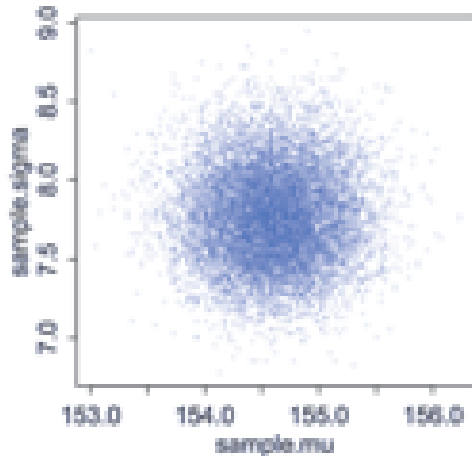


FIGURE 4.3. Samples from the posterior distribution for the heights data. The density of points is highest in the center, reflecting the most plausible combinations of μ and σ . There are many more ways for these parameter values to produce the data, conditional on the model.

I reproduce this plot in [FIGURE 4.3](#). Note that the function `col.alpha` is part of the `rethinking` R package. All it does is make colors transparent, which helps the plot in [FIGURE 4.3](#) more easily show density, where samples overlap. Adjust the plot to your tastes by playing around with `cex` (character expansion, the size of the points), `pch` (plot character), and the 0.1 transparency value.

Now that you have these samples, you can describe the distribution of confidence in each combination of μ and σ by summarizing the samples. Think of them like data and describe them, just like in Chapter 3. For example, to characterize the shapes of the *marginal* posterior densities of μ and σ , all we need to do is:

```
dens( sample.mu )
dens( sample.sigma )
```

R code
4.19

The jargon “marginal” here means “averaging over the other parameters.” Execute the above code and inspect the plots. These densities are very close to being normal distributions. And this is quite typical. As sample size increases, posterior densities approach the normal distribution. If you look closely, though, you’ll notice that the density for σ has a longer right-hand tail. I’ll exaggerate this tendency a bit later, to show you that this condition is very common for standard deviation parameters.

To summarize the widths of these densities with highest posterior density intervals, just like in Chapter 3:

```
HPDI( sample.mu )
HPDI( sample.sigma )
```

R code
4.20

Since these samples are just vectors of numbers, you can compute any statistic from them that you could from ordinary data. If you want the mean or median, just use the corresponding R functions.

Overthinking: Sample size and the normality of σ ’s posterior. Before moving on to using quadratic approximation (`map`) as shortcut to all of this inference, it is worth repeating the analysis of the height data above, but now with only a fraction of the original data. The reason to do this is to demonstrate

that, in principle, the posterior is not always so Gaussian in shape. There's no trouble with the mean, μ . For a Gaussian likelihood and a Gaussian prior on μ , the posterior distribution is always Gaussian as well, regardless of sample size. It is the standard deviation σ that causes problems. So if you care about σ —often people do not—you do need to be careful of abusing the quadratic approximation.

The deep reasons for the posterior of σ tending to have a long right-hand tail are complex. But a useful way to conceive of the problem is that variances must be positive. As a result, there must be more uncertainty about how big the variance (or standard deviation) is than about how small it is. For example, if the variance is estimated to be near zero, then you know for sure that it can't be much smaller. But it could be a lot bigger.

Let's quickly analyze only 20 of the heights from the height data to reveal this issue. To sample 20 random heights from the original list:

```
R code 4.21 d3 <- sample( d2$height , size=20 )
```

Now I'll repeat all the code from the previous subsection, modified to focus on the 20 heights in `d3` rather than the original data. I'll compress all of the code together, but it's just what you've already seen above.

```
R code 4.22 mu.list <- seq( from=150, to=170 , length.out=200 )
sigma.list <- seq( from=4 , to=20 , length.out=200 )
post2 <- expand.grid( mu=mu.list , sigma=sigma.list )
post2$LL <- sapply( 1:nrow(post2) , function(i)
  sum( dnorm( d3 , mean=post2$mu[i] , sd=post2$sigma[i] ,
    log=TRUE ) ) )
post2$prod <- post2$LL + dnorm( post2$mu , 178 , 20 , TRUE ) +
  dunif( post2$sigma , 0 , 50 , TRUE )
post2$prob <- exp( post2$prod - max(post2$prod) )
sample2.rows <- sample( 1:nrow(post2) , size=1e4 , replace=TRUE ,
  prob=post2$prob )
sample2.mu <- post2$mu[ sample2.rows ]
sample2.sigma <- post2$sigma[ sample2.rows ]
plot( sample2.mu , sample2.sigma , cex=0.5 ,
  col=col.alpha(rangi2,0.1) ,
  xlab="mu" , ylab="sigma" , pch=16 )
```

After executing the code above, you'll see another scatter plot of the samples from the posterior density, but this time you'll notice a distinctly longer tail at the top of the cloud of points. You should also inspect the marginal posterior density for σ , averaging over μ , produced with:

```
R code 4.23 dens( sample2.sigma , norm.comp=TRUE )
```

This code will also show a normal approximation with the same mean and variance. Now you can see that the posterior for σ is not Gaussian, but rather has a long tail of uncertainty towards higher values.

4.3.5. Fitting the model with `map`. Now we leave grid approximation behind and move on to one of the great engines of applied statistics, the quadratic approximation. Our interest in quadratic approximation, recall, is as a handy way to quickly make inferences about the shape of the posterior. The posterior's peak will lie at the *maximum a posteriori* estimate (MAP), and we can get a useful image of the posterior's shape by using the quadratic approximation of the posterior distribution at this peak.

To find the values of μ and σ that maximize the posterior probability, we'll use `map`, a command in the `rethinking` package. The way that `map` works is by using the model definition you were introduced to earlier in this chapter. Each line in the definition has a corresponding definition in the form of R code. The engine inside `map` then uses these definitions to define the posterior probability at each combination of parameter values. Then it can climb the posterior distribution and find the peak, its MAP.

Let's begin by repeating the code to load the data and select out the adults:

```
library(rethinking)
data(Howell1)
d <- Howell1
d2 <- d[ d$age >= 18 , ]
```

R code
4.24

Now we're ready to define the model, using R's formula syntax. The model definition in this case is just as before, but now we'll repeat it with each corresponding line of R code shown on the right-hand margin:

$h_i \sim \text{Normal}(\mu, \sigma)$	<code>height ~ dnorm(mu, sigma)</code>
$\mu \sim \text{Normal}(178, 20)$	<code>mu ~ dnorm(156, 10)</code>
$\sigma \sim \text{Uniform}(0, 50)$	<code>sigma ~ dunif(0, 50)</code>

Now place the R code equivalents into an `alist`. Here's an `alist` of the formulas above:

```
flist <- alist(
  height ~ dnorm( mu , sigma ) ,
  mu ~ dnorm( 178 , 20 ) ,
  sigma ~ dunif( 0 , 50 )
)
```

R code
4.25

Note the commas at the end of each line, except the last. These commas separate each line of the model definition.

Fit the model to the data in the data frame `d2` with:

```
m4.1 <- map( flist , data=d2 )
```

R code
4.26

After executing this code, you'll have a fit model stored in the symbol `m4.1`. Now take a look at the fit *maximum a posteriori* model:

```
precis( m4.1 )
```

R code
4.27

	Mean	StdDev	5.5%	94.5%
mu	154.61	0.41	153.95	155.27
sigma	7.73	0.29	7.27	8.20

These numbers provide Gaussian approximations for each parameter's *marginal* distribution. This means the plausibility of each value of μ , after averaging over the plausibilities of each value of σ , is given by a Gaussian distribution with mean 154.6 and standard deviation 0.4.

The 5.5% and 94.5% quantiles are percentile interval boundaries, corresponding to an 89% interval. Why 89%? It's just the default. It displays a quite wide interval, so it shows a

high-probability range of parameter values. If you want another interval, such as the conventional and mindless 95%, you can use `precis(m4.1, prob=0.95)`. But I don't recommend 95% intervals, because readers will have a hard time not viewing them as significance tests. 89 is also a prime number, so if someone asks you to justify it, you can stare at them meaningfully and incant, "Because it is prime." That's no worse justification than the conventional justification for 95%.

I encourage you to compare these 89% boundaries to the HPDIs from the grid approximation earlier. You'll find that they are almost identical. When the posterior is approximately Gaussian, then this is what you should expect.

Overthinking: Start values for `map`. `map` estimates the posterior by climbing it like a hill. To do this, it has to start climbing someplace, at some combination of parameter values. Unless you tell it otherwise, `map` starts at random values sampled from the prior. But it's also possible to specify a starting value for any parameter in the model. In the example in the previous section, that means the parameters μ and σ . Here's a good list of starting values in this case:

R code
4.28

```
start <- list(
  mu=mean(d2$height),
  sigma=sd(d2$height)
)
```

These start values are good guesses of the rough location of the MAP values.

Note that the list of start values is a regular `list`, not an `alist` like the formula list is. The two functions `alist` and `list` do the same basic thing: allow you to make a collection of arbitrary R objects. They differ in one important respect: `list` evaluates the code you embed inside it, while `alist` does not. So when you define a list of formulas, you should use `alist`, so the code isn't executed. But when you define a list of start values for parameters, you should use `list`, so that code like `mean(d2$height)` will be evaluated to a numeric value.

The priors we used before are very weak, both because they are nearly flat and because there is so much data. So I'll splice in a more informative prior for μ , so you can see the effect. All I'm going to do is change the standard deviation of the prior to 0.1, so it's a very narrow prior. I'll also build the formula right into the call to `map`, so you can see how to build it all at once.

R code
4.29

```
m4.2 <- map(
  alist(
    height ~ dnorm( mu , sigma ) ,
    mu ~ dnorm( 178 , 0.1 ) ,
    sigma ~ dunif( 0 , 50 )
  ) ,
  data=d2 )
precis( m4.2 )
```

	Mean	StdDev	5.5%	94.5%
mu	177.86	0.10	177.70	178.02
sigma	24.52	0.93	23.03	26.00

Notice that the estimate for μ has hardly moved off the prior. The prior was very concentrated around 178. So this is not surprising. But also notice that the estimate for σ has changed quite

a lot, even though we didn't change its prior at all. Once the golem is certain that the mean is near 178—as the prior insists—then the golem has to estimate σ conditional on that fact. This results in a different posterior for σ , even though all we changed is prior information about the other parameter.

At this point, it pays to play around with different priors, to get a sense of their effect on inference. There's so much data here that you'll have to use pretty extreme priors to have any effect on inference. Also keep in mind that most non-Bayesian estimates implicitly use flat priors. Before long, you're going to see that perfectly flat priors are hardly ever the best priors. But with as much data as we have in this example, and with such a simple model, it hardly matters which priors you use.

Overthinking: How strong is a prior? A prior can usually be interpreted as a former posterior inference, as previous data. So it's sometimes useful to talk about the strength of a prior in terms of which data would lead to the same posterior distribution, beginning with a flat prior. In the very narrow prior used just above, the $\mu \sim \text{Normal}(178, 0.1)$ prior, we can compute the implied amount of data easily, because there is a simple formula for the standard deviation of a Gaussian posterior for μ :

$$\sigma_{\text{post}} = 1/\sqrt{n}$$

(This is by no coincidence the same as the formula for the standard error of the sampling distribution of the mean, in non-Bayesian inference.) This formula implies that the implied amount of data (with mean 178) is $n = 1/\sigma_{\text{post}}^2$. So in this case the implied amount of data corresponding to the prior with standard deviation 0.1 is $n = 1/0.01 = 100$. So the $\mu \sim \text{Normal}(178, 0.1)$ is equivalent to having previously observed 100 heights with mean value 178. That's a pretty strong prior. In contrast, the former $\text{Normal}(178, 20)$ prior implies $n = 1/20^2 = 0.0025$ of an observation. This is an extremely weak prior. But of course exactly how strong or weak either prior is will depend upon how much data is used to update it.

4.3.6. Sampling from a map fit. The above explains how to get a MAP quadratic approximation of the posterior, using `map`. But how do you then get samples from the quadratic approximate posterior distribution? The answer is rather simple, but non-obvious, and it requires recognizing that a quadratic approximation to a posterior distribution with more than one parameter dimension— μ and σ each contribute one dimension—is just a multi-dimensional Gaussian distribution.

As a consequence, when R constructs a quadratic approximation, it calculates not only standard deviations for all parameters, but also the covariances among all pairs of parameters. Just like a mean and standard deviation (or its square, a variance) are sufficient to describe a one-dimensional Gaussian distribution, a list of means and a matrix of variances and covariances are sufficient to describe a multi-dimensional Gaussian distribution. To see this matrix of variances and covariances, for model `m4.1`, use:

```
vcov( m4.1 )
```

R code
4.30

```

              mu      sigma
mu    0.1697395865 0.0002180593
sigma 0.0002180593 0.0849057933
```

The above is a **VARIANCE-COVARIANCE** matrix. It is the multi-dimensional glue of a quadratic approximation, because it tells us how each parameter relates to every other parameter in the posterior distribution. A variance-covariance matrix can be factored into two

elements: (1) a vector of variances for the parameters and (2) a correlation matrix that tells us how changes in any parameter lead to correlated changes in the others. This decomposition is usually easier to understand. So let's do that now:

R code
4.31

```
diag( vcov( m4.1 ) )
cov2cor( vcov( m4.1 ) )
```

```
      mu      sigma
0.16973959 0.08490579

      mu      sigma
mu      1.000000000 0.001816412
sigma 0.001816412 1.000000000
```

The two-element vector in the output is the list of variances. If you take the square root of this vector, you get the standard deviations that are shown in `precis` output. The two-by-two matrix in the output is the correlation matrix. Each entry shows the correlation, bounded between -1 and $+1$, for each pair of parameters. The 1's indicate a parameter's correlation with itself. If these values were anything except 1, we would be worried. The other entries are typically closer to zero, and they are very close to zero in this example. This indicates that learning μ tells us nothing about σ and likewise that learning σ tells us nothing about μ . This is typical of simple Gaussian models of this kind. But it is quite rare more generally, as you'll see in later chapters.

Okay, so how do we get samples from this multi-dimensional posterior? Now instead of sampling single values from a simple Gaussian distribution, we sample vectors of values from a multi-dimensional Gaussian distribution. The `rethinking` package provides a convenience function to do exactly that:

R code
4.32

```
library(rethinking)
post <- extract.samples( m4.1 , n=1e4 )
head(post)
```

```
      mu      sigma
1 155.0031 7.443893
2 154.0347 7.771255
3 154.9157 7.822178
4 154.4252 7.530331
5 154.5307 7.655490
6 155.1772 7.974603
```

You end up with a data frame, `post`, with 10,000 ($1e4$) rows and two columns, one column for μ and one for σ . Each value is a sample from the posterior, so the mean and standard deviation of each column will be very close to the MAP values from before. You can confirm this by summarizing the samples:

R code
4.33

```
precis(post)
```

```
      Mean StdDev |0.89 0.89|
mu      154.61   0.42 153.95 155.27
sigma    7.73   0.29  7.26  8.19
```

The 89% HPDI is displayed now, because we used samples. $|0.89$ means the lower boundary, and $0.89|$ means the upper boundary. Compare these values to the output from `precis(m4.1)`. And you can use `plot(post)` to see how much they resemble the samples from the grid approximation in [FIGURE 4.3](#) (page 85).

These samples also preserve the covariance between μ and σ . This hardly matters right now, because μ and σ don't covary at all in this model. But once you add a predictor variable to your model, covariance will matter a lot.

Overthinking: Under the hood with multivariate sampling. The function `extract.samples` is for convenience. It is just running a simple simulation of the sort you conducted near the end of Chapter 3. Here's a peek at the motor. The work is done by a multi-dimensional version of `rnorm`, `mvnrm`. The function `rnorm` simulates random Gaussian values, while `mvnrm` simulates random vectors of multivariate Gaussian values. Here's how to use it directly to do what `extract.samples` does:

```
library(MASS)
post <- mvnrm( n=1e4 , mu=coef(m4.1) , Sigma=vcov(m4.1) )
```

R code
4.34

You don't usually need to use `mvnrm` directly like this, but sometimes you want to simulate multivariate Gaussian outcomes. In that case, you'll need to access `mvnrm` directly. And of course it's always good to know a little about how the machine operates.

Overthinking: Getting σ right. The quadratic assumption for σ can be problematic, as seen on page 85. A conventional way to improve the situation is the estimate $\log(\sigma)$ instead. Why does this help? While the posterior distribution of σ will often not be Gaussian, the distribution of its logarithm can be much closer to Gaussian. So if we impose the quadratic approximation on the logarithm, rather than the standard deviation itself, we can often get a better approximation of the uncertainty. Here's how you can do this, using `map`.

```
m4.1_logsigma <- map(
  alist(
    height ~ dnorm( mu , exp(log_sigma) ) ,
    mu ~ dnorm( 178 , 20 ) ,
    log_sigma ~ dnorm( 2 , 10 )
  ) , data=d2 )
```

R code
4.35

Notice the `exp` inside the likelihood. That converts a continuous parameter, `log_sigma`, to be strictly positive, because $\exp(x) > 0$ for any real value x . Notice also the prior for `log_sigma`. Since `log_sigma` is continuous now, it can have a Gaussian prior.

When you extract samples, it is `log_sigma` that has a Gaussian distribution. To get the distribution of `sigma`, you just need to use the same `exp` as in the model definition, to get back on the natural scale:

```
post <- extract.samples( m4.1_logsigma )
sigma <- exp( post$log_sigma )
```

R code
4.36

When you have a lot of data, this won't make any noticeable difference. But the use of `exp` to effectively constrain a parameter to be positive is a robust and useful one. And it relates to *link functions*, which will be very important when we arrive at generalized linear models in Chapter 9.

4.4. Adding a predictor

What we've done above is a Gaussian model of height in a population of adults. But it doesn't really have the usual feel of "regression" to it. Typically, we are interested in modeling how an outcome is related to some predictor variable. And by including a predictor variable in a particular way, we'll have linear regression.

So now let's look at how height in these Kalahari foragers covaries with weight. This isn't the most thrilling scientific question, I know. But it is an easy relationship to start with, and it only seems dull because you don't have a theory about growth and life history in mind. If you did, it would be thrilling. Go ahead and plot height and weight against one another to get an idea of how strongly they covary:

R code
4.37

```
plot( d2$height ~ d2$weight )
```

The resulting plot is not shown here. You really should do it yourself. Once you can see the plot, you'll see that there's obviously a relationship: Knowing a person's weight helps you predict height.

To make this vague observation into a more precise quantitative model that relates values of weight to plausible values of height, we need some more technology. How do we take our Gaussian model from the previous section and incorporate predictor variables?

Rethinking: What is "regression"? Many diverse types of models are called "regression." The term has come to mean using one or more predictor variables to model the distribution of one or more outcome variables. The original use of term, however, arose from anthropologist Francis Galton's (1822–1911) observation that the sons of tall and short men tended to be more similar to the population mean, hence *regression to the mean*.⁶⁸

This phenomenon arises statistically whenever individual measurements are assigned a common distribution, leading to *shrinkage* as each measurement informs the others. In the context of Galton's height data, attempting to predict each son's height on the basis of only his father's height is folly. Better to use the population of fathers. This leads to a prediction for each son which is similar to each father but "shrunk" towards the overall mean. Such predictions are routinely better. This same regression/shrinkage phenomenon applies at higher levels of abstraction and forms one basis of multilevel modeling (Chapter 12).

4.4.1. The linear model strategy. The strategy is to make the parameter for the mean of a Gaussian distribution, μ , into a linear function of the predictor variable and other, new parameters that we invent. This strategy is often simply called the **LINEAR MODEL**. The linear model strategy instructs the golem to assume that the predictor variable has a perfectly constant and additive relationship to the mean of the outcome. The golem then computes the posterior distribution of this constant relationship.

What this means, recall, is that the machine considers every possible combination of the parameter values. With a linear model, some of the parameters now stand for the strength of association between the mean of the outcome and the value of the predictor. For each combination of values, the machine computes the posterior probability, which is a measure of relative plausibility, given the model and data. So the posterior distribution ranks the infinite possible combinations of parameter values by their logical plausibility. As a result, the posterior distribution provides relative plausibilities of the different possible strengths of association, given the assumptions you programmed into the model.

Here's how it works, in the simplest case of only one predictor variable. We'll wait until the next chapter to confront more than one predictor. Recall the basic Gaussian model:

$$\begin{aligned} h_i &\sim \text{Normal}(\mu, \sigma) && [\text{likelihood}] \\ \mu &\sim \text{Normal}(178, 20) && [\mu \text{ prior}] \\ \sigma &\sim \text{Uniform}(0, 50) && [\sigma \text{ prior}] \end{aligned}$$

Now how do we get weight into a Gaussian model of height? Let x be the mathematical name for the column of weight measurements, `d2$weight`. Now we have a predictor variable x , which is a list of measures of the same length as h . We'd like to say how knowing the values in x can help us describe or predict the values in h . To get `weight` into the model in this way, we define the mean μ as a function of the values in x . This is what it looks like, with explanation to follow:

$$\begin{aligned} h_i &\sim \text{Normal}(\mu_i, \sigma) && [\text{likelihood}] \\ \mu_i &= \alpha + \beta x_i && [\text{linear model}] \\ \alpha &\sim \text{Normal}(178, 100) && [\alpha \text{ prior}] \\ \beta &\sim \text{Normal}(0, 10) && [\beta \text{ prior}] \\ \sigma &\sim \text{Uniform}(0, 50) && [\sigma \text{ prior}] \end{aligned}$$

Again, I've labeled each line on the right-hand side, by the type of definition it encodes. We'll discuss each in turn.

4.4.1.1. Likelihood. To decode all of this, let's begin with just the likelihood, the first line of the model. This is nearly identical to before, except now there is a little index i on the μ , as well as the h . This is necessary now, because the mean μ now depends upon unique predictor values on each row i . So the little i on μ_i indicates that *the mean depends upon the row*.

4.4.1.2. Linear model. The mean μ is no longer a parameter to be estimated. Rather, as seen in the second line of the model, μ_i is constructed from other parameters, α and β , and the predictor variable x . This line is not a stochastic relationship—there is no $\sim$ in it, but rather an $=$ in it—because the definition of μ_i is deterministic, not probabilistic. That is to say that, once we know α and β and x_i , we also know μ_i .

The value x_i is just the weight value on row i . It refers to the same individual as the height value, h_i , on the same row. The parameters α and β are more mysterious. Where did they come from? We made them up. The parameters μ and σ are necessary and sufficient to describe a Gaussian distribution. But α and β are instead devices we invent for manipulating μ , allowing it to vary systematically across cases in the data.

You'll be making up all manner of parameters as your skills improve. One way to understand these made-up parameters is to think of them as targets of learning. Each parameter is something that must be described in the posterior density. So when you want to know something about the data, you ask your golem by inventing a parameter for it. This will make more and more sense as you progress. Here's how it works in this context. The second line of the model definition is just:

$$\mu_i = \alpha + \beta x_i$$

What this tells the regression golem is that you are asking two questions about the mean of the outcome.

- (1) What is the expected height, when $x_i = 0$? The parameter α answers this question. For this reason, α is often called the *intercept*.
- (2) What is the change in expected height, when x_i changes by 1 unit? The parameter β answers this question.

Jointly these two parameters, together with x , ask the golem to find a line that relates x to h , a line that passes through α when $x_i = 0$ and has slope β . That is a task that golems are very good at. It's up to you, though, to be sure it's a good question.

Rethinking: Nothing special or natural about linear models. Note that there's nothing special about the linear model, really. You can choose a different relationship between α and β and μ . For example, the following is a perfectly legitimate definition for μ_i :

$$\mu_i = \alpha \exp(-\beta x_i)$$

This does not define a linear regression, but it does define a regression model. The linear relationship we are using instead is conventional, but nothing requires that you use it. It is very common in some fields, like ecology and demography, to use functional forms for μ that come from theory, rather than the geocentrism of linear models. Models built out of substantive theory can dramatically outperform linear models of the same phenomena.⁶⁹

Overthinking: Units and regression models. Readers who had a traditional training in physical sciences will know how to carry units through equations of this kind. For their benefit, here's the model again (omitting priors for brevity), now with units of each symbol added.

$$h_i \text{cm} \sim \text{Normal}(\mu_i \text{cm}, \sigma \text{cm})$$

$$\mu_i \text{cm} = \alpha \text{cm} + \beta \frac{\text{cm}}{\text{kg}} x_i \text{kg}$$

So you can see that β must have units of cm/kg in order for the mean μ_i to have units of cm. One of the facts that labeling with units clears up is that a parameter like β is a kind of rate. There's also a tradition called *dimensional analysis* that advocates constructing variables so that they are unit-less ratios. In this context, for example, we might divide height by a reference height, removing its units. Measurement scales are arbitrary human constructions, and sometimes the unit-less analysis is more natural.

4.4.1.3. Priors. The remaining lines in the model define priors for the parameters to be estimated: α , β , and σ . All of these are weak priors, leading to inferences that will echo non-Bayesian methods of model fitting, such as maximum likelihood. But as always, you should play around with the priors—plotting them, changing them, and refitting the model—to get a sense of their influence.

You've seen priors for α and σ before, although α was called μ back then. I've widened the prior for α , since as you'll see it is common for the intercept in a linear model to swing a long way from the mean of the outcome variable. The flat prior here with a huge standard deviation will allow it to move wherever it needs to. This won't make sense right now, but once you see the posterior distribution, it will.

The prior for β deserves explanation. Why have a Gaussian prior with mean zero? This prior places just as much probability below zero as it does above zero, and when $\beta = 0$, weight has no relationship to height. So many people see it as conservative assumption. And such a prior will pull probability mass towards zero, leading to more conservative estimates

than a perfectly flat prior will. But note that a Gaussian prior with standard deviation of 10 is still very weak, so the amount of conservatism it induces will be very small. As you make the standard deviation in this prior smaller, the amount of shrinkage towards zero increases and your model produces more and more conservative estimates about the relationship between height and weight. In Chapter 6, you'll see why such conservative priors are useful for improving inference.

Before fitting this model, though, consider whether this zero-centered prior really makes sense. Do you think there's just as much chance that the relationship between height and weight is negative as that it is positive? Of course you don't. In this context, such a silly prior is harmless, because there is a lot of data. But in other contexts, your golem may need a little nudge in the right direction.

Rethinking: What's the correct prior? People commonly ask what the correct prior is for a given analysis. The question sometimes implies that for any given set of data, there is a uniquely correct prior that must be used, or else the analysis will be invalid. This is a mistake. There is no more a uniquely correct prior than there is a uniquely correct likelihood. Statistical models are machines for inference. Many machines will work, but some work better than others. Priors can be wrong, but only in the same sense that a kind of hammer can be wrong for building a table.

In choosing priors, there are simple guidelines to get you started. Priors encode states of information before seeing data. So priors allow us to explore the consequences of beginning with different information. In cases in which we have good prior information that discounts the plausibility of some parameter values, like negative associations between height and weight, we can encode that information directly into priors. When we don't have such information, we still usually know enough about the plausible range of values. And you can vary the priors and repeat the analysis in order to study how different states of initial information influence inference. Frequently, there are many reasonable choices for a prior, and all of them produce the same inference. And conventional Bayesian priors are *conservative*, relative to conventional non-Bayesian approaches.

Making choices tends to make novices nervous. There's an illusion sometimes that default procedures are more objective than procedures that require user choice, such as choosing priors. If that's true, then all "objective" means is that everyone does the same thing. It carries no guarantees of realism or accuracy.

4.4.2. Fitting the model. The code needed to fit this model via quadratic approximation is a straightforward modification of the kind of code you've already seen. All we have to do is incorporate our new model for the mean into the model specification inside `map` and be sure to add our new parameters to the `start` list. Let's repeat the model definition, now with the corresponding R code on the right-hand side:

$h_i \sim \text{Normal}(\mu_i, \sigma)$	<code>height ~ dnorm(mu,sigma)</code>
$\mu_i = \alpha + \beta x_i$	<code>mu <- a + b*weight</code>
$\alpha \sim \text{Normal}(178, 100)$	<code>a ~ dnorm(156,100)</code>
$\beta \sim \text{Normal}(0, 10)$	<code>b ~ dnorm(0,10)</code>
$\sigma \sim \text{Uniform}(0, 50)$	<code>sigma ~ dunif(0,50)</code>

Notice that the linear model, in the R code on the right-hand side, uses the R assignment operator, `<-`, even though the mathematical definition uses the symbol `=`. This is a code convention shared by several Bayesian model fitting engines, so it's worth getting used to

the switch. You just have to remember to use `<-` instead of `=` when defining a linear model. That's it.

And the above allows us to build the MAP model fit:

```
R code
4.38 # load data again, since it's a long way back
      library(rethinking)
      data(Howell1)
      d <- Howell1
      d2 <- d[ d$age >= 18 , ]

      # fit model
      m4.3 <- map(
        alist(
          height ~ dnorm( mu , sigma ) ,
          mu <- a + b*weight ,
          a ~ dnorm( 156 , 100 ) ,
          b ~ dnorm( 0 , 10 ) ,
          sigma ~ dunif( 0 , 50 )
        ) ,
        data=d2 )
```

The parameter `mu` is no longer really a parameter here, because it has been replaced by the linear model, `a+b*weight`, where `a` is α and `b` is β and `weight` is of course our x in this instance. So there is a prior for the parameter `a` now, but not one for `mu`, since `mu` is defined by the linear model instead.

In the `start` list, `mu` is replaced by `a`, which starts at the overall mean, just like `mu` used to. We also have to add the new parameter `b` to this list, and as is usually a conservative first guess, we start the slope out at zero (0), which is equivalent to no relationship between the outcome and predictor.

Note that starting `b` at zero is not the same as having β 's prior with mean zero. The values in the `start` list don't alter the posterior probabilities, while priors definitely do. But you do have to think a little about your choice of `start` values, because if you start the MAP search far outside the high density region of the posterior, R may give up looking before it finds the MAP values.

Rethinking: Everything that depends upon parameters has a posterior distribution. In the model introduced above, the parameter μ is no longer a parameter, since it has become a function of the parameters α and β . But since the parameters α and β have a joint posterior, so too does μ . Later in the chapter, you'll work directly the posterior distribution of μ , even though it's not a parameter anymore. Since parameters are uncertain, everything that depends upon them is also uncertain. This includes statistics like μ , as well as model-based predictions, measures of fit, and everything else that uses parameters. By working with samples from the posterior, all you have to do to account for posterior uncertainty in any quantity is to compute that quantity for each sample from the posterior. The resulting quantities, one for each posterior sample, will approximate the quantity's posterior distribution.

Overthinking: Embedding linear models. It may help to understand what the linear model is doing, by seeing another way to fit the same model, but without a separate line for the linear model. You can just merge the linear model into the likelihood definition, like this:

```
m4.3 <- map(
  alist(
    height ~ dnorm( a + b*weight , sigma ) ,
    a ~ dnorm( 178 , 100 ) ,
    b ~ dnorm( 0 , 10 ) ,
    sigma ~ dunif( 0 , 50 )
  ) ,
  data=d2 )
```

R code
4.39

This is exactly the same model, but with the linear definition for μ embedded in the first line of code. The form with the explicit linear model, `mu <- a + b*weight`, is easier to read. But the form just above better reflects the actual computation. Some of the helper functions you'll use later depend upon the explicit linear model however, so I recommend using it. It'll make life a little easier.

4.4.3. Interpreting the model fit. One trouble with statistical models is that they are hard to understand. Once you've fit the model, it can only report posterior probabilities. These are the right answer to the question that is this combination of model and data. But it's your responsibility to process the answer and make sense of it.

There are two broad categories of processing: (1) reading tables and (2) plotting. For some simple questions, it's possible to learn a lot just from tables of MAP values, their standard deviations, and intervals. But most models are very hard to understand from tables of estimates alone. A major difficulty with tables alone is their apparent simplicity compared to the complexity of the model and data that generated them. Once you have more than a couple of parameters in a model, it is very hard to figure out from numbers alone how all of them act to influence prediction. Once you begin adding interaction terms (Chapter 7) or polynomials (later in this chapter), it may not even be possible to guess the direction of influence a predictor variable has on an outcome.

So throughout this book, I emphasize plotting posterior distributions and posterior predictions, instead of attempting to understand a table. Once you become experienced with a particular type of model and kind of data, you'll be able to confidently read tables of estimates. But that will be because you will have won valuable contextual knowledge about the problem at hand, knowledge that will transfer, but incompletely, to other problems.

To win such knowledge, most of us must do a lot of plotting. Plotting the implications of your estimates will allow you to inquire about several things that are sometimes hard to read from tables:

- (1) Whether or not the model fitting procedure worked correctly
- (2) The *absolute* magnitude, rather than merely *relative* magnitude, of a relationship between outcome and predictor
- (3) The uncertainty surrounding an average relationship
- (4) The uncertainty surrounding the implied predictions of the model, as these are distinct from mere parameter uncertainty

In addition, once you get the hang of processing estimates into plots, you can ask any question you can think of, for any model type. And readers of your results will appreciate a figure much more than they will a table of estimates.

So in the remainder of this section, I first spend a little time talking about tables of estimates. I use this opportunity to briefly introduce a data transformation called **CENTERING** that aids in interpreting estimates. Then I move on to show how to plot estimates that always incorporate information from the full posterior distribution, including correlations among parameters.

Rethinking: What do parameters mean? A basic issue with interpreting model-based estimates is in knowing the meaning of parameters. There is no consensus about what a parameter means, however, because different people take different philosophical stances towards models, probability, and prediction. The perspective in this book is a common Bayesian perspective: *Posterior probabilities of parameter values describe the relative compatibility of different states of the world with the data, according to the model.* These are small world (Chapter 2) numbers. So reasonable people may disagree about the large world meaning, and the details of those disagreements depend strongly upon context. Such disagreements are productive, because they lead to model criticism and revision, something that golems cannot do for themselves.

4.4.3.1. *Tables of estimates.* Before looking closely at the new table of estimates, it's important to realize that models cannot in general be understood by tables of estimates. In this simple model, a lot can be learned from the summary output. But this is not a general property of models, Bayesian or not, because of the covariation among parameters.

With the new linear regression fit to the Kalahari data, we inspect the estimates:

R code
4.40

```
precis( m4.3 )
```

	Mean	StdDev	5.5%	94.5%
a	113.90	1.91	110.85	116.94
b	0.90	0.04	0.84	0.97
sigma	5.07	0.19	4.77	5.38

The first row gives the quadratic approximation for α , the second the approximation for β , and the third approximation for σ . Let's try to make some sense of them in this very simple model.

Best to begin with b (β), because it's the new parameter. Since β is a slope, the value 0.90 can be read as *a person 1 kg heavier is expected to be 0.90 cm taller*. 89% of the posterior probability lies between 0.84 and 0.97. That suggests that β values close to zero or greatly above one are highly incompatible with these data and this model. If you were thinking that perhaps there was no relationship at all between height and weight, then this estimate indicates strong evidence of a positive relationship instead. But maybe you just wanted as precise a measurement as possible of the relationship between height and weight. This estimate embodies that measurement, conditional on the model. For a different model, the measure of the relationship might be different.

The estimate of α , a in the `precis` table, indicates that a person of weight 0 should be 114cm tall. This is nonsense, since real people always have positive weight, yet it is also true. Parameters like α are “intercepts” that tell us the value of μ when all of the predictor variables have value zero. As a consequence, the value of the intercept is frequently uninterpretable

without also studying any β parameters. This is why we need very weak priors for intercepts, in many cases.

Finally, the estimate for σ , `sigma`, informs us of the width of the distribution of heights around the mean. A quick way to interpret it is to recall that about 95% of the probability in a Gaussian distribution lies between two standard deviations. So in this case, the estimate tells us that 95% of plausible heights lie within 10cm (2σ) of the mean height. But there is also uncertainty about this, as indicated by the 89% percentile interval.

As I mentioned at the start of this section, the numbers in the default `precis` output aren't sufficient to describe the quadratic posterior completely. For that, we also require the variance-covariance matrix. We're interested in correlations among parameters—we already have their variance in the table above—so let's go straight to the correlation matrix:

```
precis( m4.3 , corr=TRUE )
```

R code
4.41

	Mean	StdDev	5.5%	94.5%	a	b	sigma
a	113.90	1.91	110.85	116.94	1.00	-0.99	0
b	0.90	0.04	0.84	0.97	-0.99	1.00	0
sigma	5.07	0.19	4.77	5.38	0.00	0.00	1

The new columns on the far right show the correlations among the parameters. This is the same information you'd get by using `cov2cor(vcov(m4.3))`. Notice that α and β are almost perfectly negatively correlated. Right now, this is harmless. It just means that these two parameters carry the same information—as you change the slope of the line, the best intercept changes to match it. But in more complex models, strong correlations like this can make it difficult to fit the model to the data. So we'll want to use some golem engineering tricks to avoid it, when possible.

The first trick is [CENTERING](#). Centering is the procedure of subtracting the mean of a variable from each value. To create a centered version of the weight variable:

```
d2$weight.c <- d2$weight - mean(d2$weight)
```

R code
4.42

You can confirm that the average value of `weight.c` is zero: `mean(d2$weight.c)`. Now let's refit the model and see what this gains us:

```
m4.4 <- map(
  alist(
    height ~ dnorm( mu , sigma ) ,
    mu <- a + b*weight.c ,
    a ~ dnorm( 178 , 100 ) ,
    b ~ dnorm( 0 , 10 ) ,
    sigma ~ dunif( 0 , 50 )
  ) ,
  data=d2 )
```

R code
4.43

The above code just replaces `weight` with `weight.c`, the new variable. Now for the new estimates:

```
precis( m4.4 , corr=TRUE )
```

R code
4.44

	Mean	StdDev	5.5%	94.5%	a	b	sigma
a	154.60	0.27	154.17	155.03	1	0	0
b	0.91	0.04	0.84	0.97	0	1	0
sigma	5.07	0.19	4.77	5.38	0	0	1

The estimates for β and σ are unchanged (within rounding error), but the estimate for α (a) is now the same as the average height value in the raw data. Try it yourself: `mean(d2$height)`. And the correlations among parameters are now all zero. What has happened here?

The estimate for the intercept, α , still means the same thing it did before: the expected value of the outcome variable, when the predictor variable is equal to zero. But now the mean value of the predictor is also zero. So the intercept also means: the expected value of the outcome, when the predictor is at its average value. This makes interpreting the intercept a lot easier.

4.4.3.2. Plotting posterior inference against the data. In truth, tables of estimates are usually insufficient for understanding the information contained in the posterior distribution. It's almost always much more useful to plot the posterior inference against the data. Not only does plotting help in interpreting the posterior, but it also provides an informal check on model assumptions. When the model's predictions don't come close to key observations or patterns in the plotted data, then you might suspect the model either did not fit correctly or is rather badly specified.

But even if you only treat plots as a way to help in interpreting the posterior, they are invaluable. For simple models like this one, it is easy to just read the table of numbers and understand what the model says. But for even slightly more complex models, especially those that include interaction effects (Chapter 7), interpreting posterior distributions is hard. Combine with this the problem of incorporating the information in `vcov` into your interpretations, and the plots are irreplaceable.

We're going to start with a simple version of that task, superimposing just the MAP values over the height and weight data. Then we'll slowly add more and more information to the prediction plots, until we've used the entire posterior distribution.

To superimpose the MAP values for mean height over the actual data:

R code
4.45

```
plot( height ~ weight , data=d2 )
abline( a=coef(m4.3) ["a"] , b=coef(m4.3) ["b"] )
```

You can see the resulting plot in [FIGURE 4.4](#). Each point in this plot is a single individual. The black line is defined by the MAP slope β and MAP intercept α . Notice that in the code above, I pulled the numbers straight from the fit model. The function `coef` returns a vector of MAP values, and the names of the parameters extract the intercept and slope.

4.4.3.3. Adding uncertainty around the mean. The MAP line is just the posterior mean, the most plausible line in the infinite universe of lines the posterior distribution has considered. Plots of the MAP line, like [FIGURE 4.4](#), are useful for getting an impression of the magnitude of the estimated influence of a variable, like `weight`, on an outcome, like `height`.

But they do a poor job of communicating uncertainty. Remember, the posterior distribution considers every possible regression line connecting height to weight. It assigns a relative plausibility to each. This means that each combination of α and β has a posterior probability. It could be that there are many lines with nearly the same posterior probability

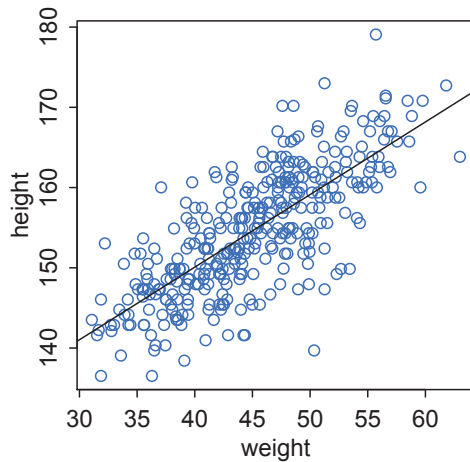


FIGURE 4.4. Height in centimeters (vertical) plotted against weight in kilograms (horizontal), with the *maximum a posteriori* line for the mean height at each weight plotted in black.

as the MAP line. Or it could be instead that the posterior distribution is rather narrow near the MAP line.

So how can we get that uncertainty onto the plot? Together, a combination of α and β define a line. And so we could sample a bunch of lines from the posterior distribution. Then we could display those lines on the plot, to visualize the uncertainty in the regression relationship.

To better appreciate how the posterior distribution contains lines, extract some samples from the model:

```
post <- extract.samples( m4.3 )
```

R code
4.46

Then inspect the first 5 rows of the samples:

```
post[1:5,]
```

R code
4.47

```
      a      b  sigma
1 114.7880 0.8822921 5.121102
2 112.7115 0.9230855 4.907987
3 114.4557 0.9018482 5.276036
4 114.7696 0.8831561 5.021958
5 112.6333 0.9383632 4.898554
```

Each row is a correlated random sample from the joint posterior of all three parameters, using the covariances provided by `vcov(m4.3)`. The paired values of a and b on each row define a line. The average of very many of these lines is the MAP line. But the scatter around that average is meaningful, because it alters our confidence in the relationship between the predictor and the outcome.

So now let's display a bunch of these lines, so you can see the scatter. This lesson will be easier to appreciate, if we use only some of the data to begin. Then you can see how adding in more data changes the scatter of the lines. So we'll begin with just the first 10 cases in `d2`. The following code extracts the first 10 cases and re-estimates the model:

```

R code
4.48 N <- 10
      dN <- d2[ 1:N , ]
      mN <- map(
        alist(
          height ~ dnorm( mu , sigma ) ,
          mu <- a + b*weight ,
          a ~ dnorm( 178 , 100 ) ,
          b ~ dnorm( 0 , 10 ) ,
          sigma ~ dunif( 0 , 50 )
        ) , data=dN )

```

Now let's plot 20 of these lines, to see what the uncertainty looks like.

```

R code
4.49 # extract 20 samples from the posterior
      post <- extract.samples( mN , n=20 )

      # display raw data and sample size
      plot( dN$weight , dN$height ,
            xlim=range(d2$weight) , ylim=range(d2$height) ,
            col=rangi2 , xlab="weight" , ylab="height" )
      mtext(concat("N = ",N))

      # plot the lines, with transparency
      for ( i in 1:20 )
        abline( a=post$a[i] , b=post$b[i] , col=col.alpha("black",0.3) )

```

The last line loops over all 20 lines, using `abline` to display each.

The result is shown in the upper-left plot in [FIGURE 4.5](#). By plotting multiple regression lines, sampled from the posterior, it is easy to see both the highly confident aspects of the relationship and the less confident aspects. The cloud of regression lines displays greater uncertainty at extreme values for weight. This is very common.

The other plots in [FIGURE 4.5](#) show the same relationships, but for increasing amounts of data. Just re-use the code from before, but change `N <- 10` to some other value. Notice that the cloud of regression lines grows more compact as the sample size increases. This is a result of the model growing more confident about the location of the mean.

4.4.3.4. Plotting regression intervals and contours. The cloud of regression lines in [FIGURE 4.5](#) is an appealing display, because it communicates uncertainty about the relationship in a way that many people find intuitive. But it's much more common to see the uncertainty displayed by plotting an interval or contour around the MAP regression line. In this section, I'll walk you through how to compute any arbitrary interval you like, using the underlying cloud of regression lines embodied in the posterior distribution. Then we'll plot a shaded region around the MAP line, to display the interval.

Here's how to plot an interval around the regression line. This interval incorporates uncertainty in both the slope β and intercept α at the same time. To understand how it works, focus for the moment on a single weight value, say 50 kilograms. You can quickly make a list of 10,000 values of μ for an individual who weighs 50 kilograms, by using your samples from the posterior:

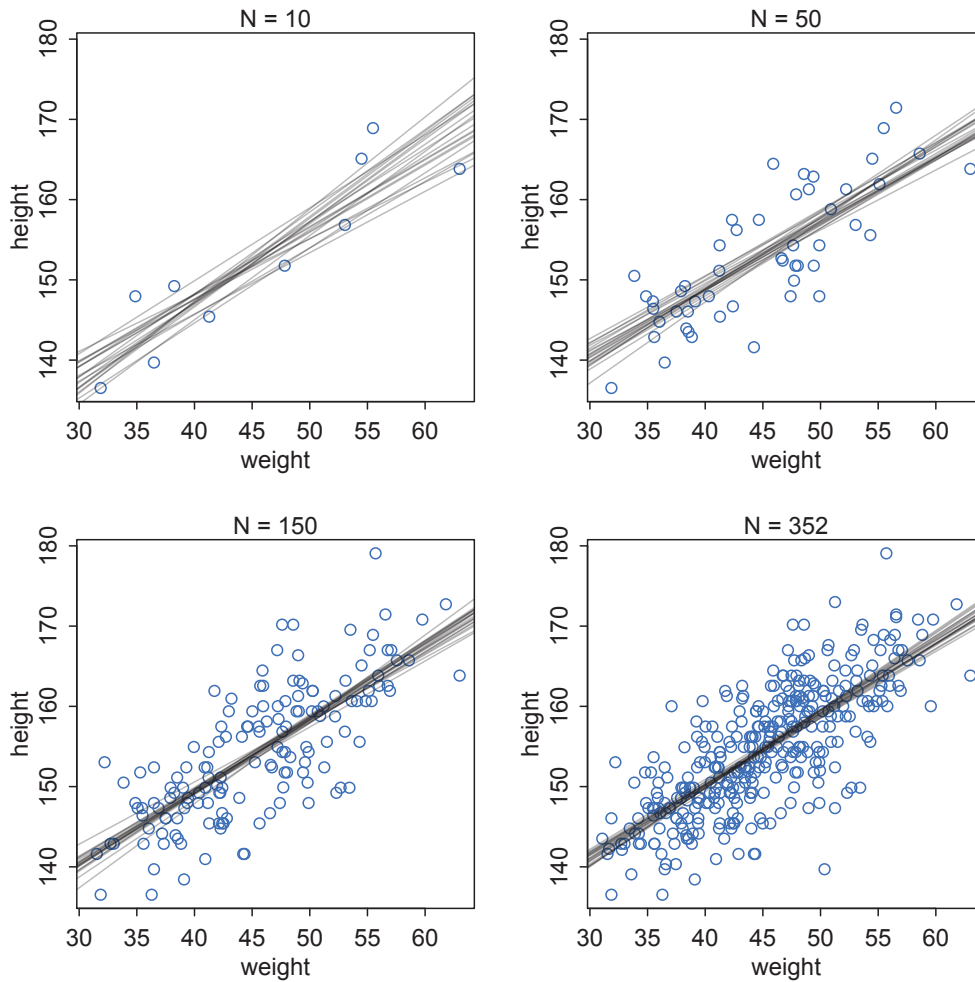


FIGURE 4.5. Samples from the quadratic approximate posterior distribution for the height/weight model, `m4.3`, with increasing amounts of data. In each plot, 20 lines sampled from the posterior distribution, showing the uncertainty in the regression relationship.

```
mu_at_50 <- post$a + post$b * 50
```

R code
4.50

The code to the right of the `<-` above takes its form from the equation for μ_i :

$$\mu_i = \alpha + \beta x_i$$

The value of x_i in this case is 50. Go ahead and take a look inside the result, `mu_at_50`. It's a vector of predicted means, one for each random sample from the posterior. Since joint `a` and `b` went into computing each, the variation across those means incorporates the uncertainty in and correlation between both parameters. It might be helpful at this point to actually plot the density for this vector of means:

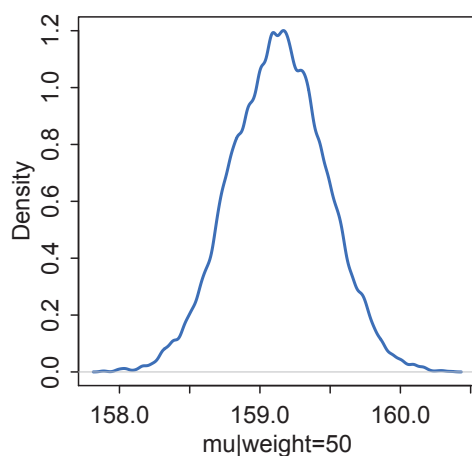


FIGURE 4.6. The quadratic approximate posterior distribution of the mean height, μ , when weight is 50 kg. This distribution represents the relative plausibility of different values of the mean.

```
R code 4.51 dens( mu_at_50 , col=rangl2 , lwd=2 , xlab="mu|weight=50" )
```

I reproduce this plot in [FIGURE 4.6](#). Since the components of μ have distributions, so too does μ . And since the distributions of α and β are Gaussian, so too is the distribution of μ (adding Gaussian distributions always produces a Gaussian distribution).

Since the posterior for μ is a distribution, you can find intervals for it, just like for any posterior distribution. To find the 89% highest posterior density interval of μ at 50 kg, just use the HPDI command as usual:

```
R code 4.52 HPDI( mu_at_50 , prob=0.89 )
```

```
      |0.89      0.89|
158.5642 159.6616
```

What these numbers mean is that the central 89% of the ways for the model to produce the data place the average height between about 159 cm and 160 cm (conditional on the model and data), assuming the weight is 50 kg.

That's good so far, but we need to repeat the above calculation for every weight value on the horizontal axis, not just when it is 50 kg. We want to draw 89% HPDIs around the MAP slope in [Figure 4.4](#).

This is made simple by strategic use of the `link` function, a part of the `rethinking` package. What `link` will do is take your map model fit, sample from the posterior distribution, and then compute μ for each case in the data and sample from the posterior distribution. Here's what it looks like for the data you used to fit the model:

```
R code 4.53 mu <- link( m4.3 )
str(mu)
```

```
num [1:1000, 1:352] 157 157 157 157 157 ...
```

You end up with a big matrix of values of μ . Each row is a sample from the posterior distribution. The default is 1000 samples, but you can use as many or as few as you like. Each column

is a case (row) in the data. There are 352 rows in `d2`, corresponding to 352 individuals. So there are 352 columns in the matrix `mu` above.

Now what can we do with this big matrix? Lots of things. The function `link` provides a posterior distribution of μ for each case we feed it. So above we have a distribution of μ for each individual in the original data. We actually want something slightly different: a distribution of μ for each unique weight value on the horizontal axis. It's only slightly harder to compute that, by just passing `link` some new data:

```
# define sequence of weights to compute predictions for
# these values will be on the horizontal axis
weight.seq <- seq( from=25 , to=70 , by=1 )

# use link to compute mu
# for each sample from posterior
# and for each weight in weight.seq
mu <- link( m4.3 , data=data.frame(weight=weight.seq) )
str(mu)
```

R code
4.54

```
num [1:1000, 1:46] 137 136 137 137 136 ...
```

And now there are only 46 columns in `mu`, because we fed it 46 different values for `weight`. To visualize what you've got here, let's plot the distribution of μ values at each height, on the plot.

```
# use type="n" to hide raw data
plot( height ~ weight , d2 , type="n" )

# loop over samples and plot each mu value
for ( i in 1:100 )
  points( weight.seq , mu[i,] , pch=16 , col=col.alpha(rangi2,0.1) )
```

R code
4.55

The result is shown on the left-hand side of [FIGURE 4.7](#). At each weight value in `weight.seq`, a pile of computed μ values are shown. Each of these piles is a Gaussian distribution, like that in [FIGURE 4.6](#). You can see now that the amount of uncertainty in μ depends upon the value of `weight`. And this is the same fact you saw in the right-hand plot in [FIGURE 4.5](#).

The final step is to summarize the distribution for each weight value. We'll use `apply`, which applies a function of your choice to a matrix.

```
# summarize the distribution of mu
mu.mean <- apply( mu , 2 , mean )
mu.HPDI <- apply( mu , 2 , HPDI , prob=0.89 )
```

R code
4.56

Read `apply(mu,2,mean)` as *compute the mean of each column (dimension “2”) of the matrix mu*. Now `mu.mean` contains the average μ at each weight value, and `mu.HPDI` contains 89% lower and upper bounds for each weight value. Be sure to take a look inside `mu.mean` and `mu.HPDI`, to demystify them. They are just different kinds of summaries of the distributions in `mu`, with each column being for a different weight value.

You can plot these summaries on top of the data with a few lines of R code:

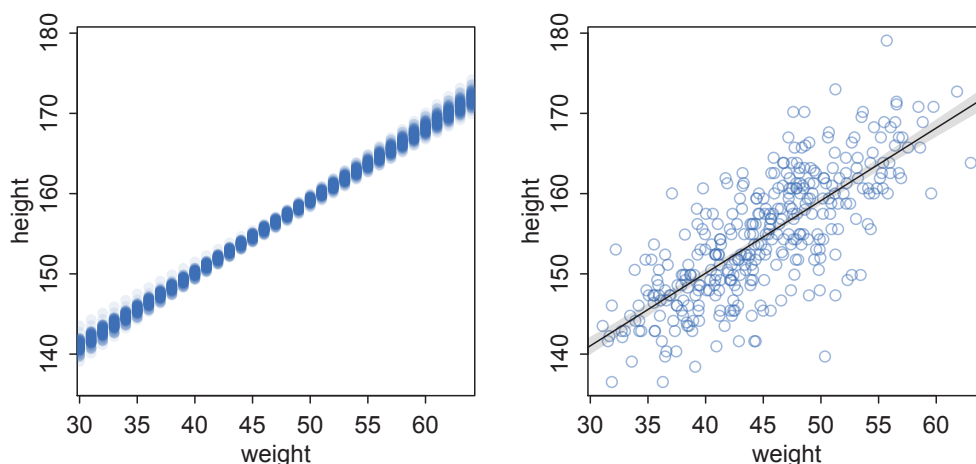


FIGURE 4.7. Left: The first 100 values in the distribution of μ at each weight value. Right: The !Kung height data again, now with 89% HPDI of the mean indicated by the shaded region. Compare this region to the distributions of blue points on the left.

R code
4.57

```
# plot raw data
# fading out points to make line and interval more visible
plot( height ~ weight , data=d2 , col=col.alpha(rangi2,0.5) )

# plot the MAP line, aka the mean mu for each weight
lines( weight.seq , mu.mean )

# plot a shaded region for 89% HPDI
shade( mu.HPDI , weight.seq )
```

You can see the results in the right-hand plot in [FIGURE 4.7](#).

Using this approach, you can derive and plot posterior prediction means and intervals for quite complicated models, for any data you choose. It's true that it is possible to use analytical formulas to compute intervals like this. I have tried teaching such an analytical approach before, and it has always been disaster. Part of the reason is probably my own failure as a teacher, but another part is that most social and natural scientists have never had much training in probability theory and tend to get very nervous around $\int$'s. I'm sure with enough effort, every one of them could learn to do the mathematics. But all of them can quickly learn to generate and summarize samples derived from the posterior distribution. So while the mathematics would be a more elegant approach, and there is some additional insight that comes from knowing the mathematics, the pseudo-empirical approach presented here is very flexible and allows a much broader audience of scientists to pull insight from their statistical modeling. And again, when you start estimating models with MCMC (Chapter 8), this is really the only approach available. So it's worth learning now.

To summarize, here's the recipe for generating predictions and intervals from the posterior of a fit model.

- (1) Use `link` to generate distributions of posterior values for μ . The default behavior of `link` is to use the original data, so you have to pass it a list of new horizontal axis values you want to plot posterior predictions across.
- (2) Use summary functions like `mean` or `HPDI` or `PI` to find averages and lower and upper bounds of μ for each value of the predictor variable.
- (3) Finally, use plotting functions like `lines` and `shade` to draw the lines and intervals. Or you might plot the distributions of the predictions, or do further numerical calculations with them. It's really up to you.

This recipe works for every model we fit in the book. As long as you know how parameters relate to the data, you can use samples from the posterior to describe any aspect of the model's behavior.

Rethinking: Overconfident confidence intervals. The confidence interval for the regression line in [FIGURE 4.7](#) clings tightly to the MAP line. Thus there is very little uncertainty about the average height as a function of average weight. But you have to keep in mind that these inferences are always conditional on the model. Even a very bad model can have very tight confidence intervals. It may help if you think of the regression line in [FIGURE 4.7](#) as saying: *Conditional on the assumption that height and weight are related by a straight line, then this is the most plausible line, and these are its plausible bounds.*

Overthinking: How `link` works. The function `link` is not really very sophisticated. All it is doing is using the formula you provided when you fit the model to compute the value of the linear model. It does this for each sample from the posterior distribution, for each case in the data. You could accomplish the same thing for any model, fit by any means, by performing these steps yourself. This is how it'd look for `m4.3`.

```
post <- extract.samples(m4.3)
mu.link <- function(weight) post$a + post$b*weight
weight.seq <- seq( from=25 , to=70 , by=1 )
mu <- sapply( weight.seq , mu.link )
mu.mean <- apply( mu , 2 , mean )
mu.HPDI <- apply( mu , 2 , HPDI , prob=0.89 )
```

R code
4.58

And the values in `mu.mean` and `mu.HPDI` should be very similar (allowing for simulation variance) to what you got the automated way, using `link`.

Knowing this manual method is useful both for (1) understanding and (2) sheer power. Whatever the model you find yourself with, this approach can be used to generate posterior predictions for any component of it. Automated tools like `link` save effort, but they are never as flexible as the code you can write yourself.

4.4.3.5. Prediction intervals. Now let's walk through generating an 89% prediction interval for actual heights, not just the average height, μ . This means we'll incorporate the standard deviation σ and its uncertainty as well. Remember, the statistical model here is (omitting priors for brevity):

$$h_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$

What you've done so far is just use samples from the posterior to visualize the uncertainty in μ_i , the linear model of the mean. But actual predictions of heights depend also upon the stochastic definition in the first line. The Gaussian distribution on the first line tells us that the model expects observed heights to be distributed around μ , not right on top of it. And the spread around μ is governed by σ . All of this suggests we need to incorporate σ in the predictions somehow.

Here's how you do it. Imagine simulating heights. For any unique weight value, you sample from a Gaussian distribution with the correct mean μ for that weight, using the correct value of σ sampled from the same posterior distribution. If you do this for every sample from the posterior, for every weight value of interest, you end up with a collection of simulated heights that embody the uncertainty in the posterior *as well as* the uncertainty in the Gaussian likelihood.

```
R code
4.59  sim.height <- sim( m4.3 , data=list(weight=weight.seq) )
      str(sim.height)
```

```
num [1:1000, 1:46] 139 144 141 140 130 ...
```

This matrix is much like the earlier one, `mu`, but it contains simulated heights, not distributions of plausible average height, μ .

We can summarize these simulated heights in the same way we summarized the distributions of μ , by using `apply`:

```
R code
4.60  height.PI <- apply( sim.height , 2 , PI , prob=0.89 )
```

Now `height.PI` contains the 89% posterior prediction interval of observable (according to the model) heights, across the values of weight in `weight.seq`.

Let's plot everything we've built up: (1) the MAP line, (2) the shaded region of 89% plausible μ , and (3) the boundaries of the simulated heights the model expects.

```
R code
4.61  # plot raw data
      plot( height ~ weight , d2 , col=col.alpha(rangi2,0.5) )

      # draw MAP line
      lines( weight.seq , mu.mean )

      # draw HPDI region for line
      shade( mu.HPDI , weight.seq )

      # draw PI region for simulated heights
      shade( height.PI , weight.seq )
```

The code above uses some objects computed in previous sections, so go back and execute that code, if you need to.

In [FIGURE 4.8](#), I plot the result. The wide shaded region in the figure represents the area within which the model expects to find 89% of actual heights in the population, at each weight. There is nothing special about the value 89% here. You could plot the boundary for other percents, such as 67% and 97% (also both primes), and add those to the plot. Doing so would help you see more of the shape of the predicted distribution of heights. I leave that as

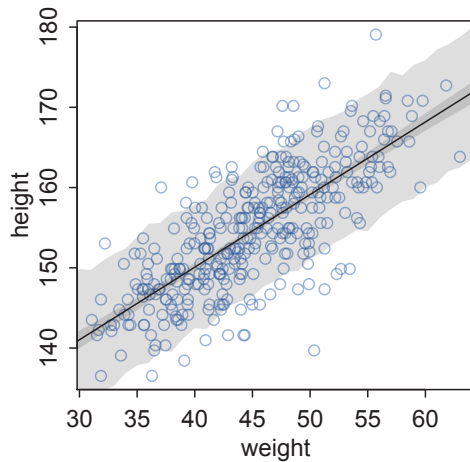


FIGURE 4.8. 89% prediction interval for height, as a function of weight. The solid line is the MAP estimate of the mean height at each weight. The two shaded regions show different 89% plausible regions. The narrow shaded interval around the line is the distribution of μ . The wider shaded region represents the region within which the model expects to find 89% of actual heights in the population, at each weight.

an exercise for the reader. Just go back to the code above and add `prob=0.67`, for example, to the call to `PI`. That will give you 67% intervals, instead of 89% ones.

Notice that the outline for the wide shaded interval is a little jagged. This is the simulation variance in the tails of the sampled Gaussian values. If it really bothers you, increase the number of samples you take from the posterior distribution. The optional `n` parameter for `sim.height` controls how many samples are used. Try for example:

```
sim.height <- sim( m4.3 , data=list(weight=weight.seq) , n=1e4 )
height.PI <- apply( sim.height , 2 , PI , prob=0.89 )
```

R code
4.62

Run the plotting code again, and you'll see the shaded boundary smooth out some. With extreme percentiles, it can be very hard to get out all of the jaggedness. Luckily, it hardly matters, except for aesthetics. Moreover, it serves to remind us that all statistical inference is approximate. The fact that we can compute an expected value to the 10th decimal place does not imply that our inferences are precise to the 10th decimal place.

Rethinking: Two kinds of uncertainty. In the procedure above, we encountered both uncertainty in parameter values and uncertainty in a sampling process. These are distinct concepts, even though they are processed much the same way and end up blended together in the posterior predictive simulation. The posterior distribution is a ranking of the relative plausibilities of every possible combination of parameter values. The distribution of simulated outcomes, like height, is instead a distribution that includes sampling variation from some process that generates Gaussian random variables. This sampling variation is still a model assumption. It's no more or less objective than the posterior distribution. Both kinds of uncertainty matter, at least sometimes. But it's important to keep them straight, because they depend upon different model assumptions. Furthermore, it's possible to view the Gaussian likelihood as a purely epistemological assumption (a device for estimating the mean and variance of a variable), rather than an ontological assumption about what future data will look like. In that case, it may not make complete sense to simulate outcomes.

Overthinking: Rolling your own `sim`. Just like with `link`, it's useful to know a little about how `sim` operates. For every distribution like `dnorm`, there is a companion simulation function. For the Gaussian distribution, the companion is `rnorm`, and it simulates sampling from a Gaussian distribution. What we want R to do is simulate a height for each set of samples, and to do this for each value of `weight`. The following will do it:

R code
4.63

```
post <- extract.samples(m4.3)
weight.seq <- 25:70
sim.height <- sapply( weight.seq , function(weight)
  rnorm(
    n=nrow(post) ,
    mean=post$a + post$b*weight ,
    sd=post$sigma ) )
height.PI <- apply( sim.height , 2 , PI , prob=0.89 )
```

The values in `height.PI` will be practically identical to the ones computed in the main text and displayed in [FIGURE 4.8](#).

4.5. Polynomial regression

In the next chapter, you'll see how to use linear models to build regressions with more than one predictor variable. But before then, it helps to see how to model the outcome as a curved function of a single predictor. The models so far all assume that a straight line describes the relationship. But there's nothing special about straight lines, aside from their simplicity.

Let's work through an example, using the full `!Kung` data:

R code
4.64

```
library(rethinking)
data(Howell1)
d <- Howell1
str(d)
```

```
'data.frame': 544 obs. of 4 variables:
 $ height: num 152 140 137 157 145 ...
 $ weight: num 47.8 36.5 31.9 53 41.3 ...
 $ age : num 63 63 65 41 51 35 32 27 19 54 ...
 $ male : int 1 0 0 1 0 1 0 1 0 1 ...
```

Go ahead and plot `height` against `weight`. The relationship is visibly curved, now that we've included the non-adult individuals.

There are many ways to model a curved relationship between two variables. Here, I'll show you a very common one, **POLYNOMIAL REGRESSION**. In this context, "polynomial" means equations for μ_i that add additional terms with squares, cubes, and even higher powers of the predictor variable. There's still only one predictor variable in the model, so this is still a bivariate regression. But the definition of μ_i has more parameters now.

While this section teaches polynomial regression, in general it's a bad thing to do. Why? Because polynomials are very hard to interpret. Better would be to have a more mechanistic model of the data, one that builds the non-linear relationship up from a principled beginning. In the practice problems for this chapter, you'll see how to do this with the height data.

But it's still worth working through a polynomial example, both because it is very common and it will expose some general issues with modeling that we'll take up in Chapter 6. So here's the most common polynomial regression, a parabolic model of the mean:

$$\mu_i = \alpha + \beta_1 x_i + \beta_2 x_i^2$$

The above is a parabolic (second order) polynomial. The $\alpha + \beta_1 x_i$ part is the same linear function of x in a linear regression, just with a little “1” subscript added to the parameter name, so we can tell it apart from the new parameter. The additional term uses the square of x_i to construct a parabola, rather than a perfectly straight line. The new parameter β_2 measures the curvature of the relationship.

Rethinking: Linear, additive, funky. The parabolic model of μ_i above is still called a “linear model” of the mean. This is so, even though the equation is clearly not of a straight line. Unfortunately, the word “linear” means different things in different contexts, and different people use it differently in the same context. What “linear” usually means in this context is that μ_i is a *linear function* of any single parameter. Such models have the advantage of being easier to fit to data. They are also often easier to interpret, because they assume that parameters act independently on the mean. They have the disadvantage of being strongly conventional. They are often used thoughtlessly. When you have real knowledge of your study system, it is often easy to do better than a linear model. These models are geocentric engines, devices for describing partial correlations among variables. We should feel embarrassed to use them, just so we don't become satisfied with the phenomenological explanations they provide.

Fitting these models to data is easy. Interpreting them can be hard. We'll begin with the easy part, fitting a parabolic model of height on weight. The first thing to do is to **STANDARDIZE** the predictor variable. This means to first center the variable and then divide it by its standard deviation. Why do this? You already have some sense of the value of centering. Going further to standardize leaves the mean at zero but also rescales the range of the data. This is helpful for two reasons:

- (1) Interpretation might be easier. For a standardized variable, a change of one unit is equivalent to a change of one standard deviation. In many contexts, this is more interesting and more revealing than a one unit change on the natural scale. And once you start making regressions with more than one kind of predictor variable, standardizing all of them makes it easier to compare their relative influence on the outcome, using only estimates. On the other hand, you might want to interpret the data on the natural scale. So standardization can make interpretation harder, not easier. With a little practice, though, you can learn to quickly convert back and forth between natural and standard scales.
- (2) More important though are the advantages for fitting the model to the data. When predictor variables have very large values in them, there are sometimes numerical glitches. Even well-known statistical software can suffer from these glitches, leading to mistaken estimates. These problems are very common for polynomial regression, because the square or cube of a large number can be truly massive. Standardizing largely resolves this issue.

To standardize `weight`, all you do is subtract the mean and then divide by the standard deviation. This will do it:

```
R code
4.65 d$weight.s <- ( d$weight - mean(d$weight) )/sd(d$weight)
```

This new variable `weight.s` has mean zero and standard deviation 1. No information has been lost in this procedure. Go ahead and plot `height` on `weight.s` to verify that. You'll see the same curved relationship as before, but now with a different range on the horizontal axis.

To fit the parabolic model, just modify the definition of μ_i . Here's the model (with very weak priors):

```

 $h_i \sim \text{Normal}(\mu_i, \sigma)$                                 height ~ dnorm(mu,sigma)
 $\mu_i = \alpha + \beta_1 x_i + \beta_2 x_i^2$                 mu <- a + b1*weight.s + b2*weight.s^2
 $\alpha \sim \text{Normal}(178, 100)$                         a ~ dnorm(140,100)
 $\beta_1 \sim \text{Normal}(0, 10)$                             b1 ~ dnorm(0,10)
 $\beta_2 \sim \text{Normal}(0, 10)$                             b2 ~ dnorm(0,10)
 $\sigma \sim \text{Uniform}(0, 50)$                         sigma ~ dunif(0,50)
```

And fitting is straightforward, as well. Just modify the definition of `mu` so that it contains both the ordinary and quadratic terms. But in general it is better to pre-process any variable transformations. So I'll also build the square of `weight.s` as a separate variable:

```
R code
4.66 d$weight.s2 <- d$weight.s^2
m4.5 <- map(
  alist(
    height ~ dnorm( mu , sigma ) ,
    mu <- a + b1*weight.s + b2*weight.s2 ,
    a ~ dnorm( 178 , 100 ) ,
    b1 ~ dnorm( 0 , 10 ) ,
    b2 ~ dnorm( 0 , 10 ) ,
    sigma ~ dunif( 0 , 50 )
  ) ,
  data=d )
```

Interpreting the estimates from the summary table can be hard, though. Let's take a look:

```
R code
4.67 precis( m4.5 )
```

	Mean	StdDev	5.5%	94.5%
a	146.66	0.37	146.07	147.26
b1	21.40	0.29	20.94	21.86
b2	-8.42	0.28	-8.87	-7.97
sigma	5.75	0.17	5.47	6.03

The parameter α (a) is still the intercept, so it tells us the expected value of `height` when `weight.s` is zero. But it is no longer equal to the mean height in the sample, since there is no guarantee it should in a polynomial regression.⁷⁰ And those β_1 and β_2 parameters are the linear and square components of the curve, respectively. But that doesn't make them transparent.

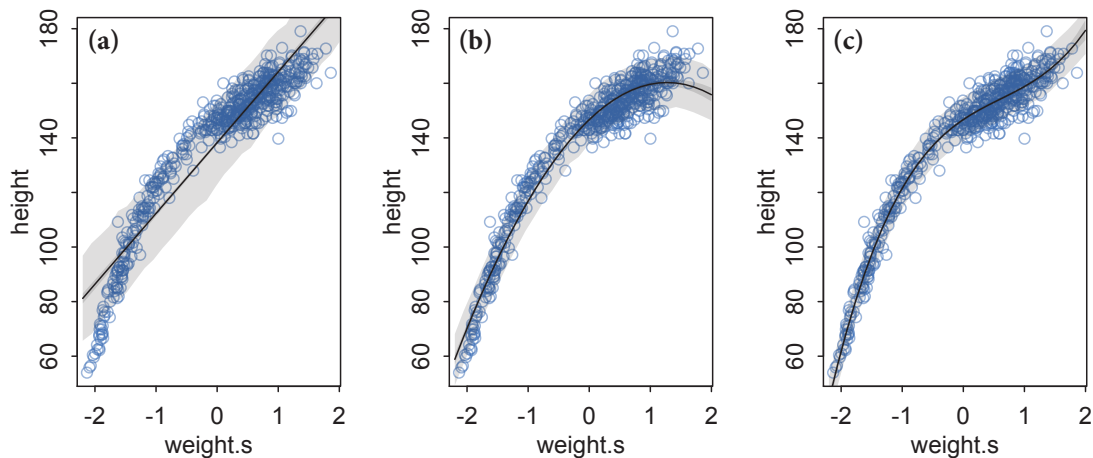


FIGURE 4.9. Polynomial regressions of height on weight (standardized), for the full !Kung data. In each plot, the raw data are shown by the circles. The solid curves show the path of μ in each model, and the shaded regions show the 89% interval of the mean (close to the solid curve) and the 89% interval of predictions (wider). (a) Linear regression. (b) A second order polynomial, a parabolic regression. (c) A third order polynomial, a cubic regression.

You have to plot these model fits to understand what they are saying. So let's do that. We'll calculate the mean relationship and the 89% intervals of the mean and the predictions, like in the previous section. Here's the working code:

```
weight.seq <- seq( from=-2.2 , to=2 , length.out=30 )
pred_dat <- list( weight.s=weight.seq , weight.s2=weight.seq^2 )
mu <- link( m4.5 , data=pred_dat )
mu.mean <- apply( mu , 2 , mean )
mu.PI <- apply( mu , 2 , PI , prob=0.89 )
sim.height <- sim( m4.5 , data=pred_dat )
height.PI <- apply( sim.height , 2 , PI , prob=0.89 )
```

R code
4.68

Plotting all of this is straightforward:

```
plot( height ~ weight.s , d , col=col.alpha(rangi2,0.5) )
lines( weight.seq , mu.mean )
shade( mu.PI , weight.seq )
shade( height.PI , weight.seq )
```

R code
4.69

The results are shown in [FIGURE 4.9](#). Panel (a) of the figure shows the familiar linear regression from earlier in the chapter, but now with the standardized predictor and full data with both adults and non-adults. The linear model makes some spectacularly poor predictions, at both very low and middle weights. Compare this to panel (b), our new parabolic regression. The curve does a much better job of finding a central path through the data.

Panel (c) in [FIGURE 4.9](#) shows a higher-order polynomial regression, a cubic regression on weight. The model is (hiding the priors, but they are the same as before):

$$h_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3$$

Fit the model with a slight modification of the parabolic model's code:

```
R code
4.70 d$weight.s3 <- d$weight.s^3
      m4.6 <- map(
        alist(
          height ~ dnorm( mu , sigma ) ,
          mu <- a + b1*weight.s + b2*weight.s2 + b3*weight.s3 ,
          a ~ dnorm( 178 , 100 ) ,
          b1 ~ dnorm( 0 , 10 ) ,
          b2 ~ dnorm( 0 , 10 ) ,
          b3 ~ dnorm( 0 , 10 ) ,
          sigma ~ dunif( 0 , 50 )
        ) ,
        data=d )
```

Computing the curve and intervals is similarly a small modification of the previous code. This cubic curve is even more flexible than the parabola, so it fits the data even better.

But it's not clear that any of these models make a lot of sense. They are good geocentric descriptions of the sample, yes. But later in the book (Chapter 6), you'll see how a better fit might not actually be a better model. For the moment, consider that a pregnant promise. The same issue arises in regression models with more than one type of predictor. That's the subject of the next chapter.

Overthinking: Converting back to natural scale. The plots in [FIGURE 4.9](#) have standard units on the horizontal axis. These units are sometimes called *z-scores*. But suppose you fit the model using standardized variables, but want to plot the estimates on the original scale. All that's really needed is first to turn off the horizontal axis when you plot the raw data:

```
R code
4.71 plot( height ~ weight.s , d , col=col.alpha(rangi2,0.5) , xaxt="n" )
```

The `xaxt` at the end there turns off the horizontal axis. Then you explicitly construct the axis, using the `axis` function.

```
R code
4.72 at <- c(-2,-1,0,1,2)
      labels <- at*sd(d$weight) + mean(d$weight)
      axis( side=1 , at=at , labels=round(labels,1) )
```

The first line above defines the location of the labels, in standardized units. The second line then takes those units and converts them back to the original scale. The third line draws the axis. Take a look at the help `?axis` for more details.

4.6. Summary

This chapter introduced the simple linear regression model, a framework for estimating the association between a predictor variable and an outcome variable. The Gaussian distribution comprises the likelihood in such models, because it counts up the relative numbers of ways different combinations of means and standard deviations can produce an observation. To fit these models to data, the chapter introduced *maximum a prior* (MAP) estimation. It also introduced new procedures for visualizing posterior distributions and posterior predictions. The next chapter expands on these concepts by introducing regression models with more than one predictor variable.

4.7. Practice

Easy.

4E1. In the model definition below, which line is the likelihood?

$$\begin{aligned}y_i &\sim \text{Normal}(\mu, \sigma) \\ \mu &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Uniform}(0, 10)\end{aligned}$$

4E2. In the model definition just above, how many parameters are in the posterior distribution?

4E3. Using the model definition above, write down the appropriate form of Bayes' theorem that includes the proper likelihood and priors.

4E4. In the model definition below, which line is the linear model?

$$\begin{aligned}y_i &\sim \text{Normal}(\mu, \sigma) \\ \mu_i &= \alpha + \beta x_i \\ \alpha &\sim \text{Normal}(0, 10) \\ \beta &\sim \text{Normal}(0, 1) \\ \sigma &\sim \text{Uniform}(0, 10)\end{aligned}$$

4E5. In the model definition just above, how many parameters are in the posterior distribution?

Medium.

4M1. For the model definition below, simulate observed heights from the prior (not the posterior).

$$\begin{aligned}y_i &\sim \text{Normal}(\mu, \sigma) \\ \mu &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Uniform}(0, 10)\end{aligned}$$

4M2. Translate the model just above into a map formula.

4M3. Translate the map model formula below into a mathematical model definition.

```
flist <- alist(
  y ~ dnorm( mu , sigma ),
  mu <- a + b*x,
  a ~ dnorm( 0 , 50 ),
  b ~ dunif( 0 , 10 ),
  sigma ~ dunif( 0 , 50 )
)
```

4M4. A sample of students is measured for height each year for 3 years. After the third year, you want to fit a linear regression predicting height using year as a predictor. Write down the mathematical model definition for this regression, using any variable names and priors you choose. Be prepared to defend your choice of priors.

4M5. Now suppose I tell you that the average height in the first year was 120 cm and that every student got taller each year. Does this information lead you to change your choice of priors? How?

4M6. Now suppose I tell you that the variance among heights for students of the same age is never more than 64cm. How does this lead you to revise your priors?

Hard.

4H1. The weights listed below were recorded in the !Kung census, but heights were not recorded for these individuals. Provide predicted heights and 89% intervals (either HPDI or PI) for each of these individuals. That is, fill in the table below, using model-based predictions.

Individual	weight	expected height	89% interval
1	46.95		
2	43.72		
3	64.78		
4	32.59		
5	54.63		

4H2. Select out all the rows in the `Howell1` data with ages below 18 years of age. If you do it right, you should end up with a new data frame with 192 rows in it.

(a) Fit a linear regression to these data, using `map`. Present and interpret the estimates. For every 10 units of increase in weight, how much taller does the model predict a child gets?

(b) Plot the raw data, with height on the vertical axis and weight on the horizontal axis. Superimpose the MAP regression line and 89% HPDI for the mean. Also superimpose the 89% HPDI for predicted heights.

(c) What aspects of the model fit concern you? Describe the kinds of assumptions you would change, if any, to improve the model. You don't have to write any new code. Just explain what the model appears to be doing a bad job of, and what you hypothesize would be a better model.

4H3. Suppose a colleague of yours, who works on allometry, glances at the practice problems just above. Your colleague exclaims, "That's silly. Everyone knows that it's only the *logarithm* of body weight that scales with height!" Let's take your colleague's advice and see what happens.

(a) Model the relationship between height (cm) and the natural logarithm of weight (log-kg). Use the entire `Howell1` data frame, all 544 rows, adults and non-adults. Fit this model, using quadratic approximation:

$$\begin{aligned}
 h_i &\sim \text{Normal}(\mu_i, \sigma) \\
 \mu_i &= \alpha + \beta \log(w_i) \\
 \alpha &\sim \text{Normal}(178, 100) \\
 \beta &\sim \text{Normal}(0, 100) \\
 \sigma &\sim \text{Uniform}(0, 50)
 \end{aligned}$$

where h_i is the height of individual i and w_i is the weight (in kg) of individual i . The function for computing a natural log in R is just `log`. Can you interpret the resulting estimates?

(b) Begin with this plot:

```
plot( height ~ weight , data=Howell1 ,  
      col=col.alpha(rangi2,0.4) )
```

R code
4.73

Then use samples from the quadratic approximate posterior of the model in (a) to superimpose on the plot: (1) the predicted mean height as a function of weight, (2) the 97% HPDI for the mean, and (3) the 97% HPDI for predicted heights.

